

Einführung in Kubernetes

Tag 2

Ihr Veranstalter

IT-Schulungen.com

Technology & Transformation

Ihr Referent

Ben
Cöppicus

NETZWERKVERBINDUNGEN BEREITSTELLEN

Netzwerke sind ein Grundstein jedes IT Systems, sowie auch von Kubernetes. Wie jedoch stellt man eine sichere Verwendung dessen bereit?

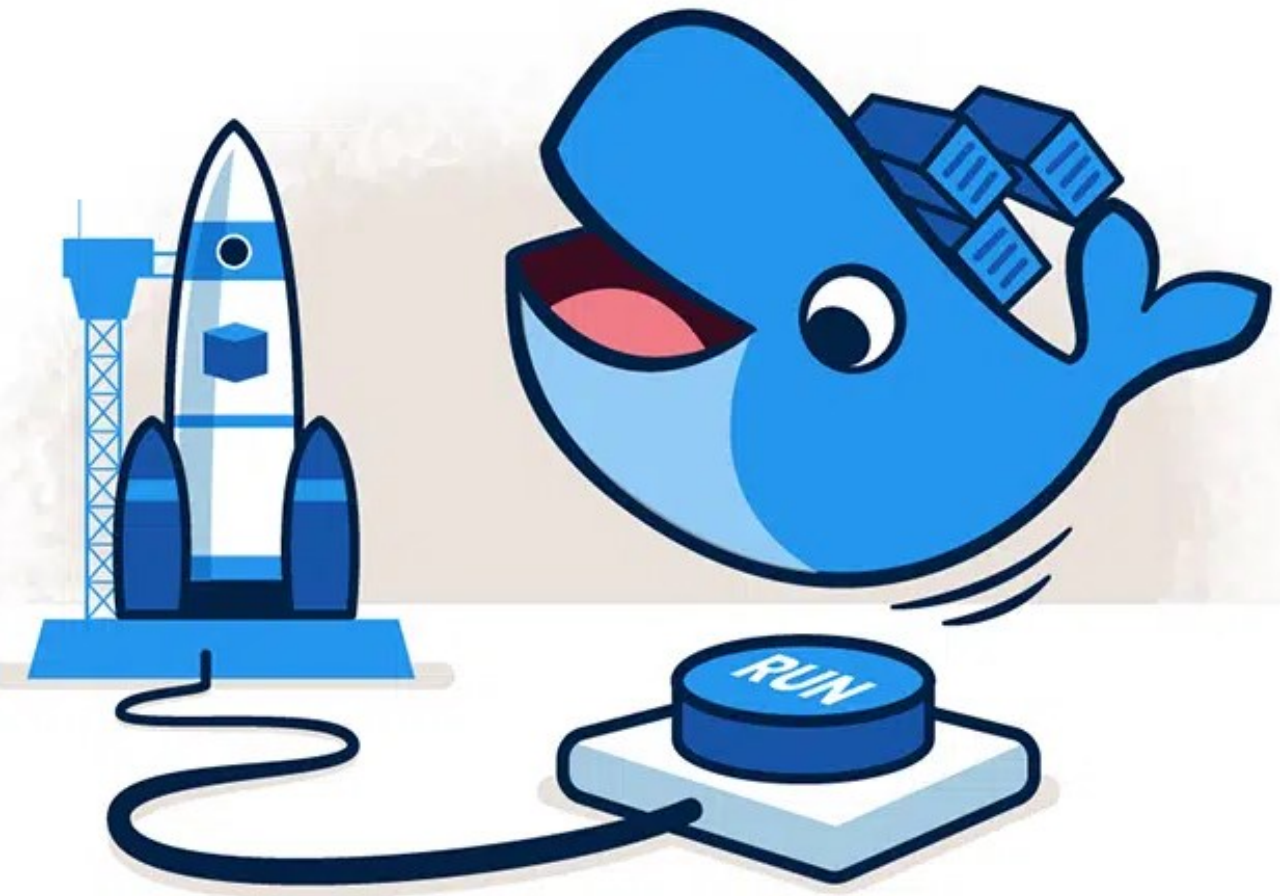
STEUERUNG, ÜBERWACHUNG UND KONTROLLE VON ANWENDUNGEN

Vertrauen ist gut Kontrolle ist aber immer besser, in diesem Kapitel geht es vollständig um die Überwachung und Steuerung Kubernetes.

KOMPLEXE ANWENDUNGEN EINFACH DEPLOYEN: DER HELM PAKET MANAGER

Was ist Helm, wie funktioniert Helm und wieso spielt Helm so eine große Rolle





KOMPLEXE STATEFUL ANWENDUNGEN (DATENBANK) DEPLOYEN

Was sind komplexe Anwendungen, wieso sind diese Stateful und warum sind diese so ein besonderer Typ den wir beachten müssen

TROUBLESHOOTING

Es können immer Fehler und oder Probleme entstehen in diesem Kapitell werden wir einen genauen Blick darauf werfen wie man diese Fehler beheben kann.

ZUGRIFFSKONTROLLE

Es sollte nie jemand alle Rechte an einem System haben, und wir als Betreiber eines Clusters sollten sicher gehen das nur jeder, Service, Benutzer oder Pod nur die Rechte und Zugriff bekommt die er benötigt.

CLUSTER ERWEITERUNGEN

Wie kann man Cluster erweitern, und welche Möglichkeiten bestehen hierzu.

NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

Wie holt man das meiste aus den bestehenden Netzwerken raus, wie sorgt man dafür das die Entwicklung so gut und Leistungsstark eingesetzt werden kann wie möglich.

CNCF, AUSBLICK

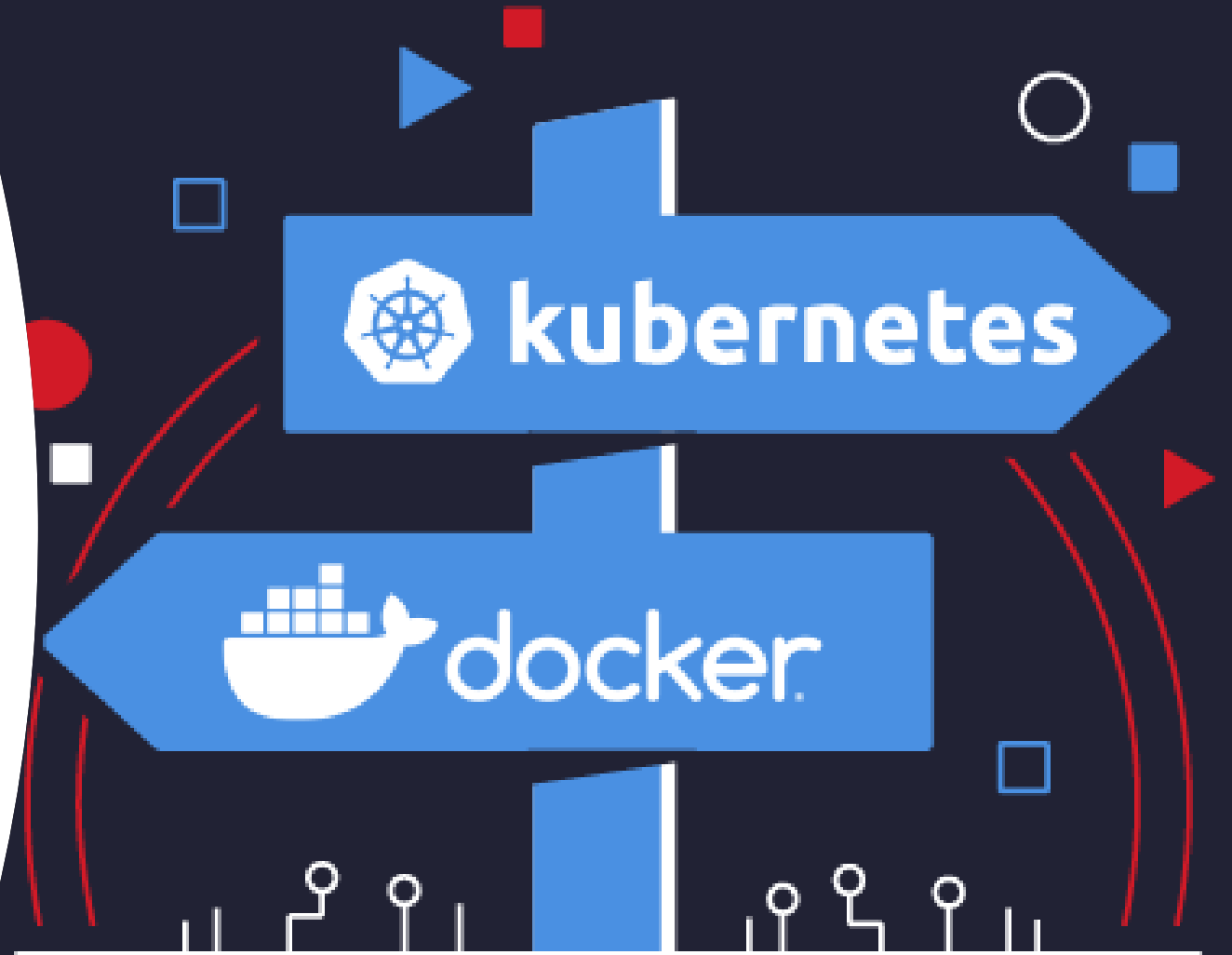
Zum Schluss klären wir Fragen und Ausblicke auf die Zukunft von Kubernetes und dessen Einsatz in der Industrie.



NETZWERKVERBINDUNGEN BEREITSTELLEN

Architektur des Kubernetes

Netzwerks





NETZWERKE IN KUBERNETES VERSTEHEN

Netzwerke sind ein essentieller Teil jeder Anwendung Orchestrierung und müssen auf eine Art gepflegt sein die es den Entwicklern sowie Benutzern erlaubt sicher und verständnisvoll damit umzugehen.

Im folgenden werden die einzelnen Bestandteile dieser Netzwerke und wie deren Technologie funktioniert aufgelistet und näher beleuchtet.

THEORIE

Als erstes wird in kurzen Zügen die Theorie hinter dieser Funktionalität angeschaut und darauf eingegangen wie genau die Netzwerke in Kubernetes aufgebaut sind.

VERANSCHAULICHUNG

Danach folgt eine visuelle Veranschaulichung in dem die Theorie versucht wird darzustellen und genauer zu betrachten. Versuchen Sie hier selber ihr Verständnis zu verdeutlichen.

FRAGEN

Ganz am Ende können Sie Fragen zu dieser Thematik stellen und ich kann mir die Zeit nehmen diese genauer zu erklären.

Cluster Networking

Networking is a central part of Kubernetes, but it can be challenging to understand exactly how it is expected to work. There are 4 distinct networking problems to address:

01.

PROBLEM 1

Eng vernetzte Kommunikation zwischen Containern und anderen Containern. Dies kann behoben werden indem Pods sowie die Container in Ihnen über den localhost kommunizieren.

02.

PROBLEM 2

Die Kommunikation zwischen Pods zu anderen Pods. Eines der größte Themen im Bereich Kubernetes und der Fokus für diesen Abschnitt.

03.

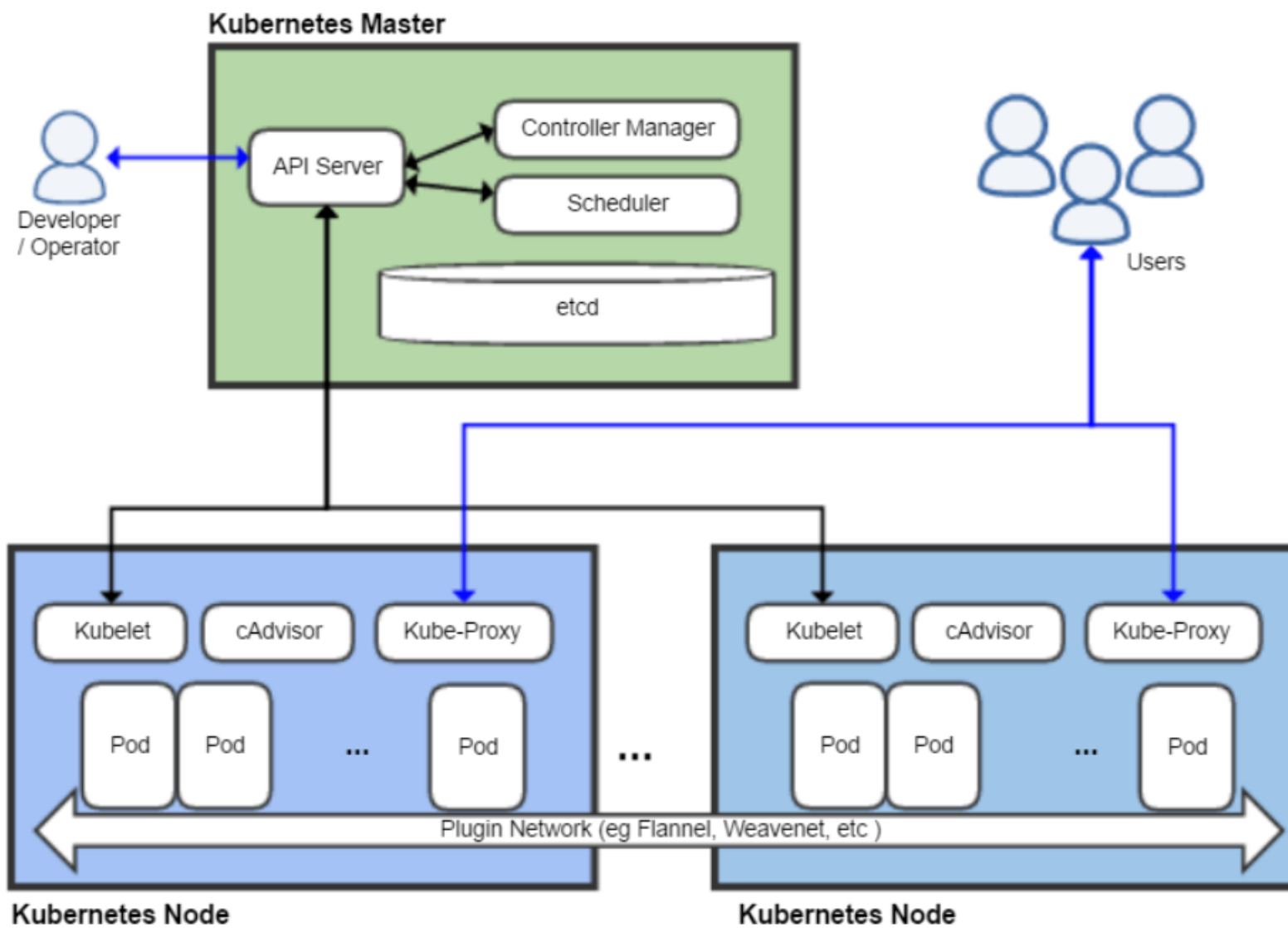
PROBLEM 3

Die Kommunikation zwischen Pods und Services, seinen dies nun Kubernetes Services und oder Services die dazu geschaltet werden müssen. Dies kann über die Kubernetes eigene Service Funktionalität behoben werden.

04.

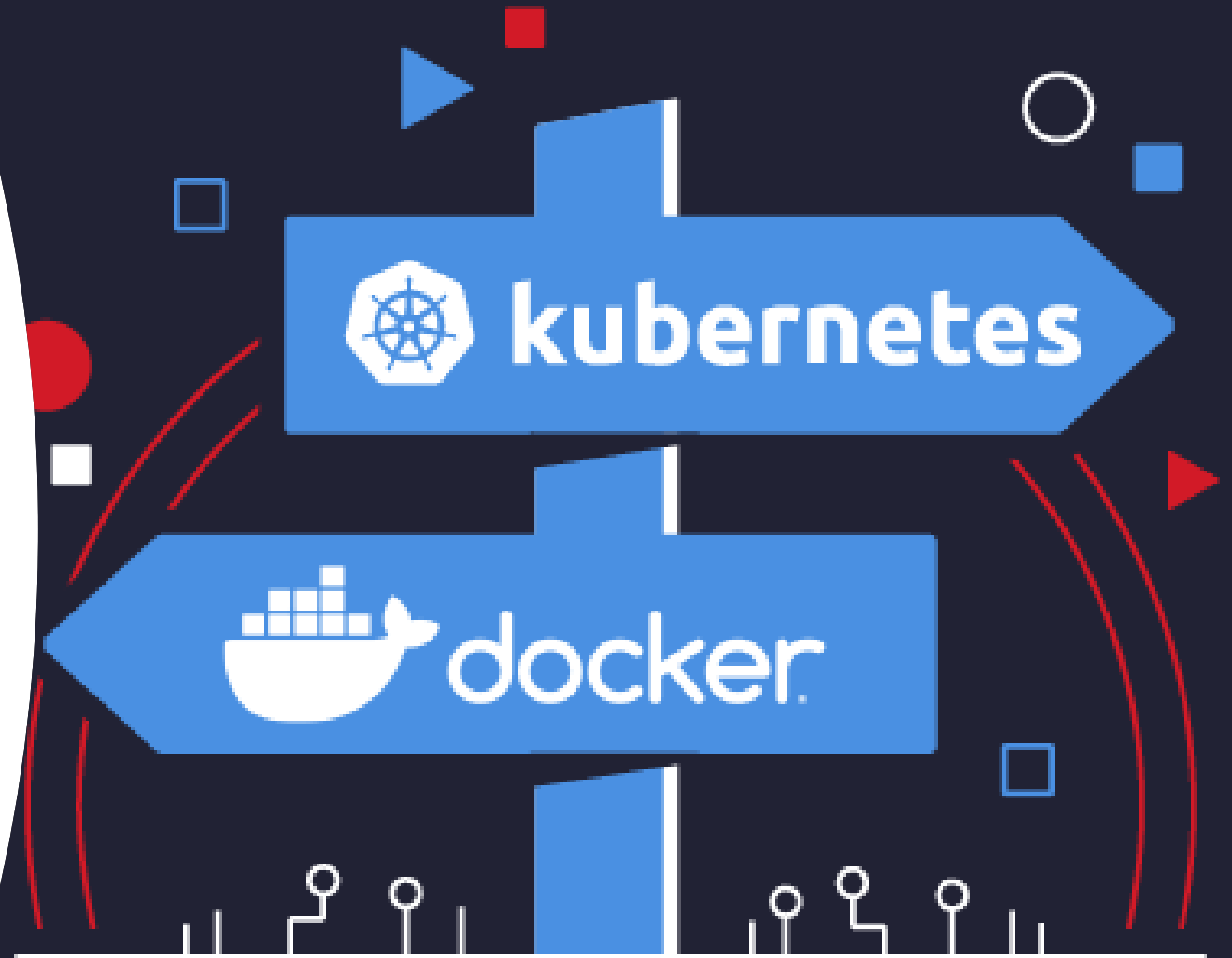
PROBLEM 4

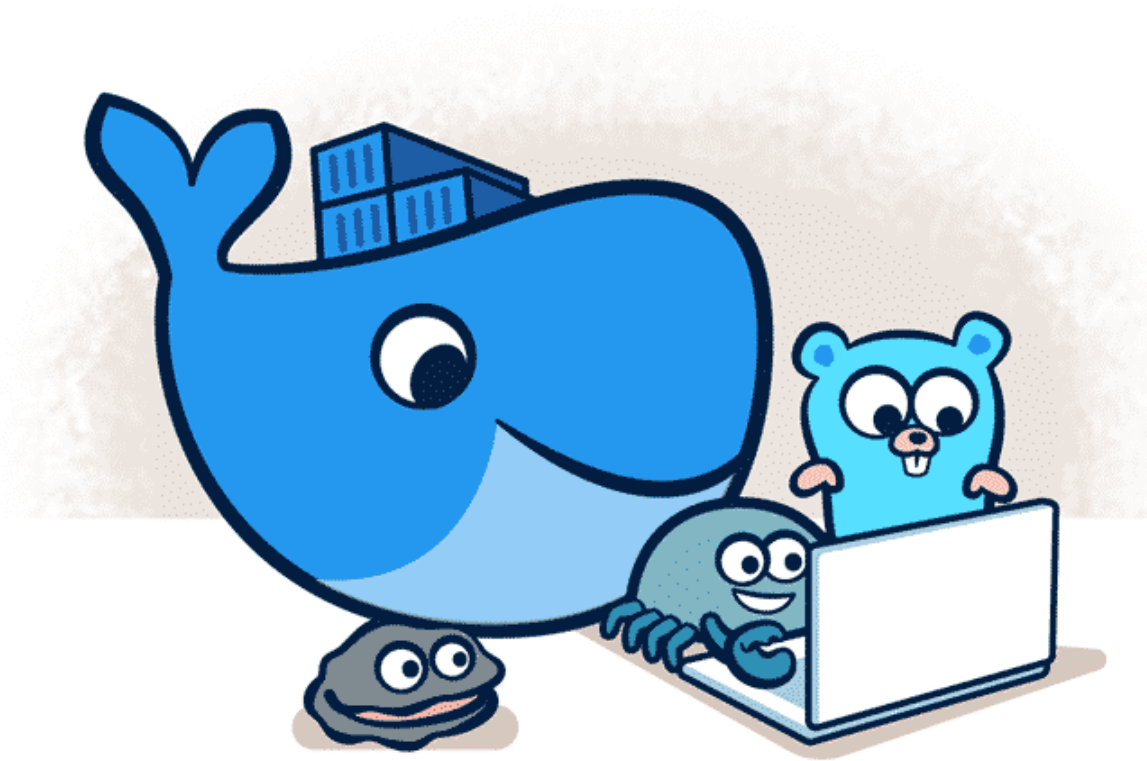
Das letzte große Problem ist die Kommunikation zwischen Services und Externen Quellen. Dies wird auch über Kubernetes Services abgedeckt. Ist jedoch immer stets ein wichtiges Problem.



NETZWERKVERBINDUNGEN BEREITSTELLEN

Verbindungen zwischen Containern,
Verbindungen nach außen





WIESO IST DIES WICHTIG?

In manchen Anwendungsfällen kann es relevant sein das Container unter einander mit einander kommunizieren. Dies kann dafür sein um Ressourcen und oder Informationen auszutauschen, aber auch einfach nur aus dem Grund um auf andere gleiche Ressourcen geordnet zuzugreifen.

In den nächsten Folien wird näher auf die verschiedenen Möglichkeiten eingegangen wie Container innerhalb Kubernetes mit anderen Containern kommunizieren können.

LOCALHOST

01.

Um mit Containern die im gleichen Pod liegen zu kommunizieren ist es nur nötig die Port Nummer in localhost einzugeben.

IP ADDRESS

02.

Falls der Container mit dem man kommunizieren möchte in einem anderen Pod liegt ist es möglich per IP Adresse und Port mit diesem Container zu kommunizieren.

SERVICE

03.

Die letzte Möglichkeit mit einem anderen Container zu kommunizieren ist mittels eines Services. Dies kann man als zwischenstelle Betrachten.

```

1 import 'package:flutter/material.dart';
2
3 void main() => runApp(new MyApp());
4
5 class MyApp extends StatelessWidget {
6   @override
7   Widget build(BuildContext context) {
8     return new MaterialApp(
9       title: 'Welcome to Flutter',
10      home: new Scaffold(
11        appBar: new AppBar(
12          title: 'Flutter Demo Home Page',
13        ),
14        body: new Center(
15          child: new Text('Hello World!'),
16        ),
17      ),
18    );
19  }
20 }

```

package:flutter/src/material/animated_icons.dart: error: line 9 pos 1: Library handler failed
import 'dart:ui' as ui show Paint, Path, Canvas;

[Done] exited with code=254 in 0.152 seconds

[Running] dart "/Users/lizaiyang/CodeProject/flutter-project/myapp/lib/main.dart"
The built-in library 'dart:ui' is not available on the stand-alone VM.
'package:flutter/src/material/animated_icons.dart': error: line 9 pos 1: Library handler failed
import 'dart:ui' as ui show Paint, Path, Canvas;

[Done] exited with code=254 in 0.207 seconds

[Running] dart "/Users/lizaiyang/CodeProject/flutter-project/myapp/lib/main.dart"
The built-in library 'dart:ui' is not available on the stand-alone VM.
'package:flutter/src/material/animated_icons.dart': error: line 9 pos 1: Library handler failed
import 'dart:ui' as ui show Paint, Path, Canvas;

```

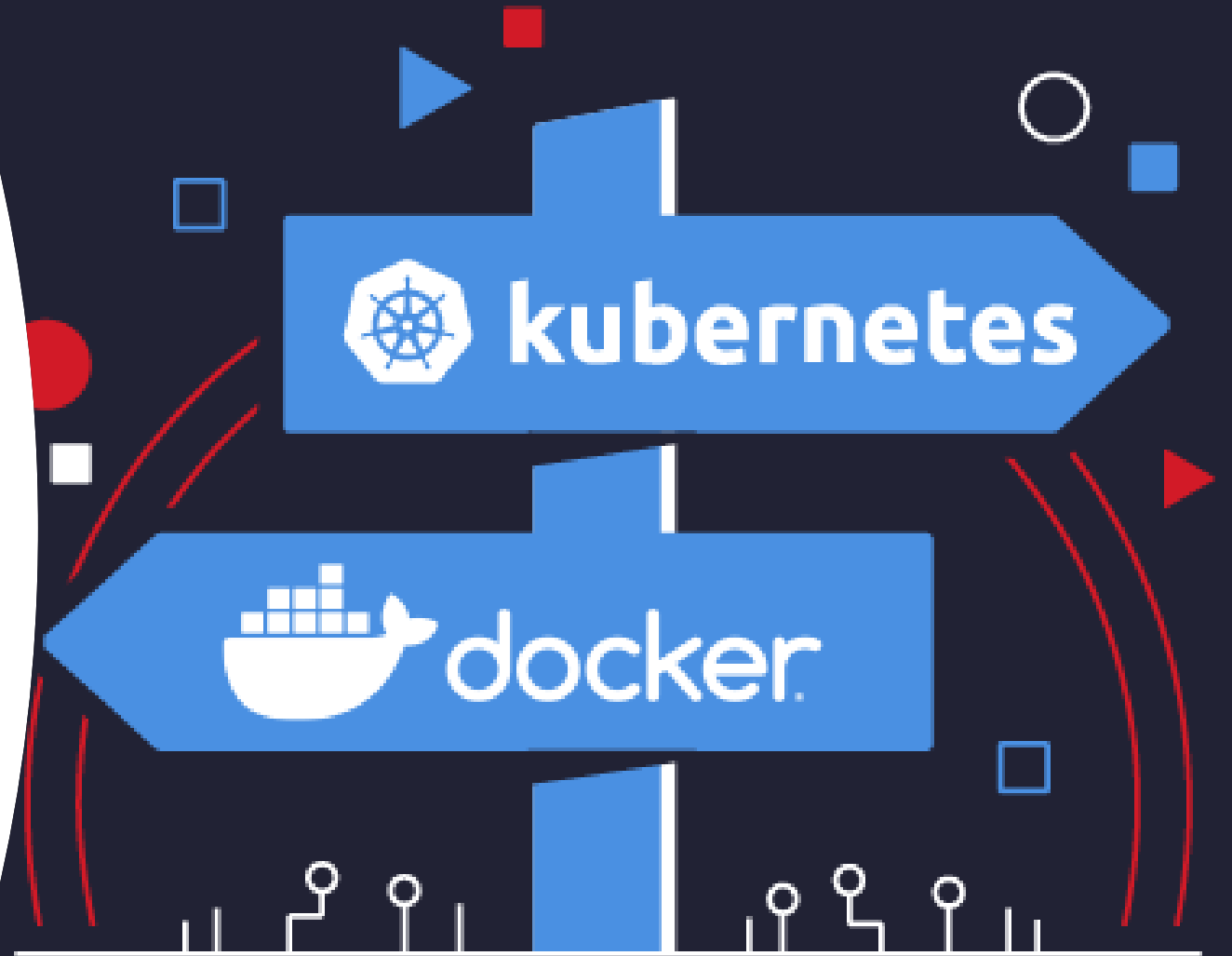
SuffixOrigin : WellKnown
AddressState : Preferred
ValidLifetime : Infinite ([TimeSpan]::Max)
PreferredLifetime : Infinite ([TimeSpan]::Max)
SkipAsSource : False
PolicyStore : ActiveStore
IPAddress : 192.168.65.17
InterfaceIndex : 28
InterfaceAlias : vEthernet (Default Switch)
AddressFamily : IPv4
Type : Unicast
PrefixLength : 28
PrefixOrigin : Manual
SuffixOrigin : Manual
AddressState : Preferred

```

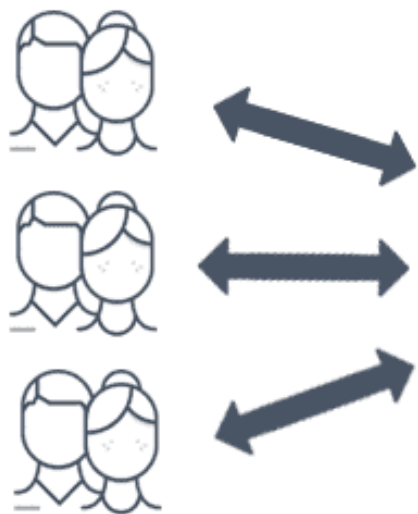


NETZWERKVERBINDUNGEN BEREITSTELLEN

Load Balancing



Application Clients (End Users)



Load Balancers (Software / Hardware)



Application Container

Kubernetes Clusters





RECHERCHE

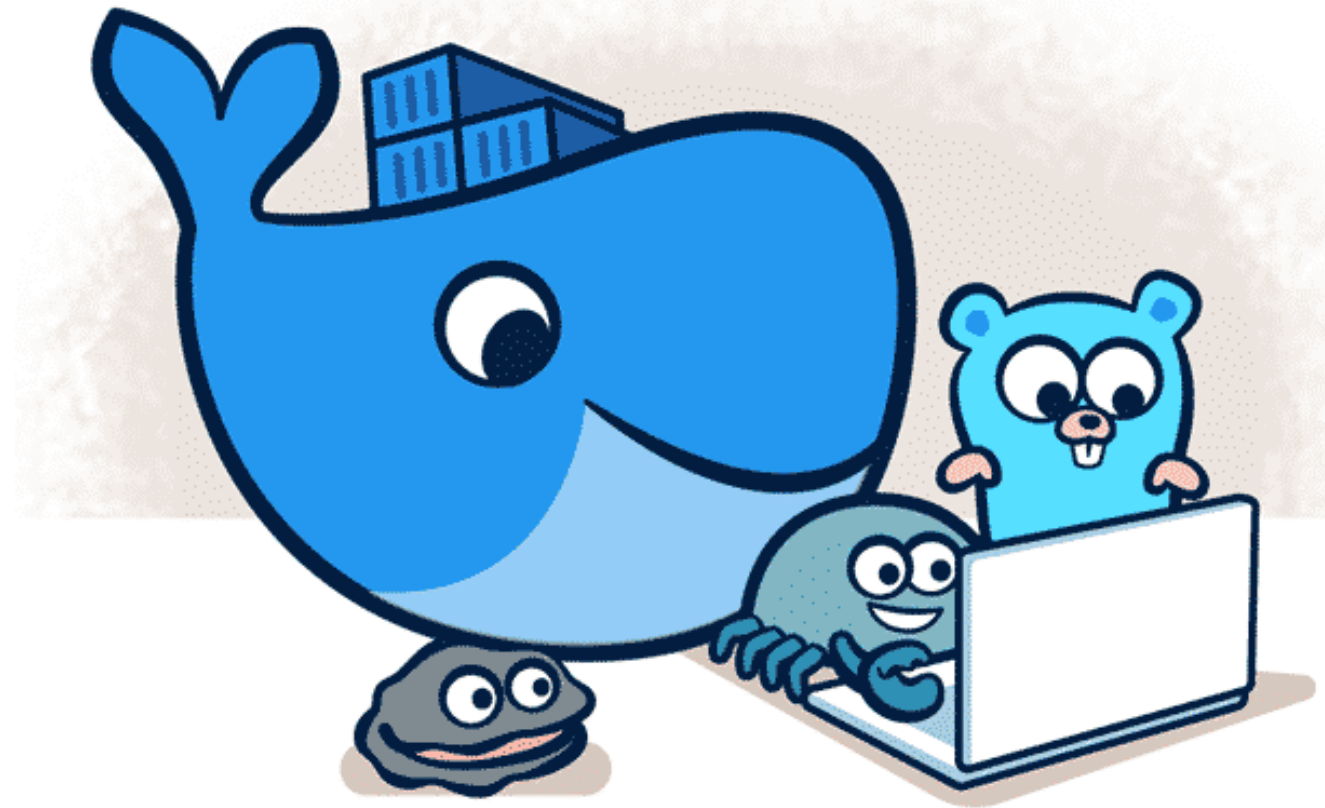
Finden Sie eine Anleitung und oder ein Tutorial welches die Installation und Einrichtung eines Load Balancers erklärt und darstellt. Versuchen Sie die einzelnen Schritte des Prozesses zu verstehen und darzustellen.

DISKUSSION

Diskutieren Sie in der Gruppe wann genau ein Load Balancer Einsatz findet und unter welchen Umständen er einen großen Vorteil da stellt. Gehen Sie besonders auf persönliche Erfahrungen ein sowie schon bekannte Verfahren falls Sie welche haben und oder kennen.

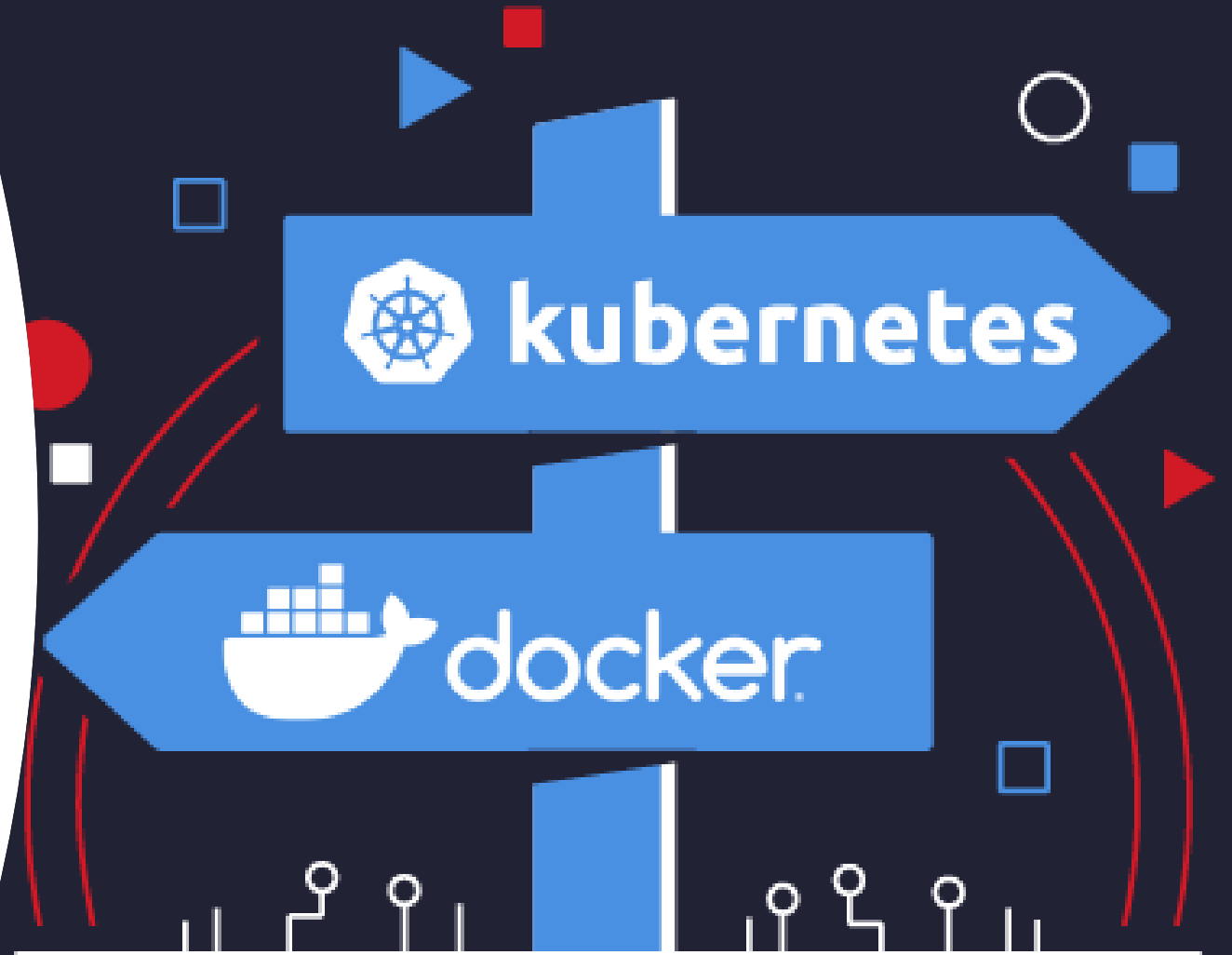
01.

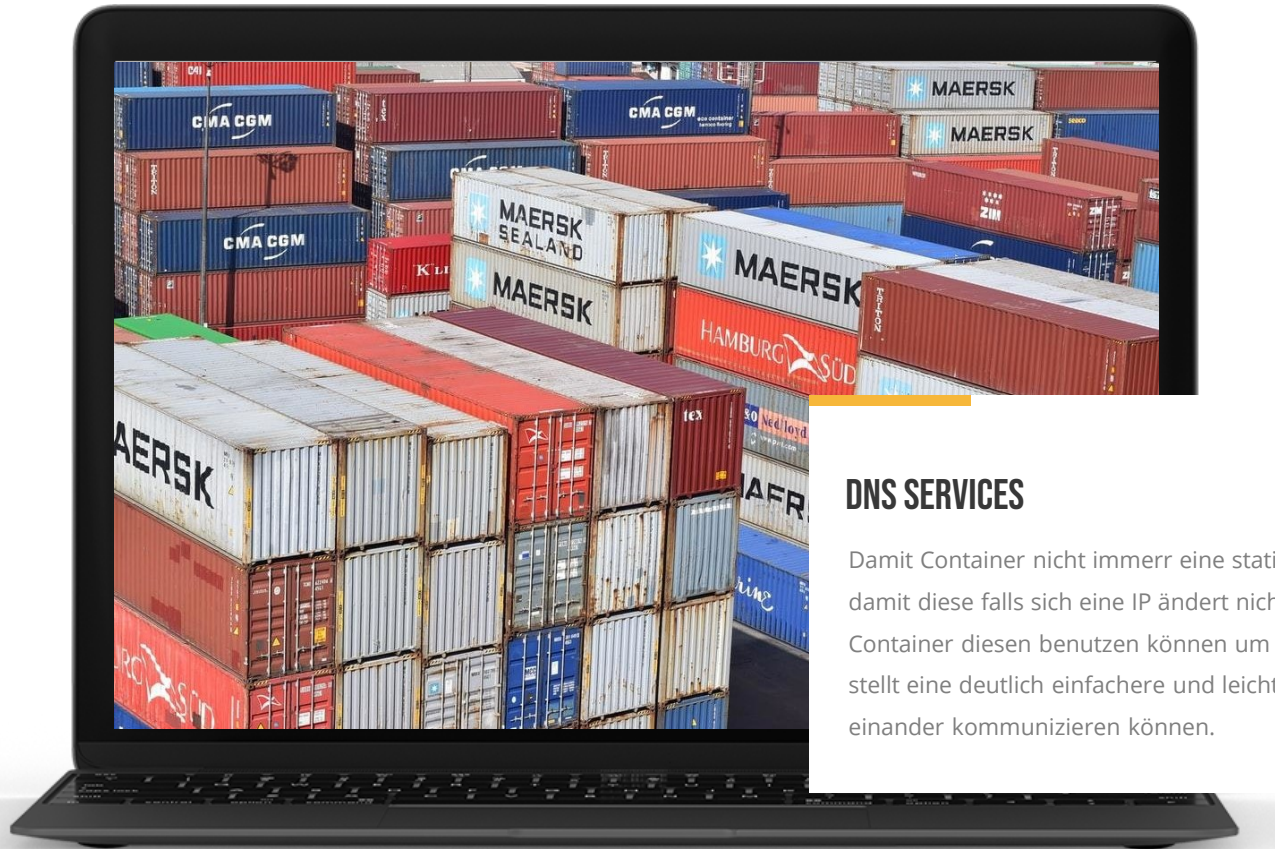
02.



NETZWERKVERBINDUNGEN BEREITSTELLEN

DNS basierte Verbindungen



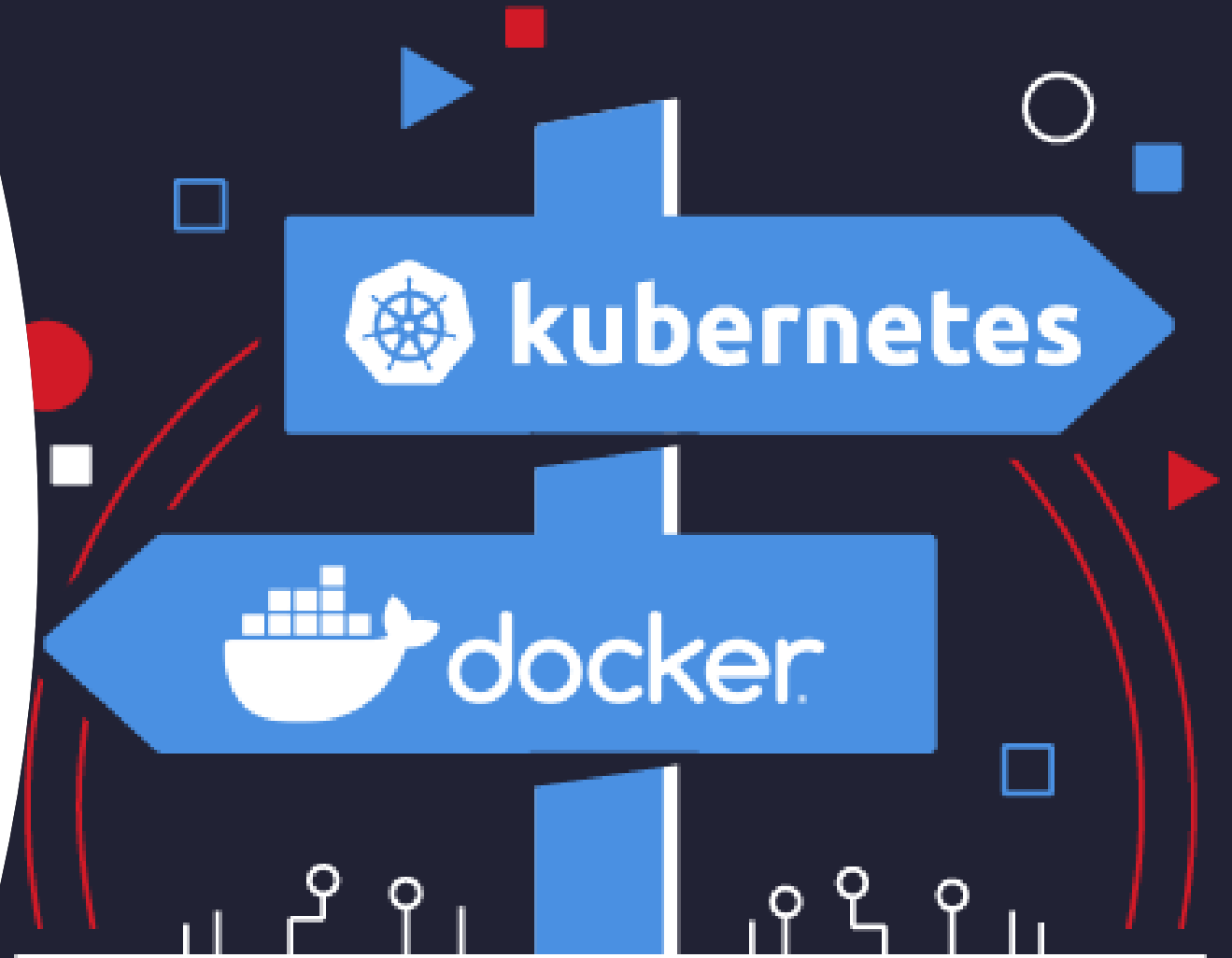


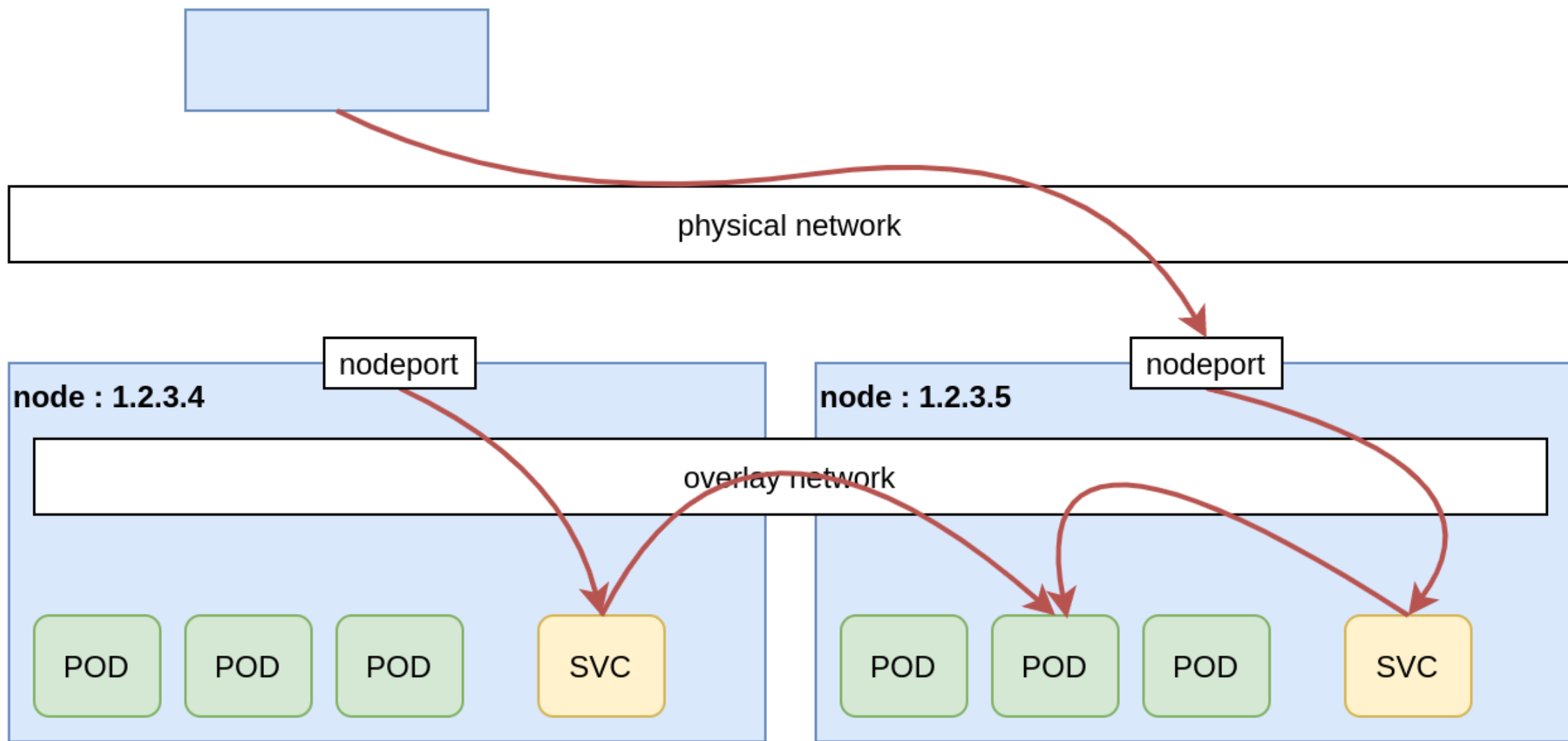
DNS SERVICES

Damit Container nicht immerr eine statische IP für jeden anderen Container sich merken müssen bzw. damit diese falls sich eine IP ändert nicht verlieren kann ein DNS Service verwendet werden damit alle Container diesen benutzen können um andere Container und oder Pods aufzufinden und zu erreichen. Dies stellt eine deutlich einfachere und leichtere Möglichkeit zu sein dafür zu sorgen das Container immer mit einander kommunizieren können.

NETZWERKVERBINDUNGEN BEREITSTELLEN

NodePort, Ingress





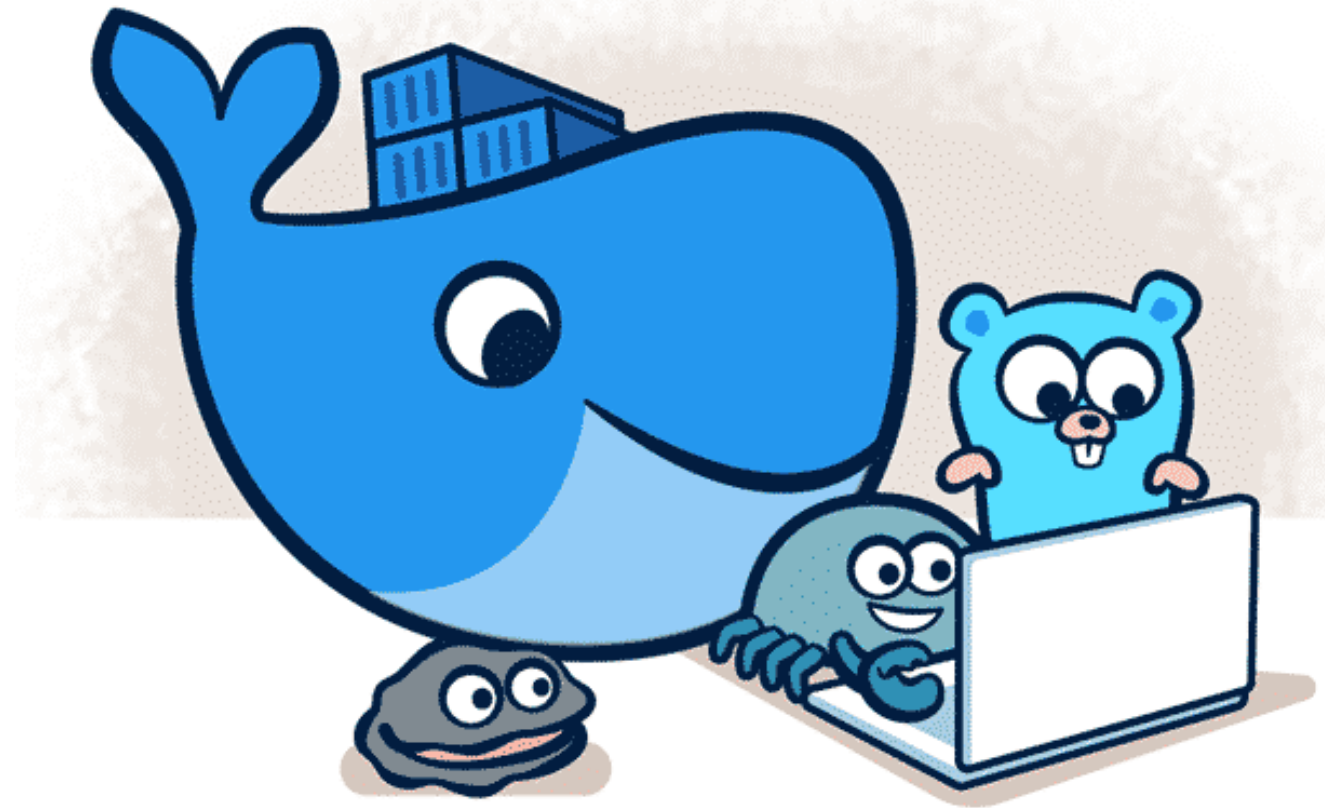
ALLES DURCH EINEN PUNKT

Wie der Name Node Port schon sagt handelt es sich hier um einen Port den jede Node in jedem Cluster offen hat mittels dem es möglich ist mit der Node und den dort beinhalteten dem Service zu kommunizieren.

VEREINFACHTE BENUTZUNG

Node Ports sind die einfachste weise um Traffic in eine Node und oder an einen Service zu bekommen. Besonders hilfreich wenn es darumm geht schnell und effizient eine Verbindung aufzubauen.

01.



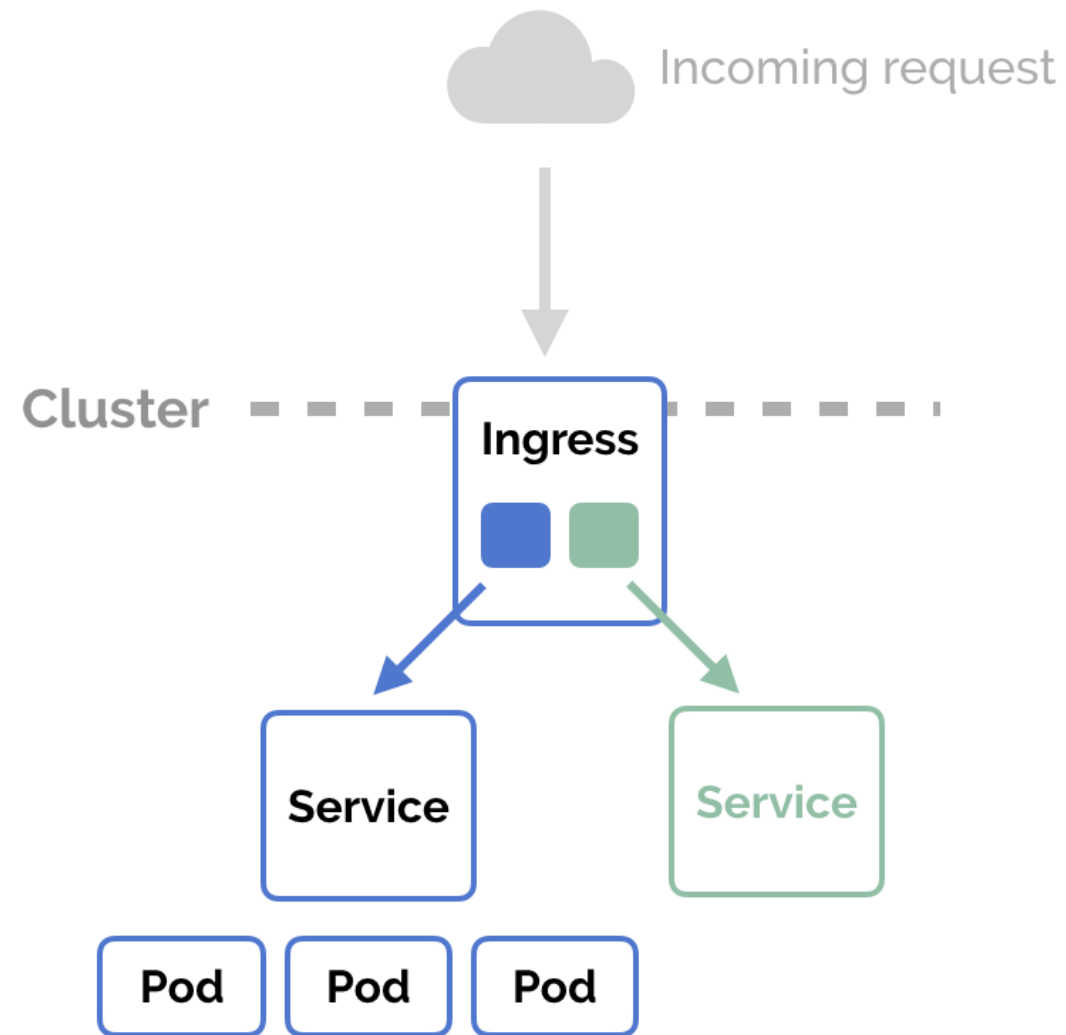
02.

What is Ingress?

[Ingress](#) exposes HTTP and HTTPS routes from outside the cluster to [services](#) within the cluster. Traffic routing is controlled by rules defined on the Ingress resource.

Here is a simple example where an Ingress sends all its traffic to one Service:

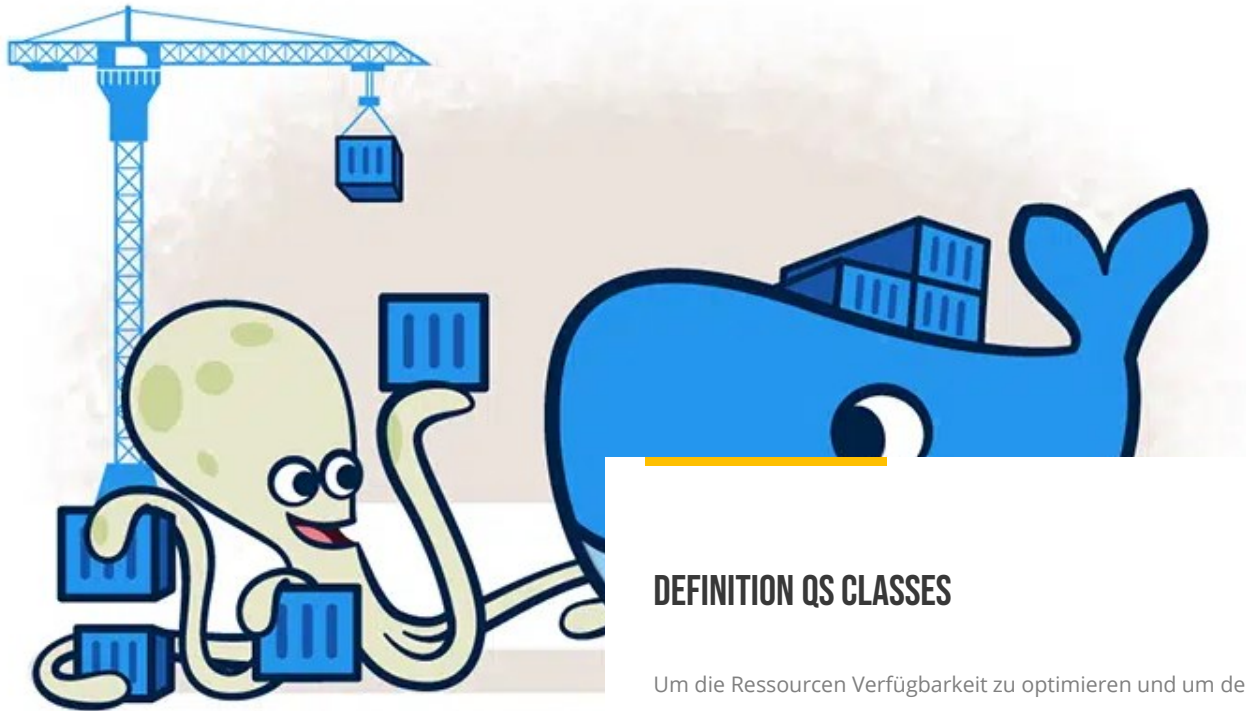




STEUERUNG, ÜBERWACHUNG UND KONTROLLE VON ANWENDUNGEN

Quality Class

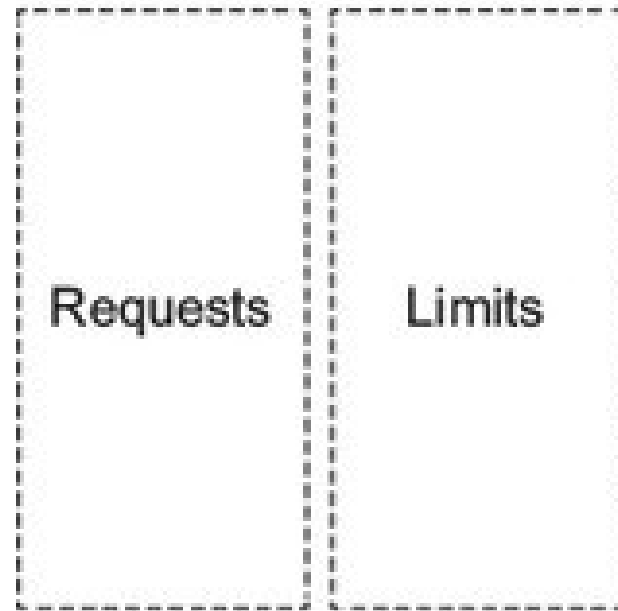




DEFINITION QS CLASSES

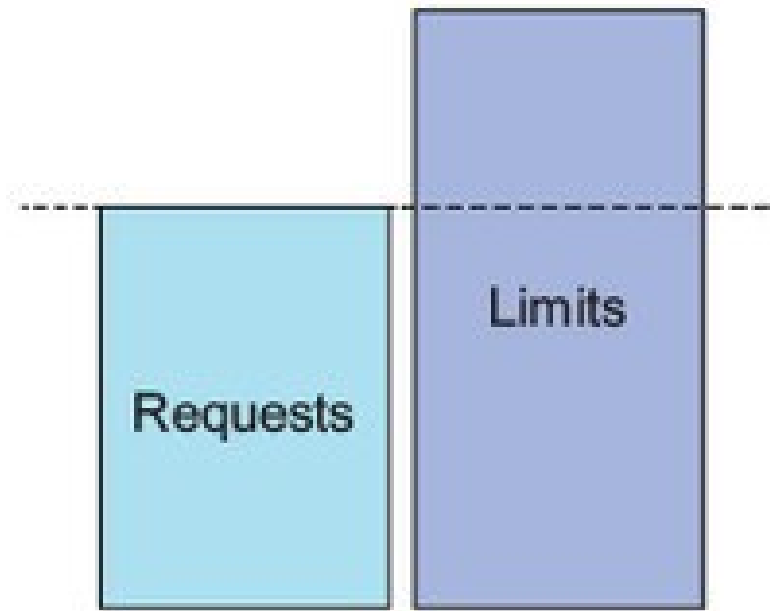
Um die Ressourcen Verfügbarkeit zu optimieren und um den Containern die Ressourcen zu geben die benötigt werden bietet Kuernetes verschiedene Qualität Stufen für Services an die verwendet werden können um diesen Services beizubringen wie viele und unter welchen Umständen diese Ressourcen bekommen können und wie mit diesen Ressourcen umgegangen werden soll.

**Requests and
limits are not set**



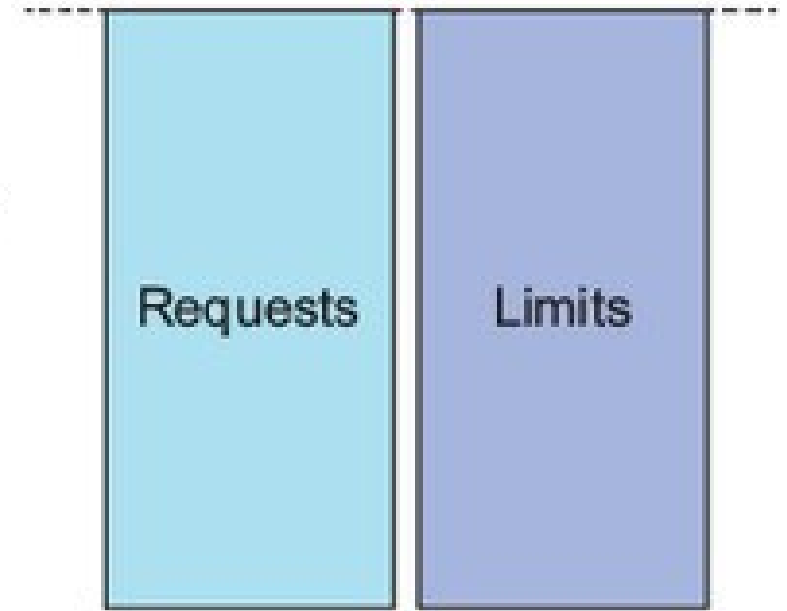
**BestEffort
QoS**

**Requests are
below limits**



**Burstable
QoS**

**Requests
equal limits**



**Guaranteed
QoS**

RECHERCHE

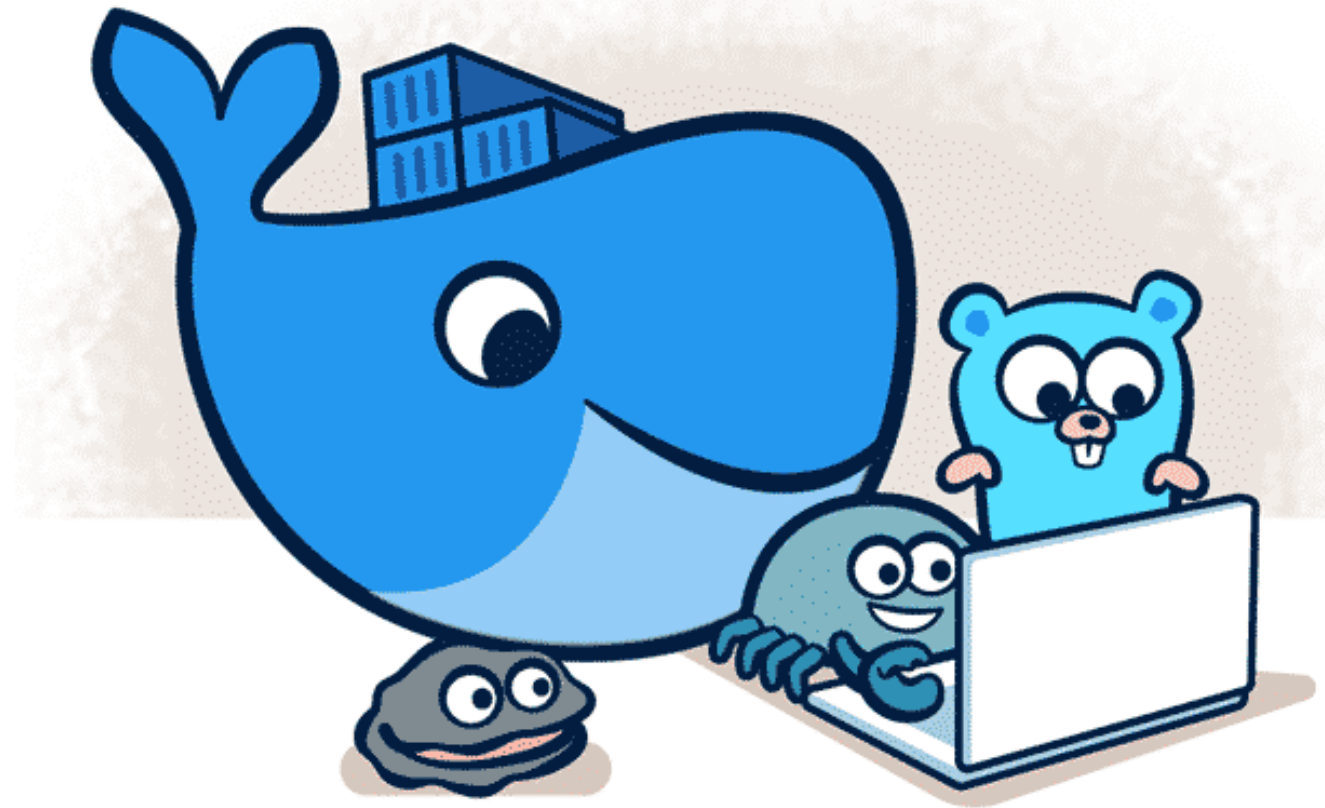
Finden Sie heraus wie genau man einem Service eine Quality Class gibt und wie genau hiermit umgegangen werden kann. Finden Sie hierzu eine Anleitung die Sie befolgen könnten.

GRÜNDE UND WICHTIGKEIT

Diskutieren Sie in der Gruppe wann genau Sie Quality Classes verwenden würden und welche Gründe es hierzu gibt. Seien Sie offen in der Verwendung sowie den Umständen die hierfür eintreten könnten.

01.

02.

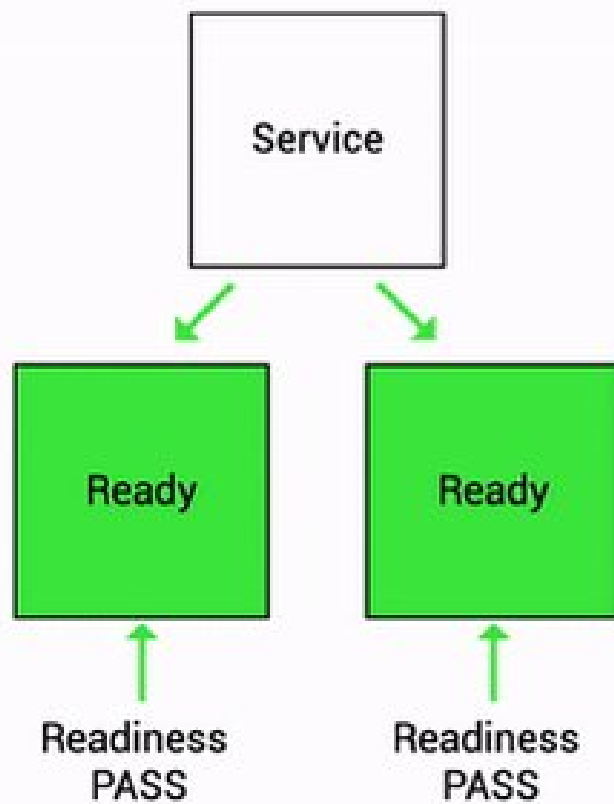


STEUERUNG, ÜBERWACHUNG UND KONTROLLE VON ANWENDUNGEN

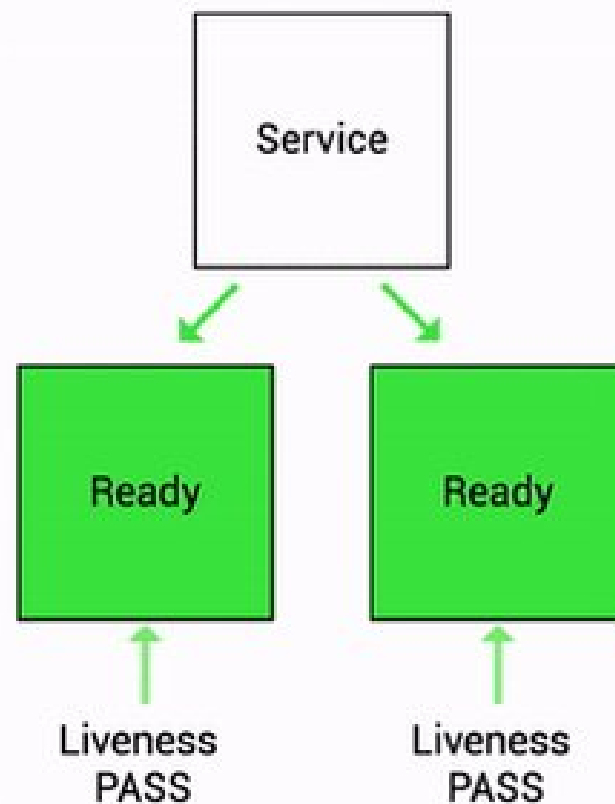
Health Checks für Pods (Container)

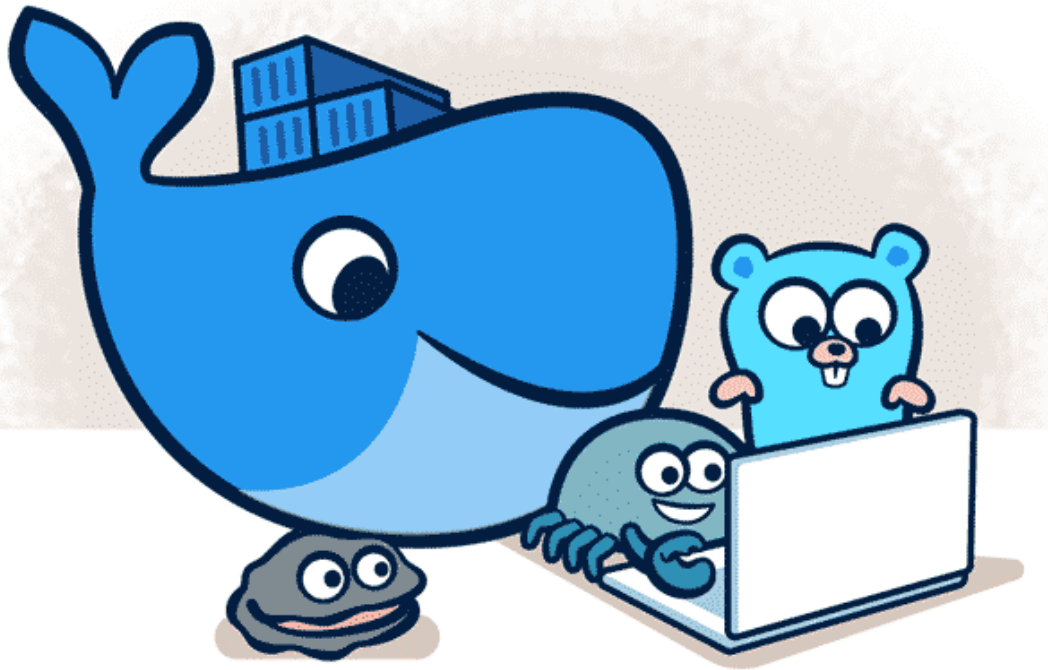


Readiness Probe



Liveness Probe





01.

LIVENESS CHECK

Many applications running for long periods of time eventually transition to broken states, and cannot recover except by being restarted. Kubernetes provides liveness probes to detect and remedy such situations.

03.

STARTUP CHECK

Sometimes, you have to deal with legacy applications that might require an additional startup time on their first initialization. In such cases, it can be tricky to set up liveness probe parameters without compromising the fast response to deadlocks that motivated such a probe.

02.

READINESS CHECK

Sometimes, applications are temporarily unable to serve traffic. For example, an application might need to load large data or configuration files during startup, or depend on external services after startup. In such cases, you don't want to kill the application, but you don't want to send it requests either. Kubernetes provides readiness probes to detect and mitigate these situations. A pod with containers reporting that they are not ready does not receive traffic through Kubernetes Services.

AUFGABE

Um das Verständnis hiervon zu stärken recherchieren Sie wie genau Sie diese Checks implementieren. Versuchen Sie zudem einen dieser Checks auszutesten.

STEUERUNG, ÜBERWACHUNG UND KONTROLLE VON ANWENDUNGEN

Scheduling steuern (Taints und
Tolerations)

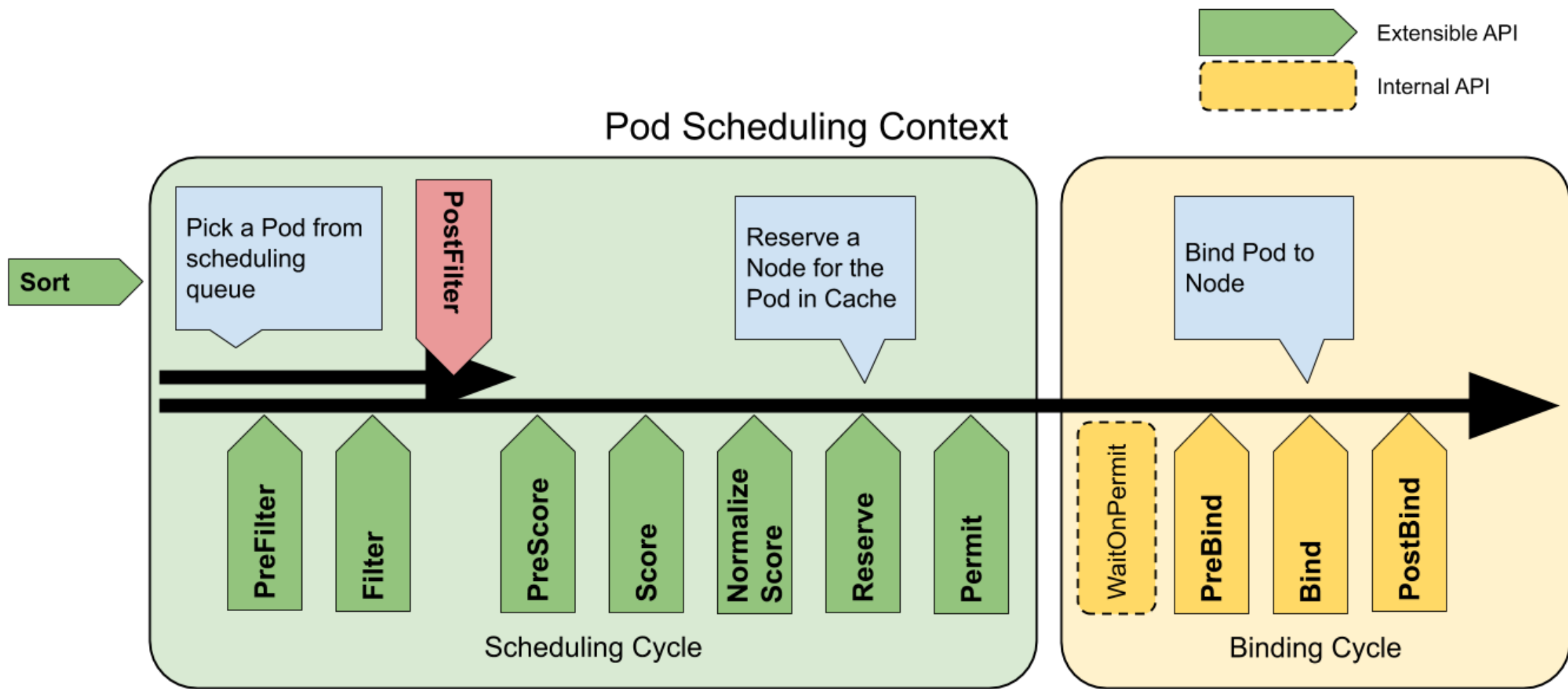


Taints and Tolerations

Node affinity is a property of Pods that *attracts* them to a set of nodes (either as a preference or a hard requirement). *Taints* are the opposite -- they allow a node to repel a set of pods.

Tolerations are applied to pods. Tolerations allow the scheduler to schedule pods with matching taints. Tolerations allow scheduling but don't guarantee scheduling: the scheduler also *evaluates other parameters* as part of its function.

Taints and tolerations work together to ensure that pods are not scheduled onto inappropriate nodes. One or more taints are applied to a node; this marks that the node should not accept any pods that do not tolerate the taints.



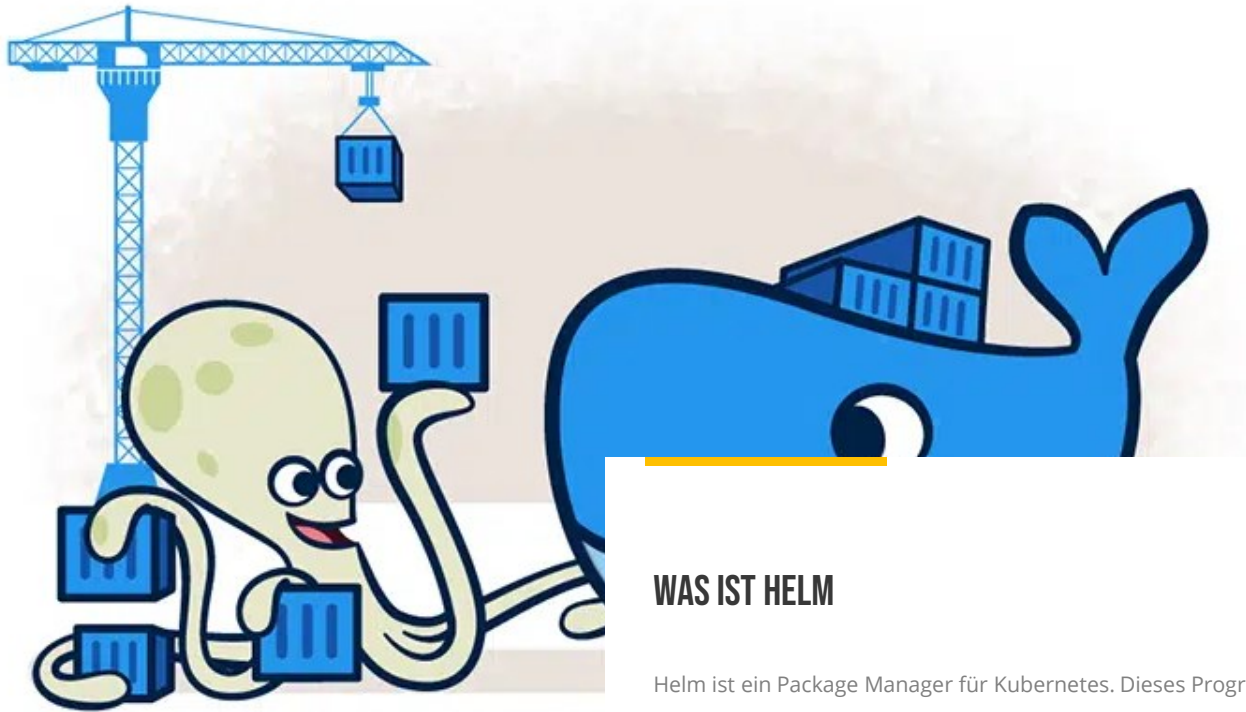
**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

Beschreibung der Anwendungen





**The
package manager
for Kubernetes**



WAS IST HELM

Helm ist ein Package Manager für Kubernetes. Dieses Programm erlaubt es verschiedene andere Anwendungen und oder Software die für die Orchestrierung sowie Auslieferung von Kubernetes relevant sein könnte zu installieren. Zudem kann man mittels Helm einfach alle Hand von vorkonfigurierten Pods, Containern und Konfigurationen schnell und unkompliziert ausliefern und benutzen. Es ist so etwas wie die allgemein Lösung für alle Themen und oder Probleme in der Welt von Kubernetes.

**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

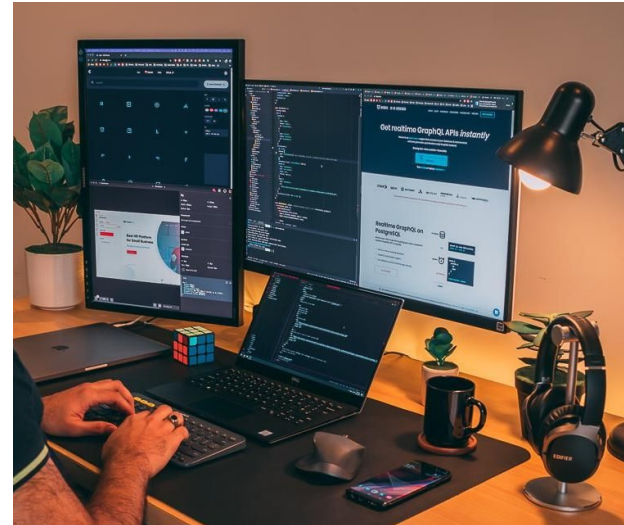
Arbeiten mit Helm



Prerequisites

The following prerequisites are required for a successful and properly secured use of Helm.

1. A Kubernetes cluster
2. Deciding what security configurations to apply to your installation, if any
3. Installing and configuring Helm.



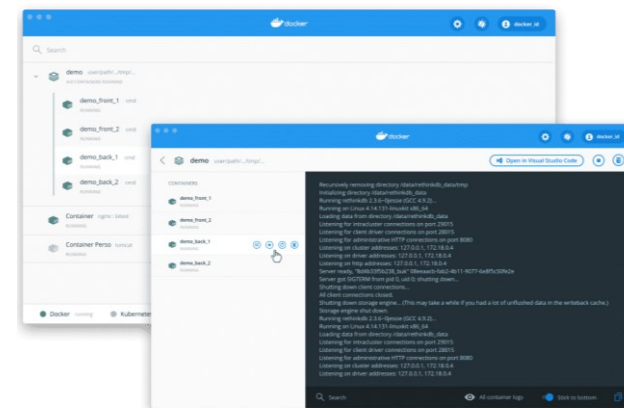
AUFTRAG 1

Lesen Sie sich folgende Anleitung durch <https://helm.sh/docs/intro/install/> folgen Sie dieser und gehen Sie besonders darauf ein wie genau Ihre Installation verläuft und was Sie genau erfahren.

AUFTRAG 2

Testen Sie Helm nach der Installation aus und schauen Sie sich an welche verschiedenen Packages sie benutzen können.

Insbesondere dokumentieren Sie sich bitte Fehlschläge und oder Probleme damit diese im Nachhinein in der Gruppe behandelt und gelöst werden können.



**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

Lifecycle Management



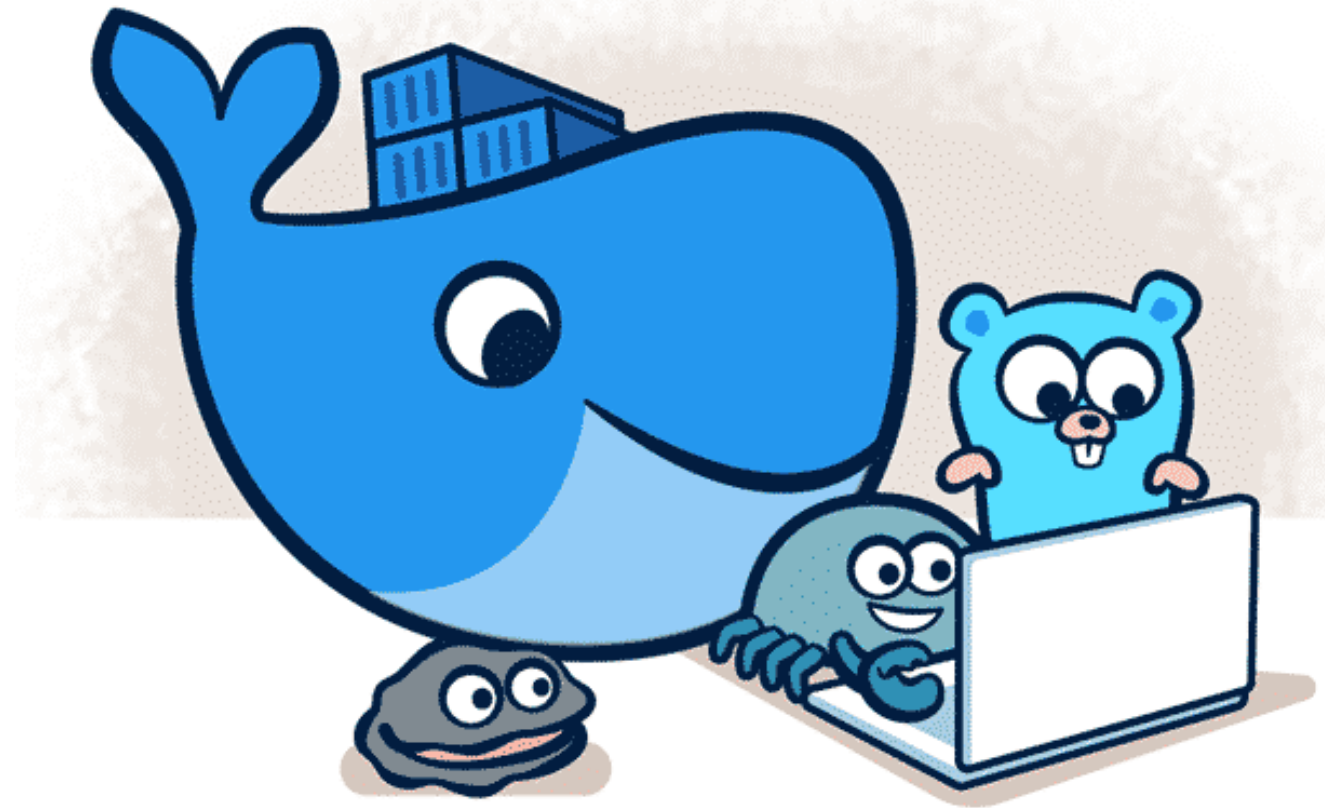
WAS IST EIN LIFECYCLE

Während der Entwicklung sowie der Bereitstellung einer Anwendung wird diese häufig verändert und Updates und Änderungen müssen ausgerollt werden. Um dafür zu sorgen, dass die Anwendung stets verfügbar und benutzt werden kann, wird ein Lifecycle Management benutzt.

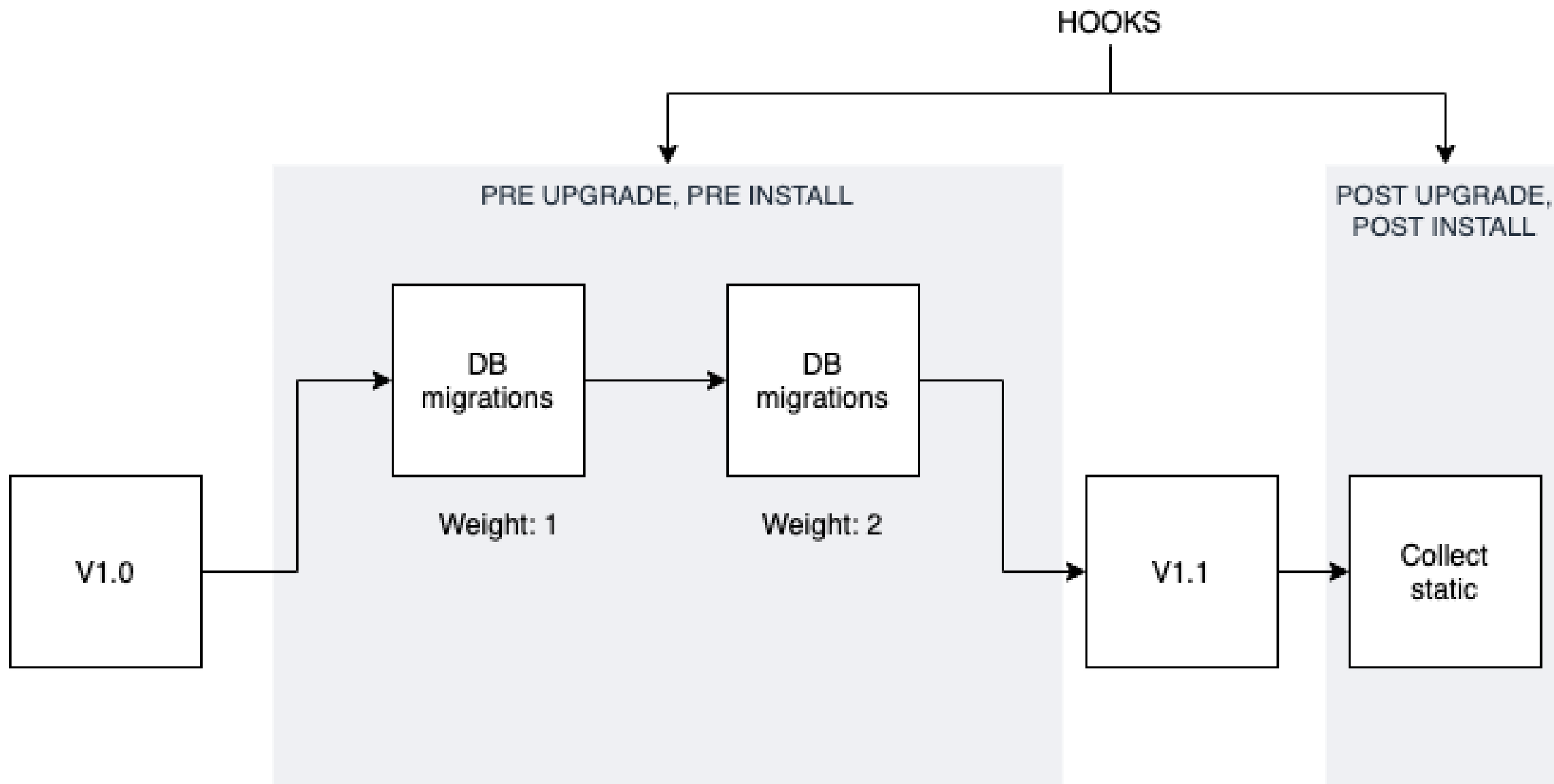
FUNKTIONSWEISE

Helm erstellt von allen nicht statischen Ressourcen ein „Backup“, welches während des Änderungsprozesses gewechselt wird und sobald die Änderung vollzogen wird wieder gelöscht werden kann.

01.



02.



**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

Deployment



DEPLOYMENT

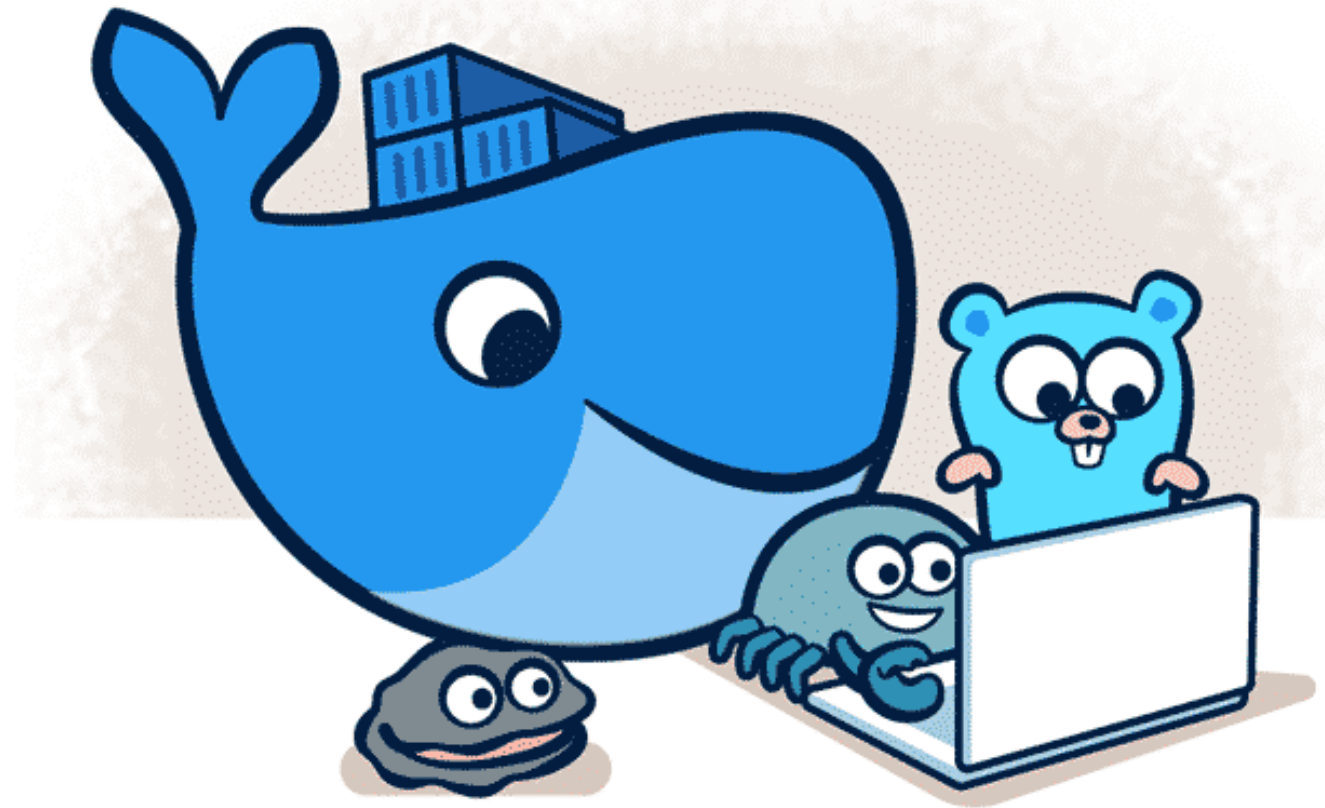
Ein Deployment ist wie der Englische Begriff aussagt das erstellen eines neuen Pods und oder Containers mittels eines Charts. Diesen Vorgang bezeichnet man als Deployment.

AUFGABE

Versuchen Sie ein eigenes Deployment zu erstellen und dieses zu verwalteten. Geben Sie sich den Status Ihres Deployments aus und diskutieren Sie in der Gruppe was Sie hier genau sehen.

01.

02.



**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

Upgrade / Rollback



UPGRADE

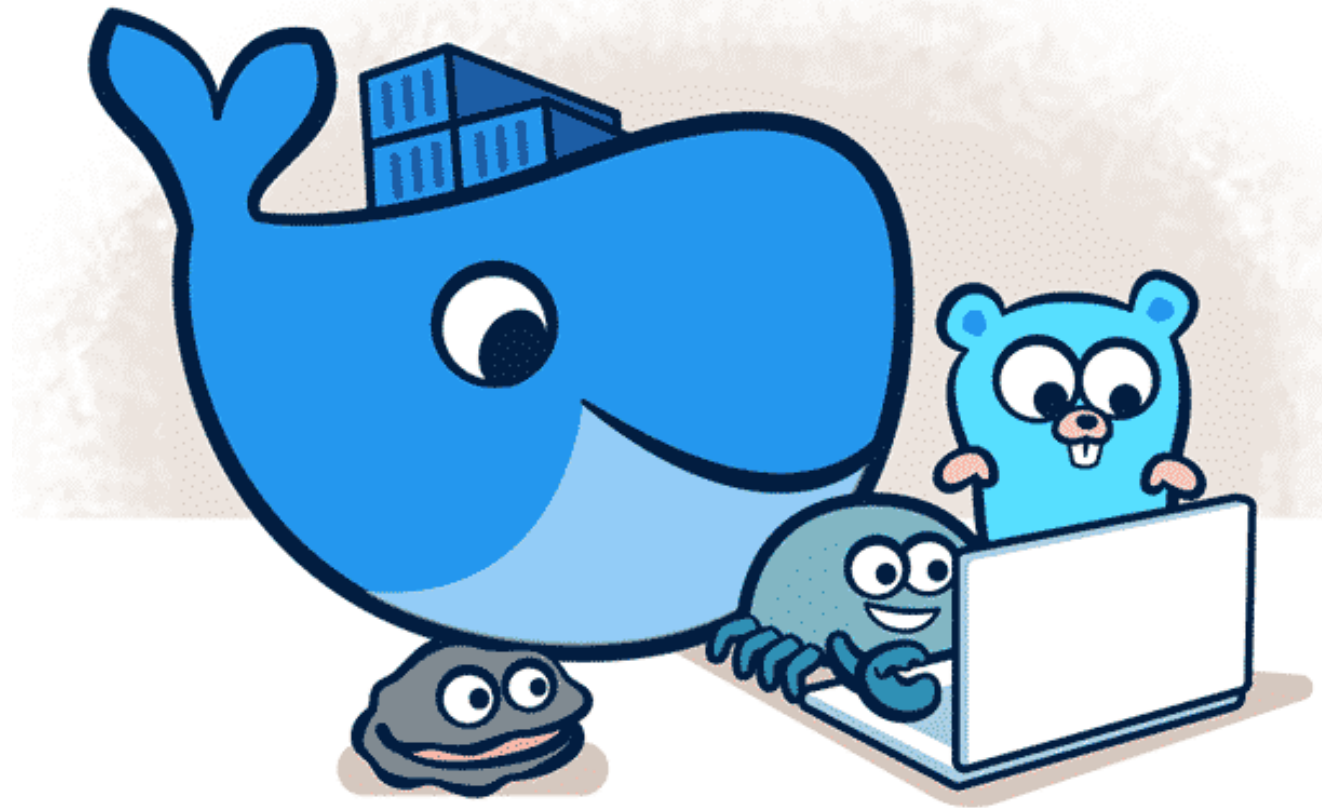
Mittels des Update Befehls ist es möglich ein schon vorhandenen Chart mit einer neuen Version zu überschreiben. Diesen Befehl führt man standardmäßig aus wenn man einen Chart auf eine neue Version ziehen möchte und oder ein Update vorhanden ist.

ROLLBACK

Rollback macht genau das Gegenteil von upgrade und ist in der Lage auf die vorherige Version zurück zu wechseln. Besonders hilfreich wenn Fehler beim Wechsel auf eine neue Version passieren und oder bestimmte Eigenschaften mit anderen Charts und Deployments kollidieren.

01.

02.





HELM ROLLBACK <RELEASE> [REVISION] [FLAGS]

**KOMPLEXE ANWENDUNGEN
EINFACH DEPLOYEN:
DER HELM PAKET MANAGER**

Helm Charts aus der Community
nutzen



Explore and discover packages



registry-ui

USER: Joaquinito2051 REPO: Bryanalves

VERSION: 0.2.0 APP VERSION: 0.0.2

A simple web ui frontend for v2 docker registries

★ 0

 Helm chart

Updated 5 months ago

F Images Security Rating



kubeform-provider-azurerm-kubernetescluster-crds

ORG: AppsCode Inc. REPO: appscore

★ 0

 Helm chart

Updated 3 months ago

Explore and discover packages

01



steadybit-platform

ORG: steadybit REPO: steadybit Kubernetes Hel...

VERSION: 0.5.0 APP VERSION: latest

★ 0

Helm chart

Updated 25 days ago

steadybit Platform Helm chart for Kubernetes.

Verified Publisher

Security scanner disabled



predator

ORG: Helm REPO: zooz

VERSION: 1.6.3 APP VERSION: 1.6

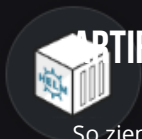
★ 2

Helm chart

Updated 4 months ago

Predator is an open-source distributed performance testing platform

Images Security Rating



velero-crd

ORG: Wiremind REPO: Wiremind

VERSION: 2.28.0 APP VERSION: 1.8.0

★ 0

Helm chart

Updated 5 months ago

So ziemlich jedes große und kleine Package kann auf dieser Seite gefunden werden.

Hier sollten Sie immer als erstes suchen, da Sie hier auch am ehesten fündig werden.

A Helm chart to manage velero CRDs

Verified Publisher

02

basic_crud_app_flutter

Public

A simple but basci CRUD App for you to start with. A Project out of my A Guide to Flutter book.

● C++ ☆ 1

basic_time_tracker_flutter

Public

A simple but easy time tracker for every Flutter Lover

● C++ ☆ 1

data_visualization_tool_flutter

Public

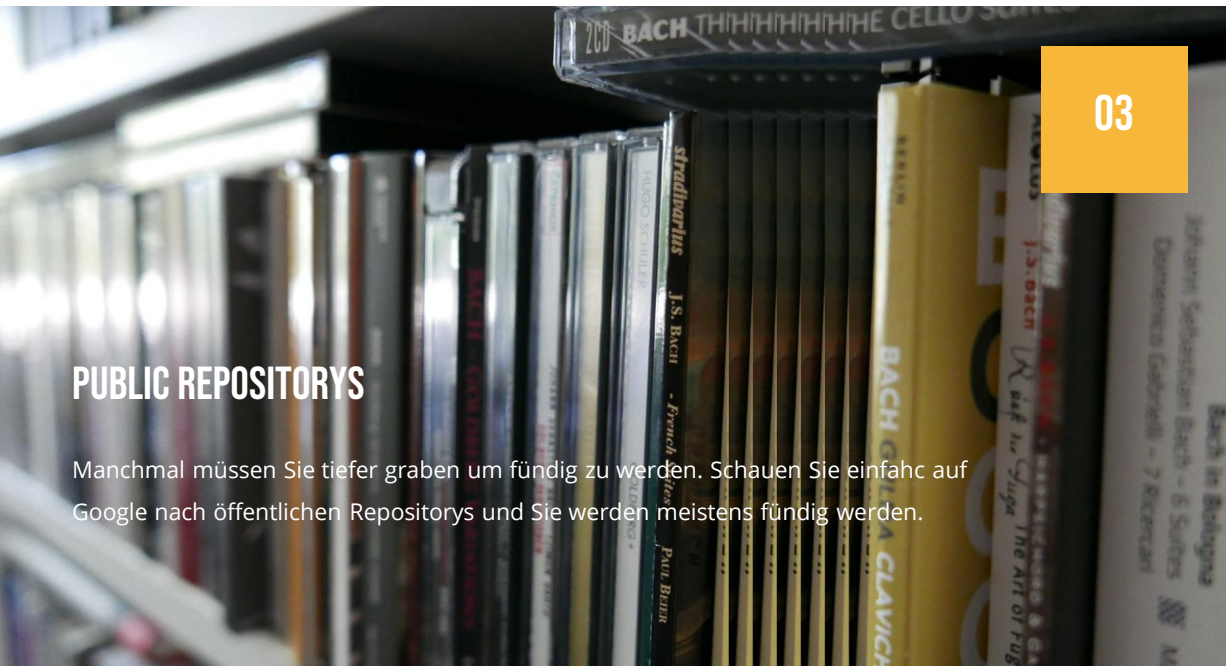
● C++ ☆ 1

GITHUB

330 contributions in the last year

Contribution settings

So wie Helm Open Source ist, so sind dies auch viele der Packages die entwickelt werden. Schauen Sie einfach auf GitHub vorbei und finden Sie wonach Sie suchen.



03

PUBLIC REPOSITORIES

Manchmal müssen Sie tiefer graben um fündig zu werden. Schauen Sie einfach auf Google nach öffentlichen Repositories und Sie werden meistens fündig werden.

KOMPLEXE STATEFUL ANWENDUNGEN (DATENBANK) DEPLOYEN

Das Operator Konzept



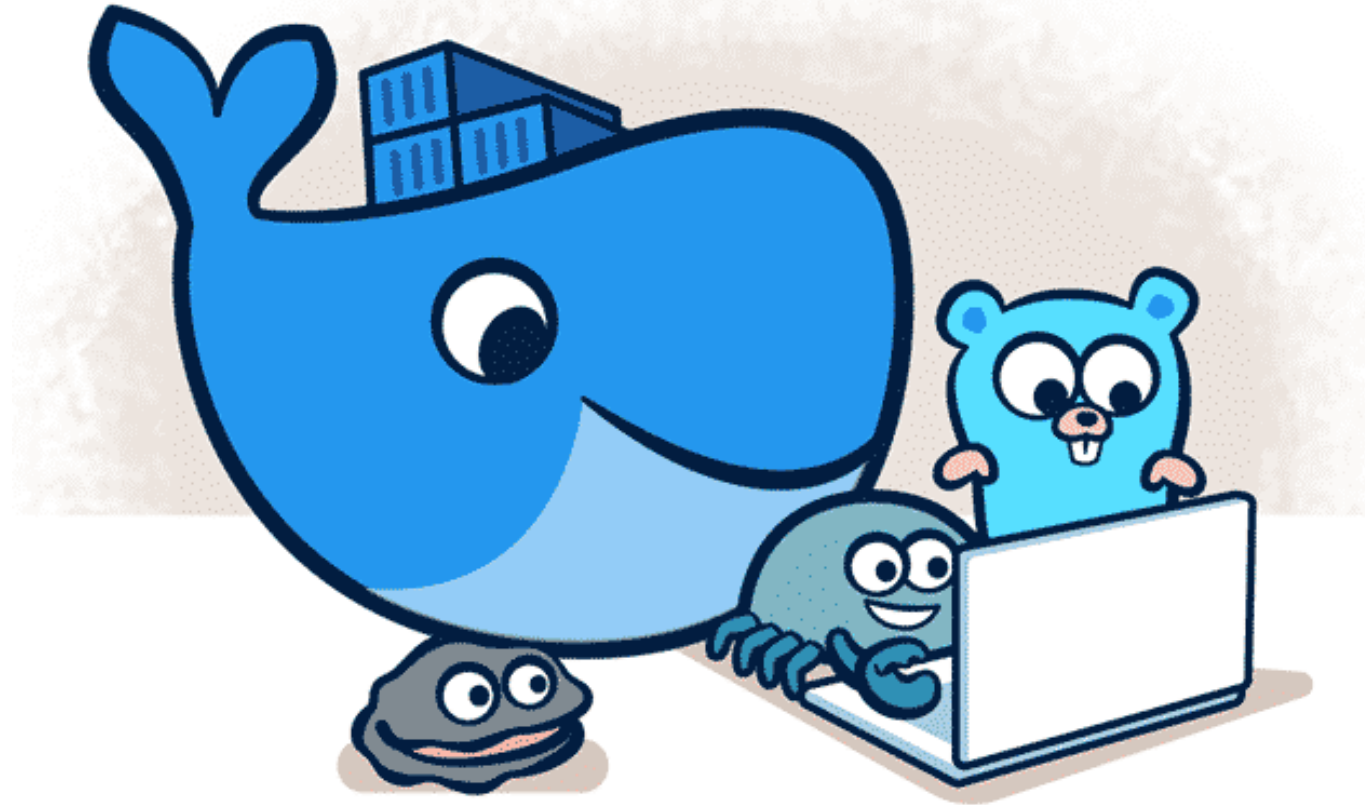
DEFINITION

Operator ist eine erweiternde Software für Kubernetes welche es erlaubt eigene Ressourcen zu benutzen um Anwendungen und deren Komponente zu verwalten. Operatorn folgen den Kubernetes Prinzipien, im besonderen dem des Control Loops.

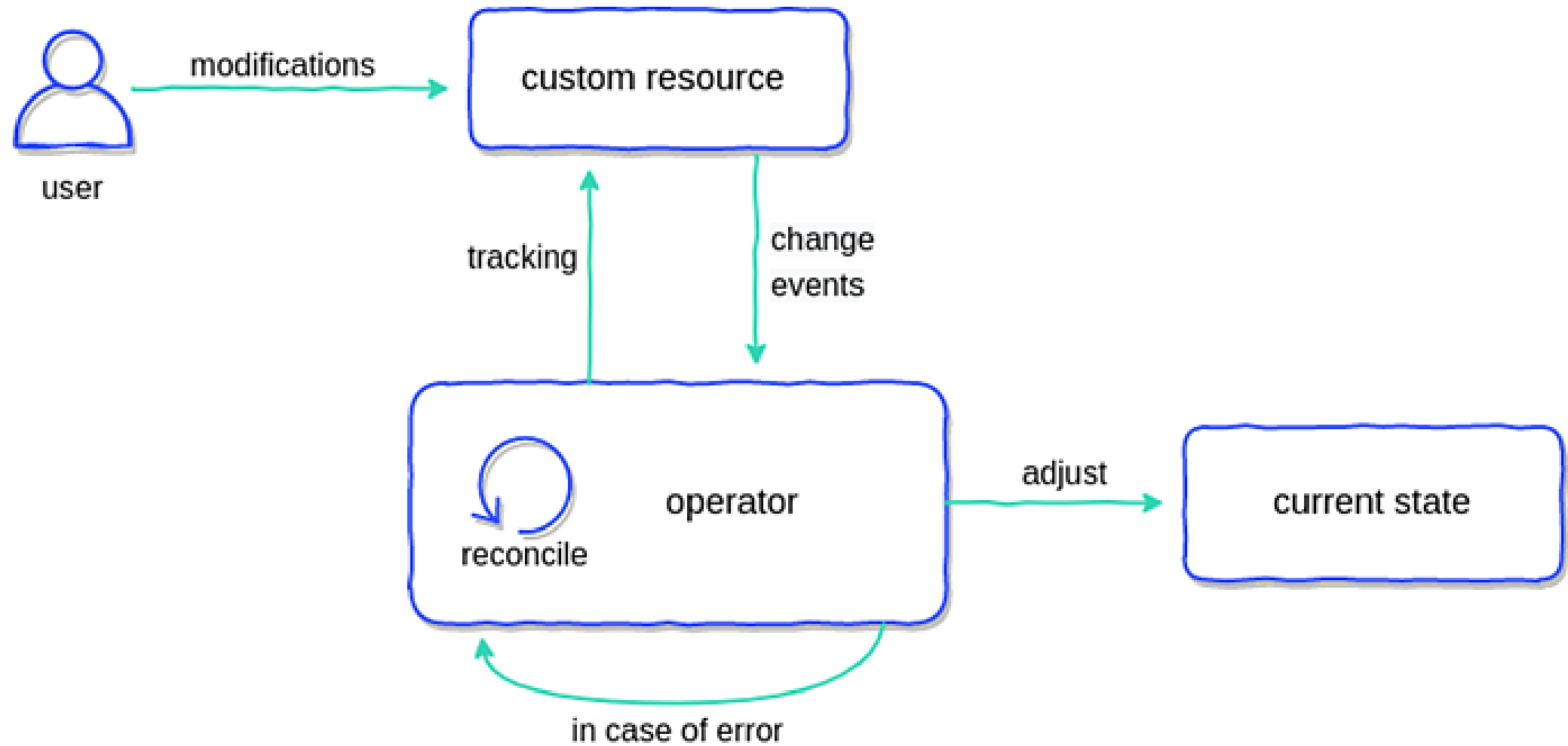
MOTIVATION

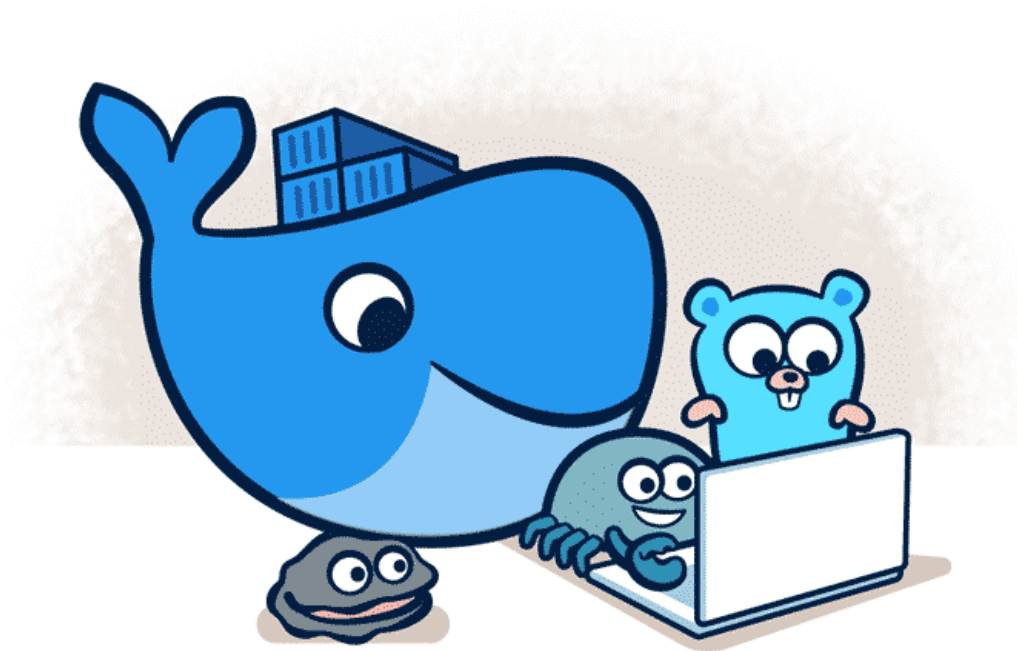
Das Konzept des Operators versucht das aufzunehmen wie ein „menschlicher“ Anwender Tätigkeiten ausführen würde. Diese sollen dann automatisiert und vereinfacht werden. Dies erlaubt es sicherzugehen das Konzepte und oder Tätigkeiten so ausgeführt werden wie es ein Mensch tun würde.

01.



02.





01.

DEPLOYMENT

Das Anlegen und starten von Anwendungen ist zeit aufwändig und kann per Operator so vereinfacht werden das bei einer spezifischen Änderungen automatisch neue Deployments erstellt werden.

02.

UPDATING

Manche Anwendungen erlauben es nicht das Updates in allen Ressourcen durchgeführt werden können. Zum Beispiel Datenbanken. Per Operator ist es möglich einen Prozess zu automatisieren der sich um diese Thematik kümmert.

03.

BACKUP

Um Datensicherheit und Integrität zu wahren ist es möglich einen Operator zu entwickeln der wenn Daten angelegt werden diese automatisch sichert und ein Backup erstellt welches dann weiter verwendet werden kann.

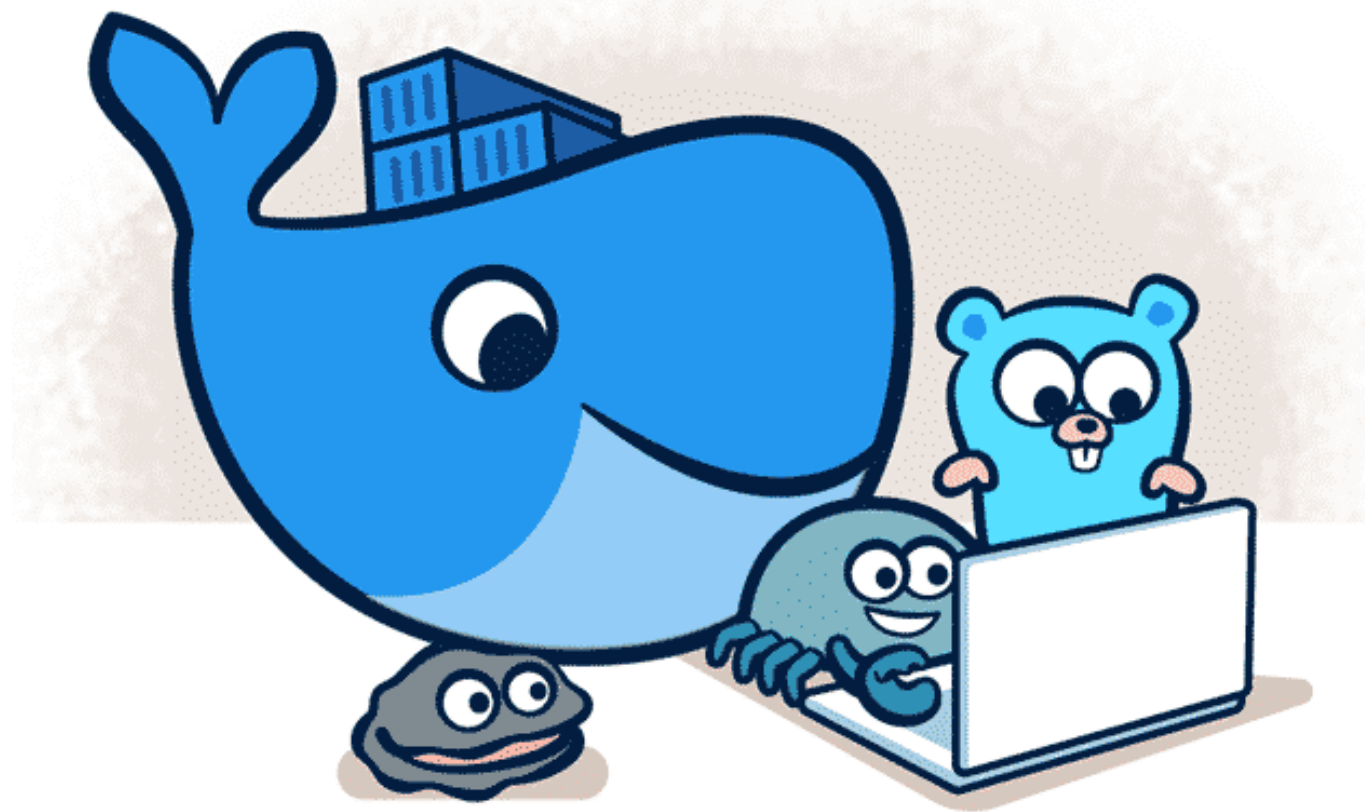
FEHLER

Manchmal ist es notwendig Fehler zu simulieren sei es nun in Tests und oder einfach nur um bestimmte Reaktionen aus der Anwendung zu erzeugen. Ins besonderem wenn Sie zum Beispiel nicht genau sicher seien können wie mit diesen Themen umgegangen werden soll.

IHRE MEINUNG

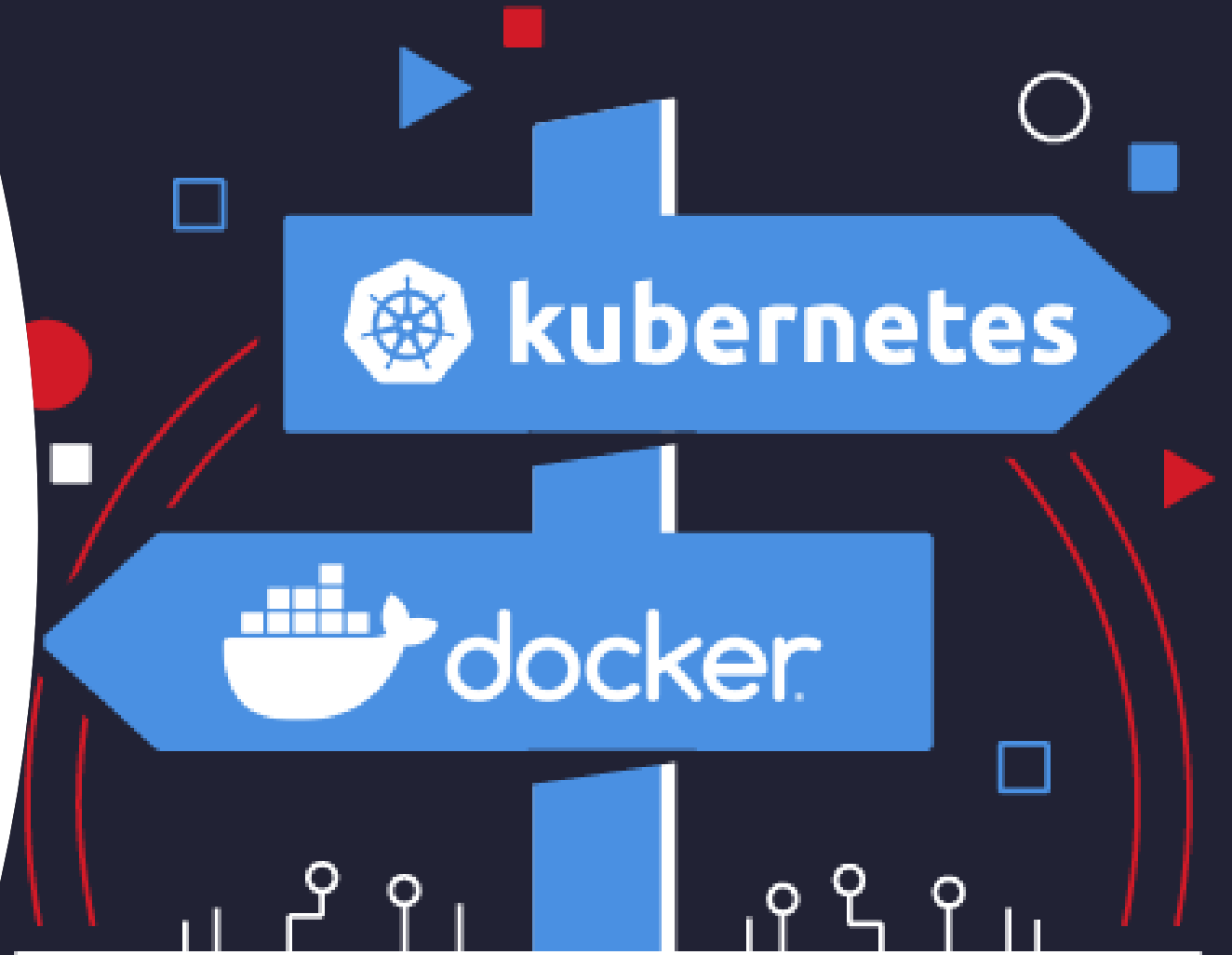
Denken Sie kurz darüber nach wofür Sie Operatoren verwenden würden und welche Vorteile diese für Sie im Unternehmerischen Sinn haben. Gehen Sie insbesondere darauf ein was Sie mit diesen anfangen könnten.

02.



TROUBLESHOOTING

Zugriff auf einen Pod



Create the Pod:

```
kubectl apply -f https://k8s.io/examples/application/shell-demo.yaml
```

Verify that the container is running:

```
kubectl get pod shell-demo
```

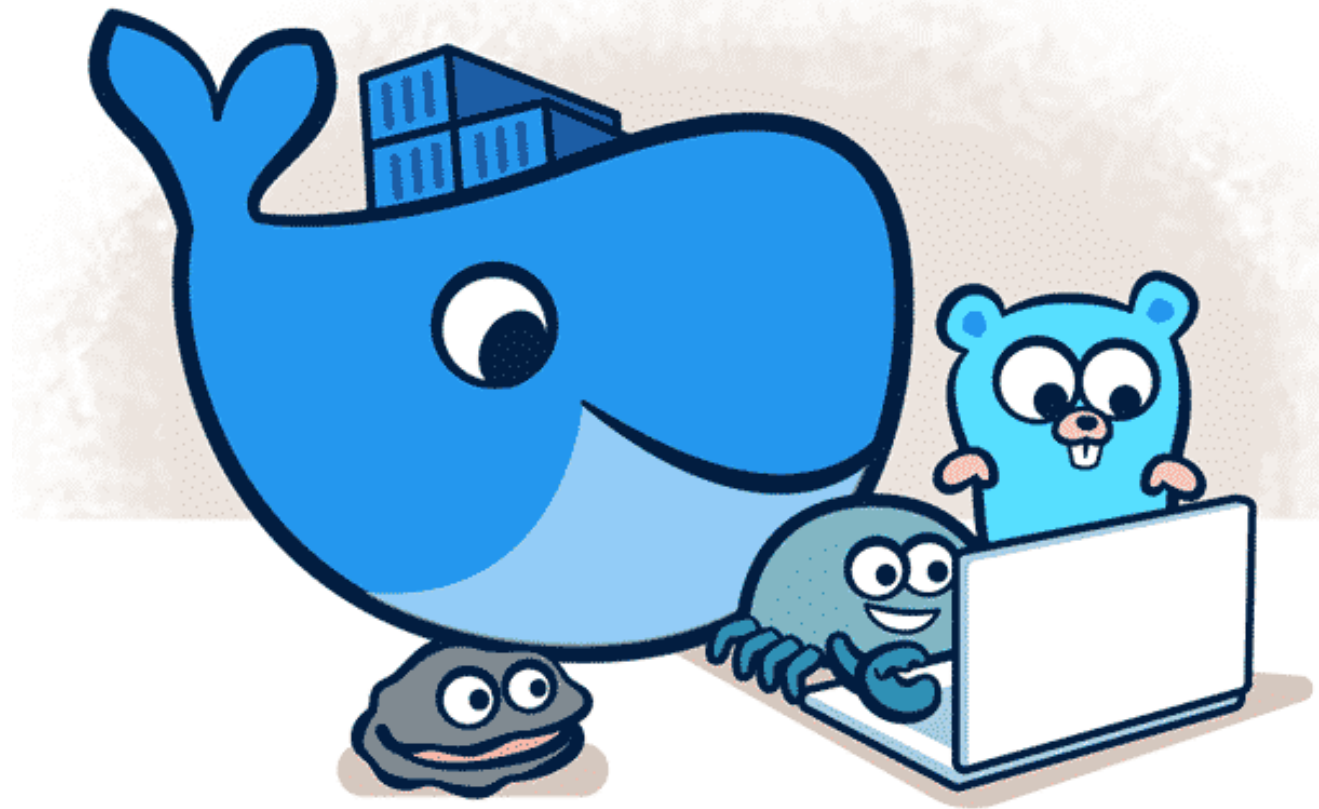
Get a shell to the running container:

```
kubectl exec --stdin --tty shell-demo -- /bin/bash
```

DISKUSSION

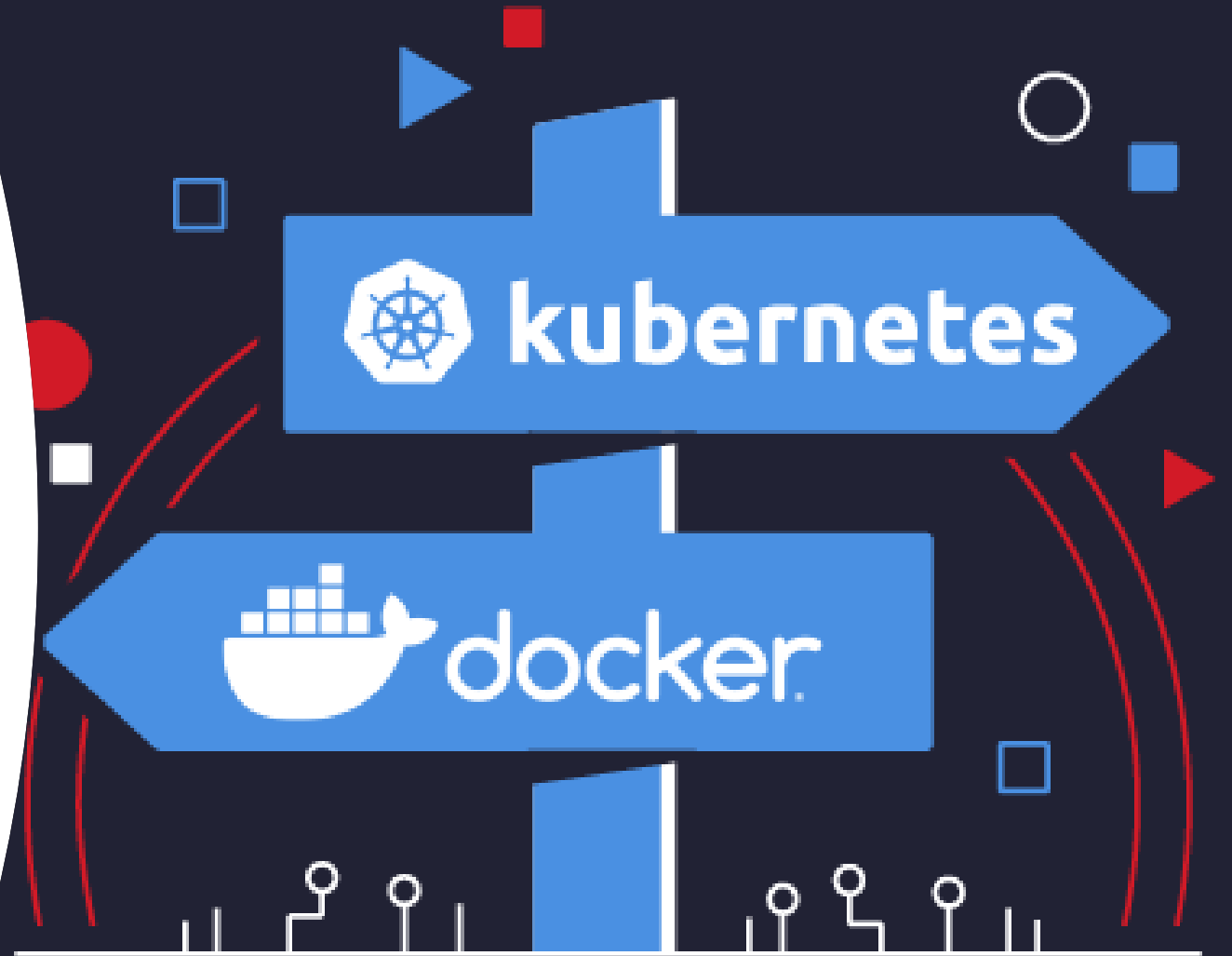
Können Sie sich vorstellen aus welchen Gründen man in einen Container rein gehen muss um in diesem Befehle auszuführen? Diskutieren Sie in der Gruppe nach Gründen die es für solch eine Funktionalität gibt.

02.



TROUBLESHOOTING

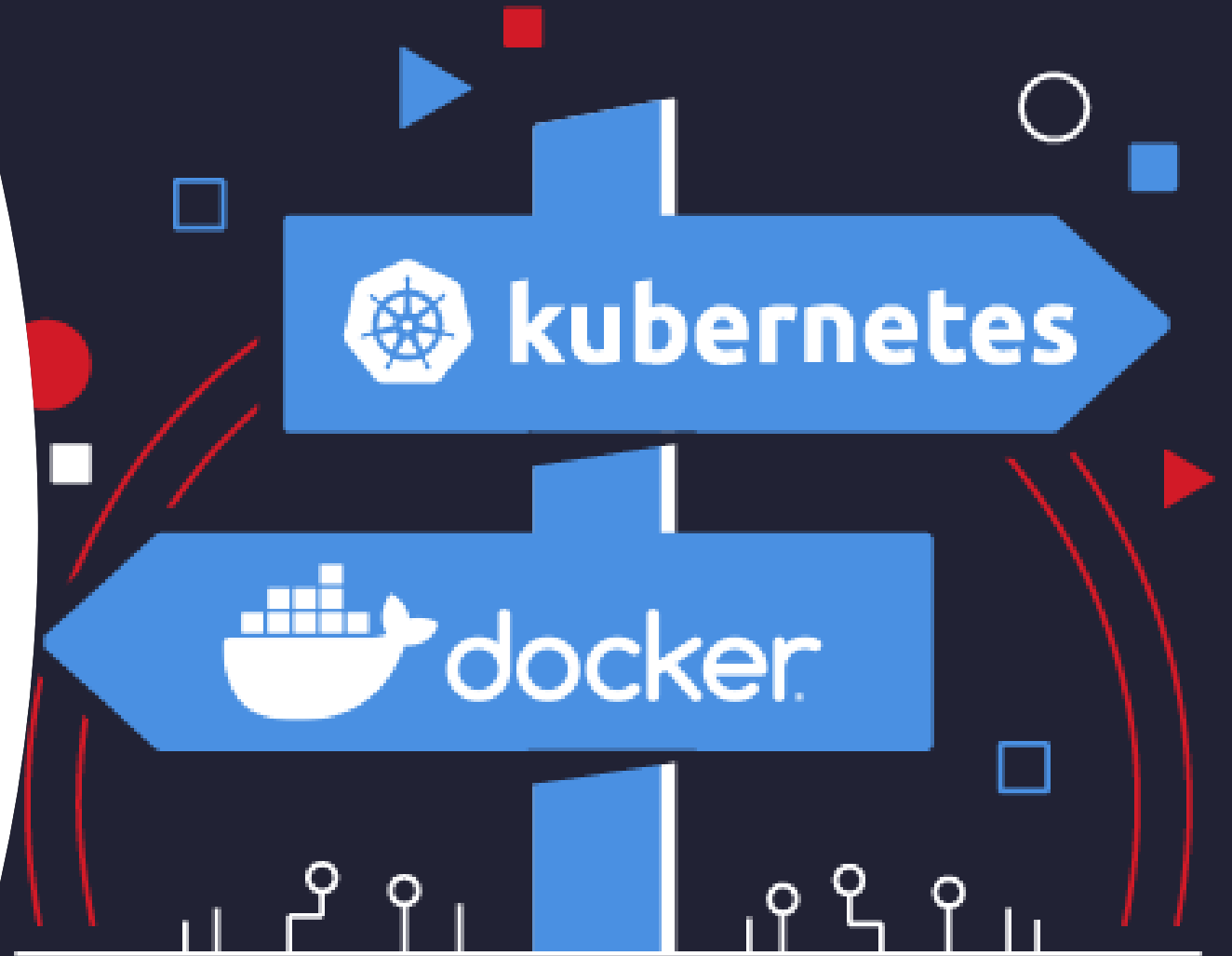
Netzwerkverbindungen testen



PING <IP> | | CAT <IP>

TROUBLESHOOTING

Logging / Event Infos des CLI Tools



KUBECTL LOGS POD-NAME

KUBECTL LOGS POD-NAME CONTAINER-NAME

```
kubectl logs pod-name --since=2h
```

```
kubectl logs job/my-job  
kubectl logs deployment/my-deployment
```



PAUSE EINFÜHRUNG IN KUBERNETES TAG 2

Nun folgt für alle Schulungsteilnehmer eine Pause. Bitte beachten Sie die 60 Minuten zeitliche Begrenzung der Pause da das Seminar direkt wieder starten wird.

Falls eine längere Pause von Nöten ist, bitte jetzt mir mitteilen.

■ FRISCHE LUFT

Bei gutem Wetter verlasse ich den Seminar Raum und schnappe frische Luft dabei sind Sie gerne bei eingeladen.

■ VERPFLEGUNG

Nun ist auch die Zeit zum essen gekommen. Falls Sie in den Räumlichkeiten essen möchten bitte denken Sie an Ihren Müll und lüften Sie an schließlich.

ZEIT: 60 MINUTEN

ZUGRIFFSKONTROLLE

Rollenbasierte Zugriffskontrolle



DEFINITION



Linux ist ein Multiuser-System: Mehrere Benutzer nutzen gleichzeitig den selben Rechner, und für verschiedene Aufgaben richtet man separate Accounts ein. Diese Zuteilung stellt die Kommandos für die Zugangsverwaltung vor.

Dabei sei gesagt das Docker Root Rechte an der Maschine benötigt, dies gilt auch automatisch für Kubernetes. Wie Sie sich sicher vorstellen können ist dies eine große Sicherheitslücke die nicht außer Acht gelassen werden kann.

Hier für wird in den folgenden Folien genauer darauf eingegangen wie mittels des Managements von Benutzern dafür gesorgt werden kann das ein sicheres und gut laufendes System vorliegt.

WARUM IST BENUTZER MANAGEMENT SO EIN GROßER PUNKT



01.

RESTRICT USER ACCESS

Benutzer sind in jedem Linux System häufig eine der ersten Stellen wo angegriffen wird. Dies ist hauptsächlich ein Angriffspunkt weil viele Benutzer zu hohe Rechte im System haben. Bzw. Auf fast alles Zugriff haben.

02.

RESTRICTED SUDO

Sudo ist ein sehr starkes und mächtiges Werkzeug, es kann aber auch jedem Anwender zum Verhängnis werden. Besonders mit der Arbeit mit Docker ist es wichtig das sudo sicher und Vertrauensvoll benutzt wird. Sollte dies nicht der Fall sein können die Konsequenzen ausschlagend sein.

03.

RESTRICT DOCKER

Mittels Docker können Anwendungen sowie Dienste gestartet werden die ein gesamtes System langlegen und oder angreifbar machen. Aus diesem Grund sollte immer nur eine Hand voll Benutzern Zugriff auf die Docker CLI bekommen.

04.

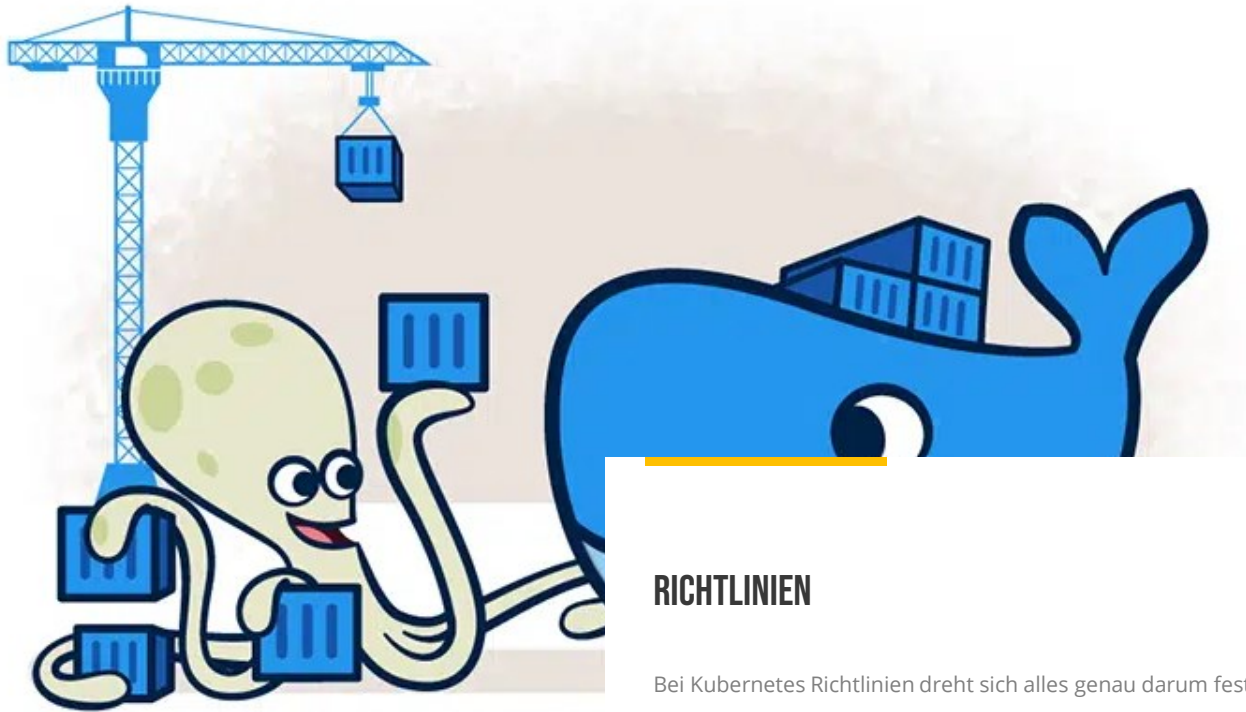
CLEAN & DESTROY

Jeder Start, Stopp und oder Verwendung von Docker Containern hinterlässt Spuren. Diese Spuren müssen stets beseitigt werden um sicherzugehen das keine Rest Informationen wie Passwörter auf dem System vorliegen.

ZUGRIFFSKONTROLLE

Richtlinien





RICHTLINIEN

Bei Kubernetes Richtlinien dreht sich alles genau darum fest zu legen wer wo, wie und warum drauf zugreifen darf und unter welchen Umständen dieser Zugriff soweit wie möglich eingeschränkt wird. Für Richtlinien gibt es immer verschiedene Gründe und oder Ideen wie diese angelegt werden sollen. Im folgenden wird Ihnen hierzu eine Recherche Aufgabe gestellt.

AUFGABE

RECHERCHE



Recherchieren Sie nach Beispielen für Richtlinien bei Kubernetes und gehen Sie besonders ins Detail wofür diese Richtlinien sind. Versuchen Sie zu verstehen warum man diese Richtlinien erstellt hat,

ÜBERLEGUNGEN

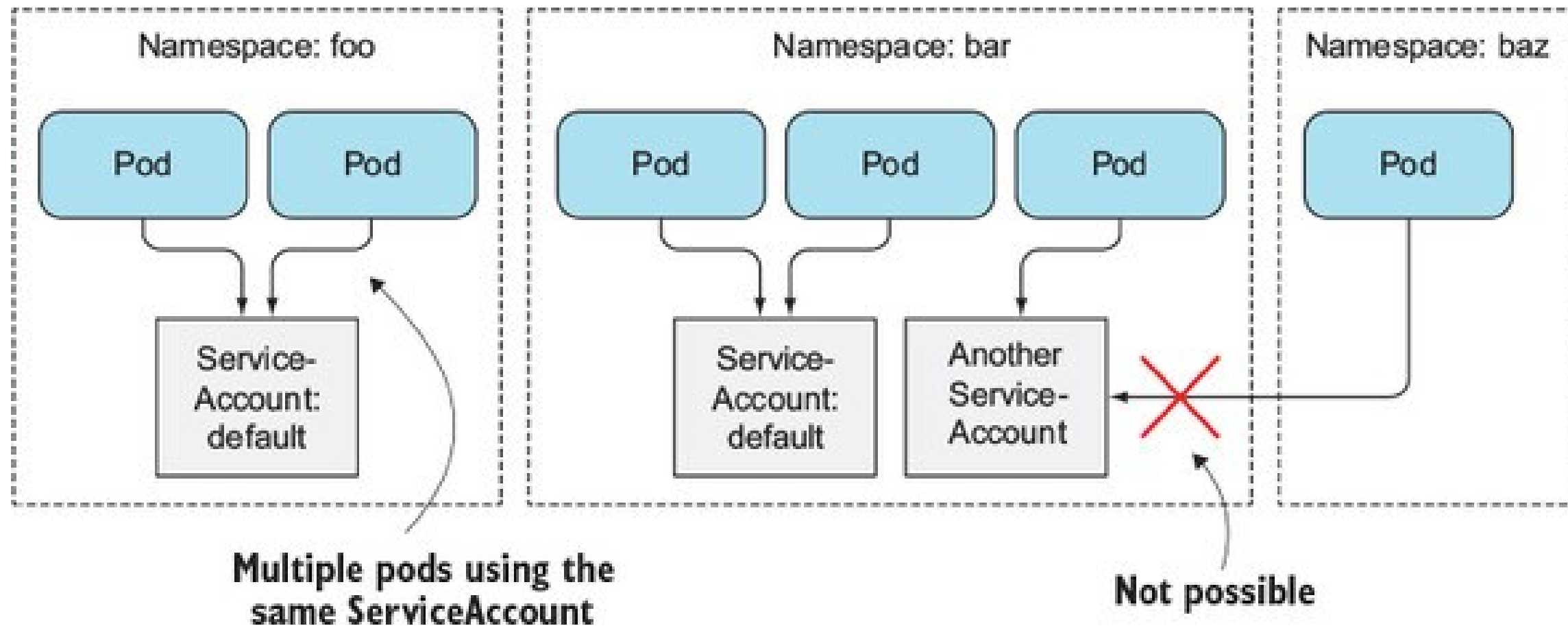


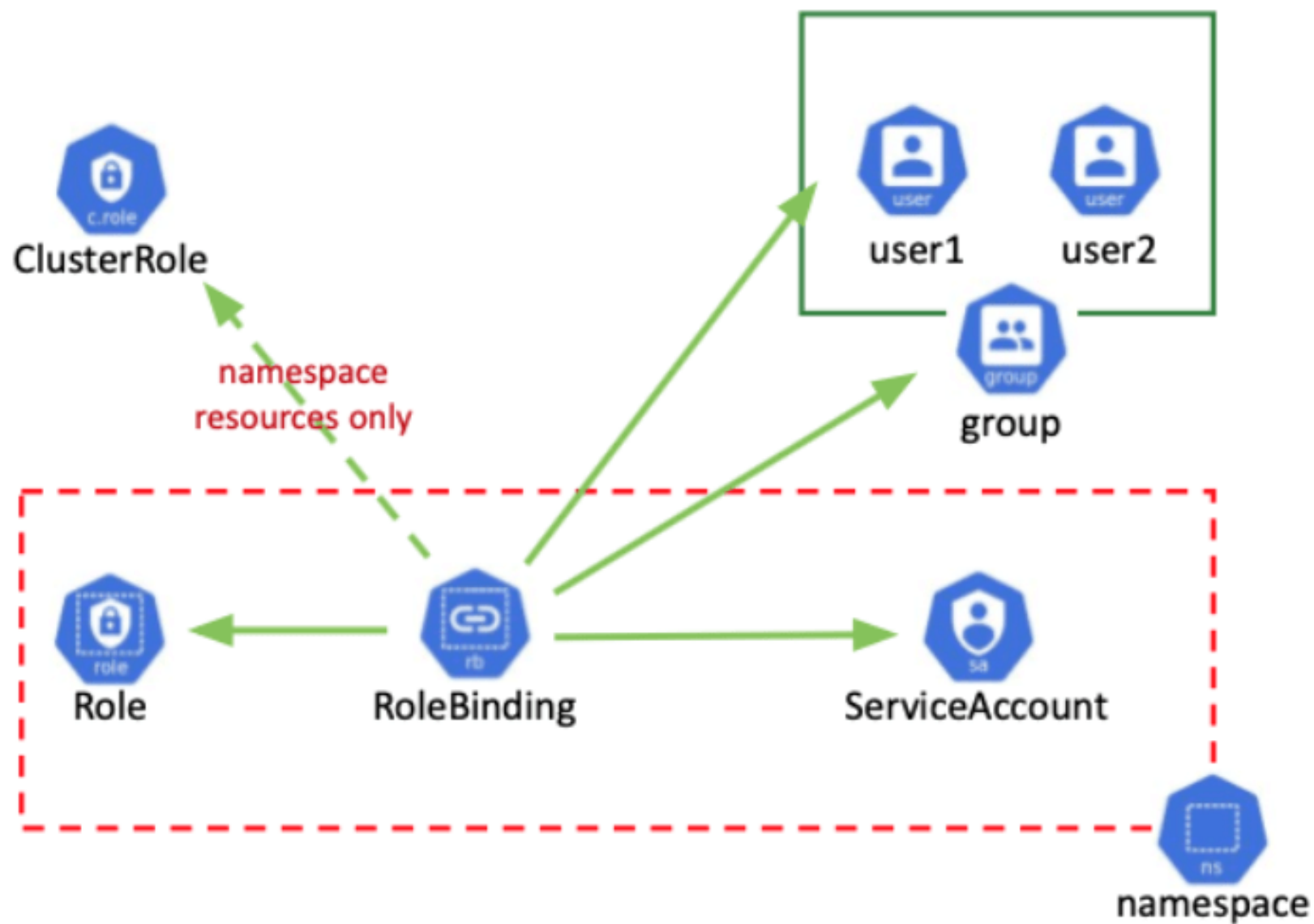
Denken Sie sich eine eigene Richtlinie in der Theorie aus, zeichnen Sie diese vielleicht Grafisch auf um Sie besser zu verdeutlichen.

ZUGRIFFSKONTROLLE

Service Accounts



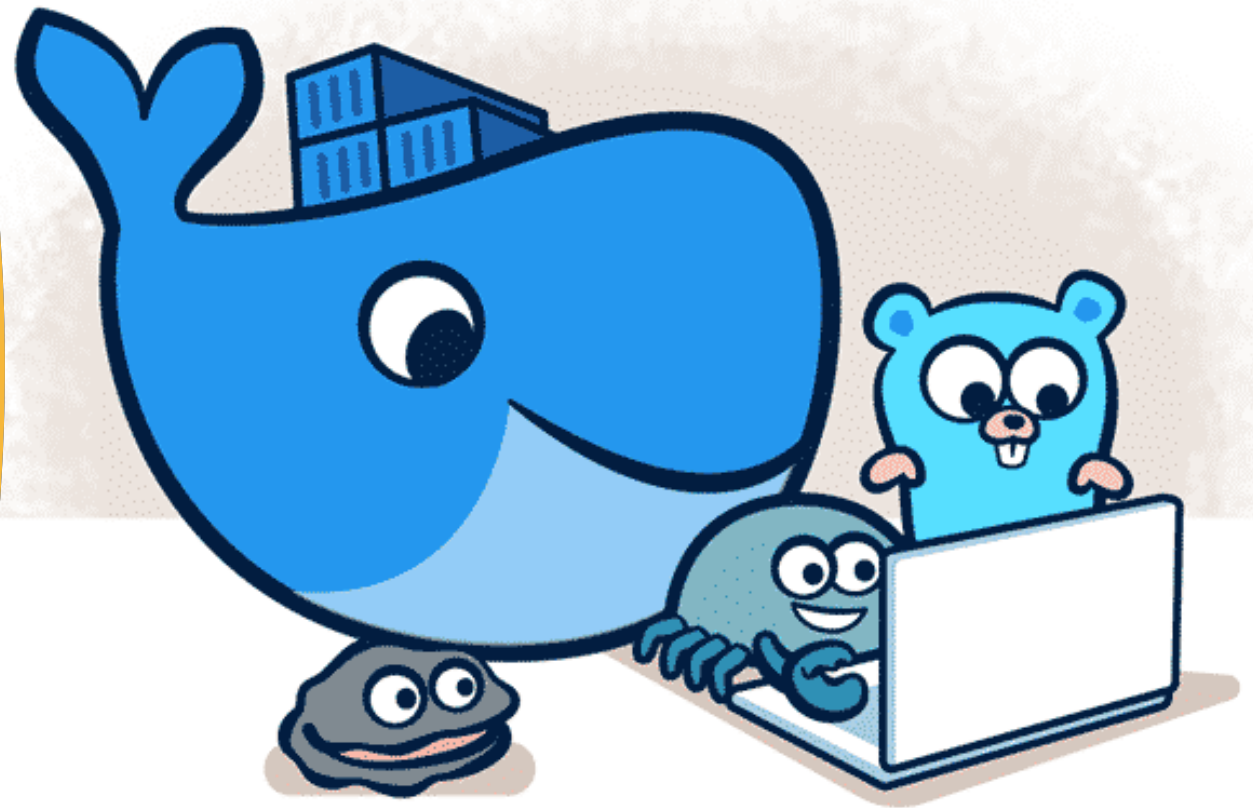




When you (a human) access the cluster (for example, using `kubectl`), you are authenticated by the apiserver as a particular User Account (currently this is usually `admin`, unless your cluster administrator has customized your cluster). Processes in containers inside pods can also contact the apiserver. When they do, they are authenticated as a particular Service Account (for example, `default`).

CLUSTER ERWEITERUNGEN

Monitoring und Logging (Fluentd,
Elastic, Prometheus)



Access logs

Apache

App logs

Frontend

Backend

System logs

syslogd

Databases

Alerting

Nagios

Analysis

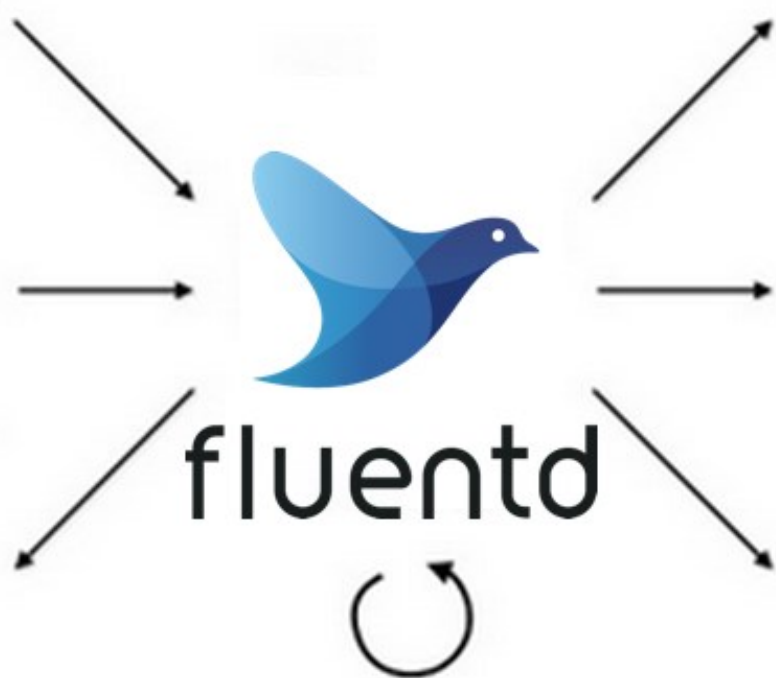
MongoDB

MySQL

Hadoop

Archiving

Amazon S3

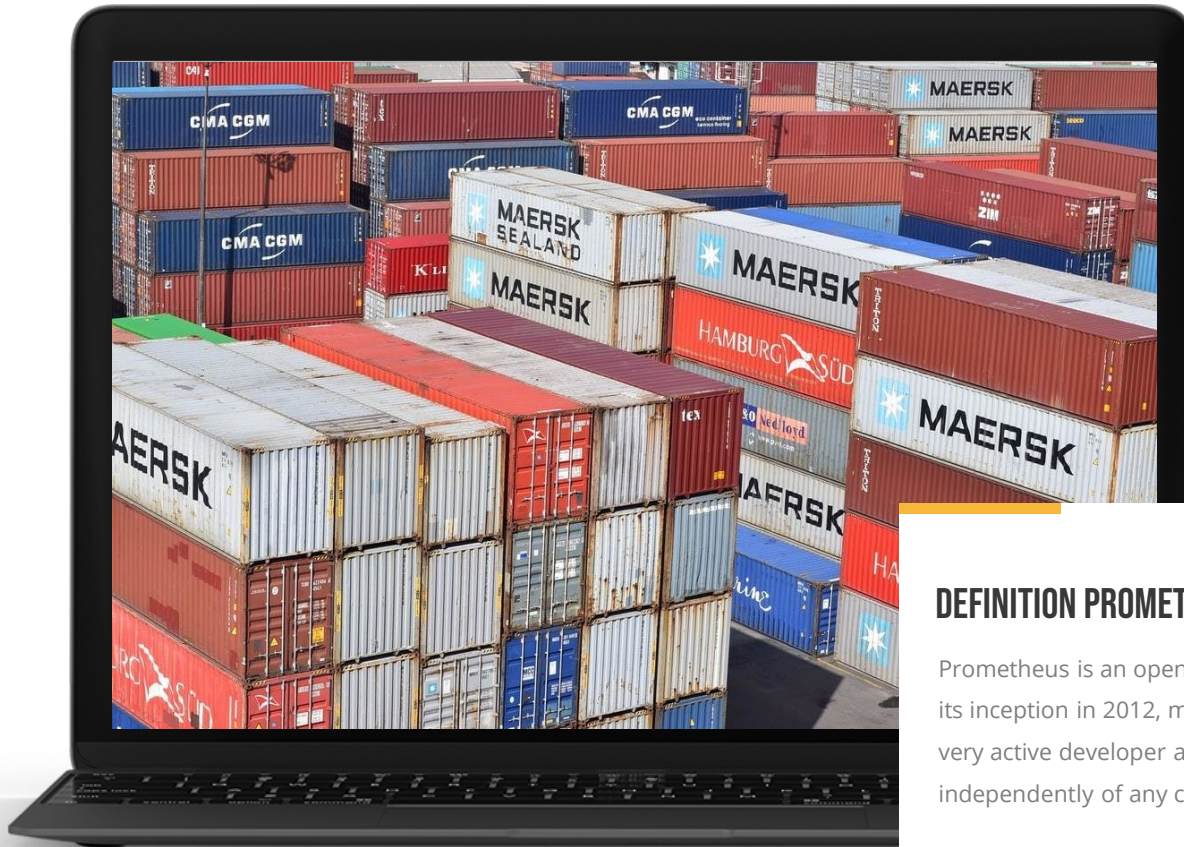


filter / buffer / routing



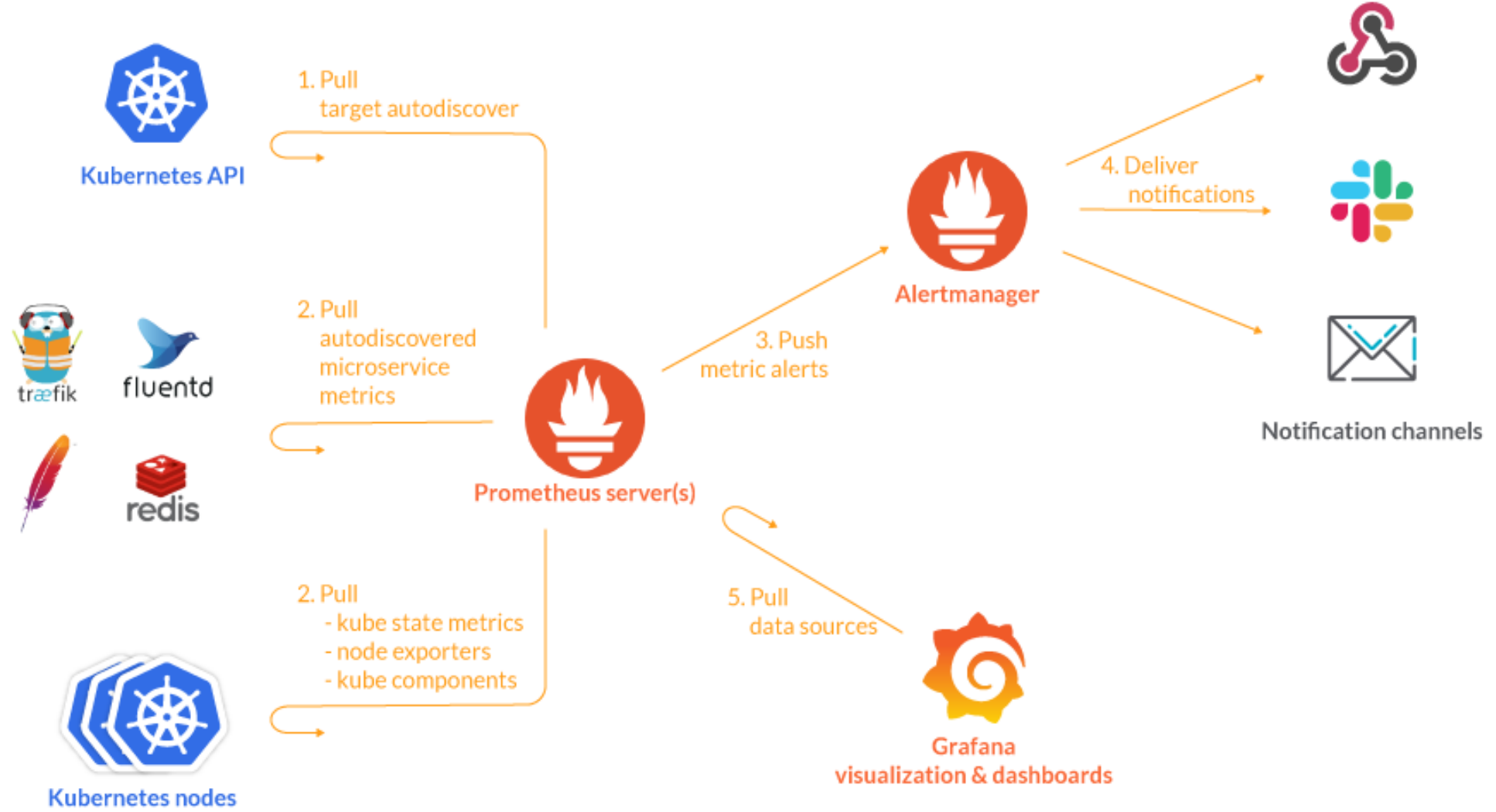
[Kubernetes](#) provides two logging end-points for applications and cluster logs: Stackdriver Logging for use with Google Cloud Platform and Elasticsearch. Behind the scenes there is a logging agent that takes care of log collection, parsing and distribution: [Fluentd](#).

The following document focuses on how to deploy Fluentd in Kubernetes and extend the possibilities to have different destinations for your logs.



DEFINITION PROMETHEUS

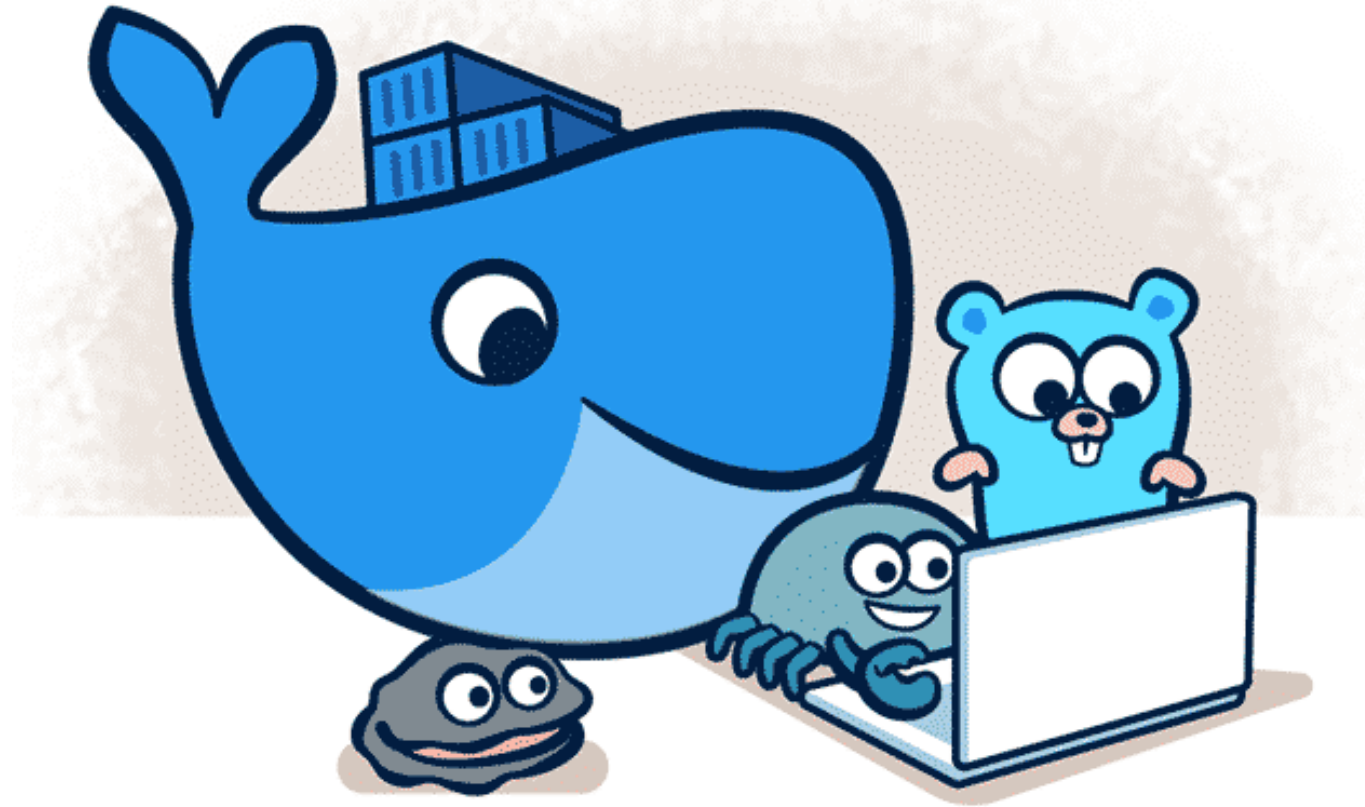
Prometheus is an open-source systems monitoring and alerting toolkit originally built at SoundCloud. Since its inception in 2012, many companies and organizations have adopted Prometheus, and the project has a very active developer and user community. It is now a standalone open source project and maintained independently of any company.



RECHERCHE

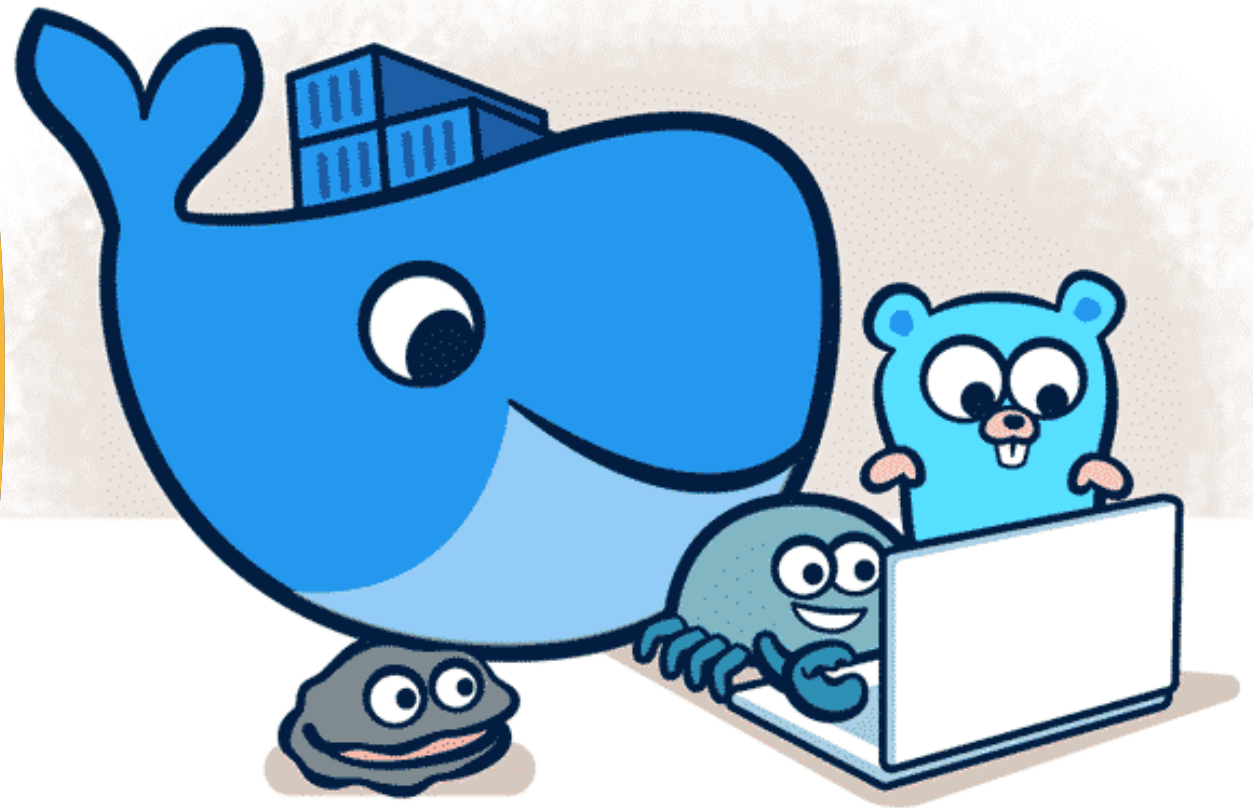
Schauen Sie sich folgenden Link an <https://github.com/prometheus-operator/kube-prometheus> und versuchen Sie zu verstehen in wie fern hier Prometheus angesetzt wird. Finden Sie zudem Beispiele für was Prometheus eingesetzt werden kann und für welche Fälle es besonders gut geeignet ist.

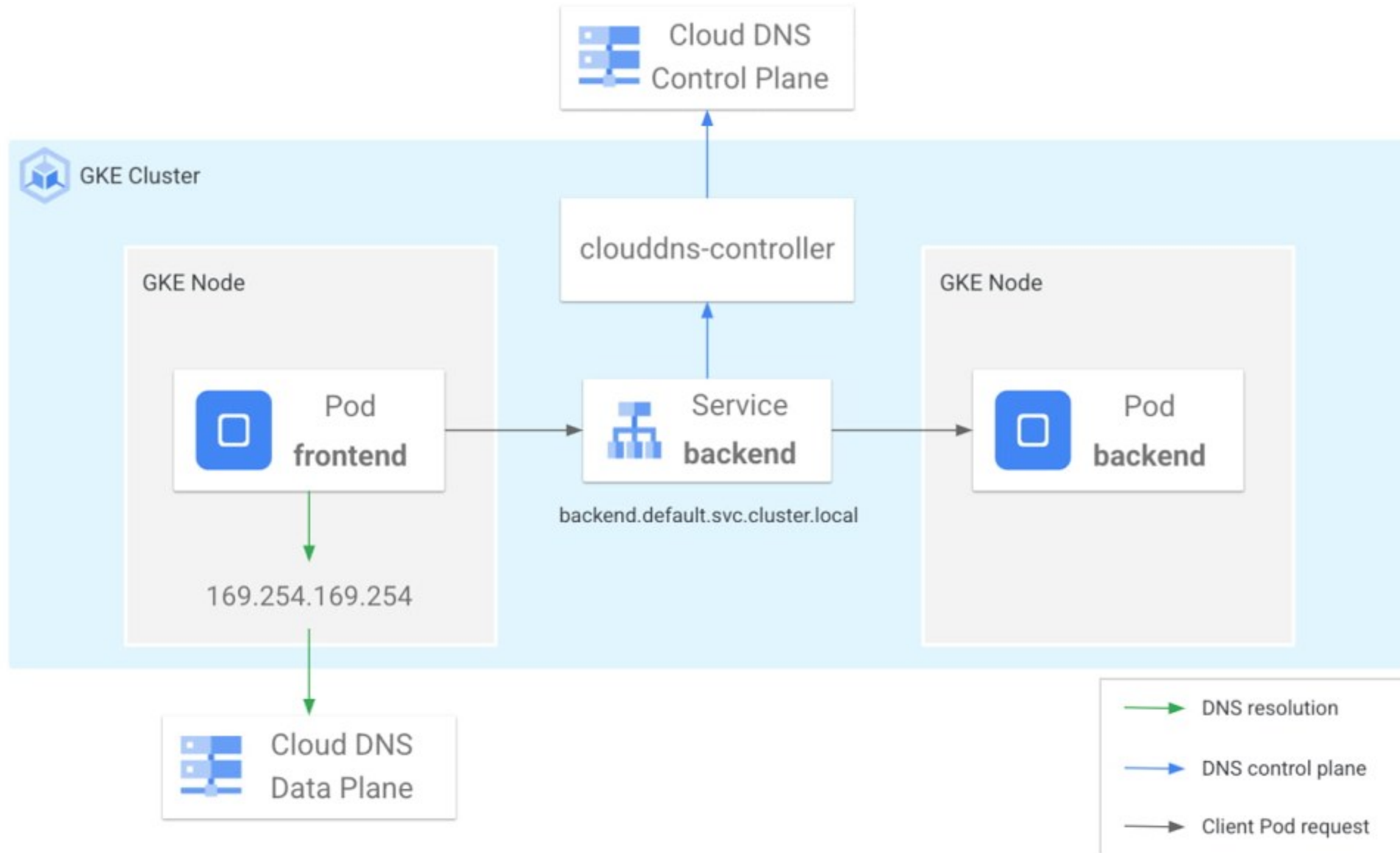
02.



CLUSTER ERWEITERUNGEN

Cluster DNS





CLUSTER DNS

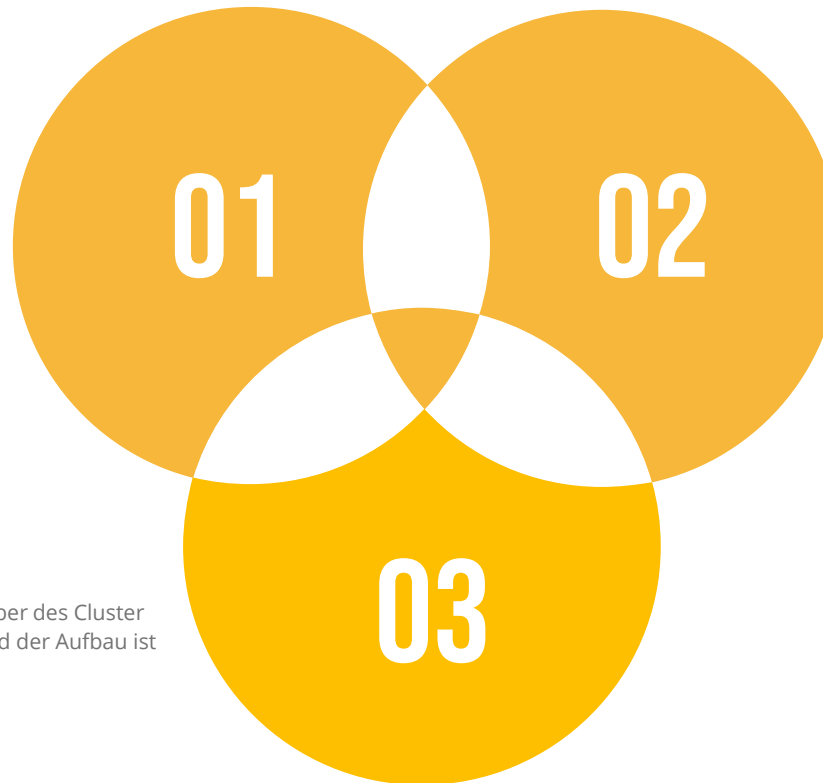
Warum ist diese Thematik so wichtig?

VEREINFACHUNG

Mittels eines Cluster DNS ist es nicht mehr nötig die IP Adresse eines Containers zu wissen, der DNS Name reicht aus.

VERSTÄNDNIS

Für die Entwickler und die Betreiber des Cluster wird die Komplexität geringer und der Aufbau ist einfacher zu verstehen.

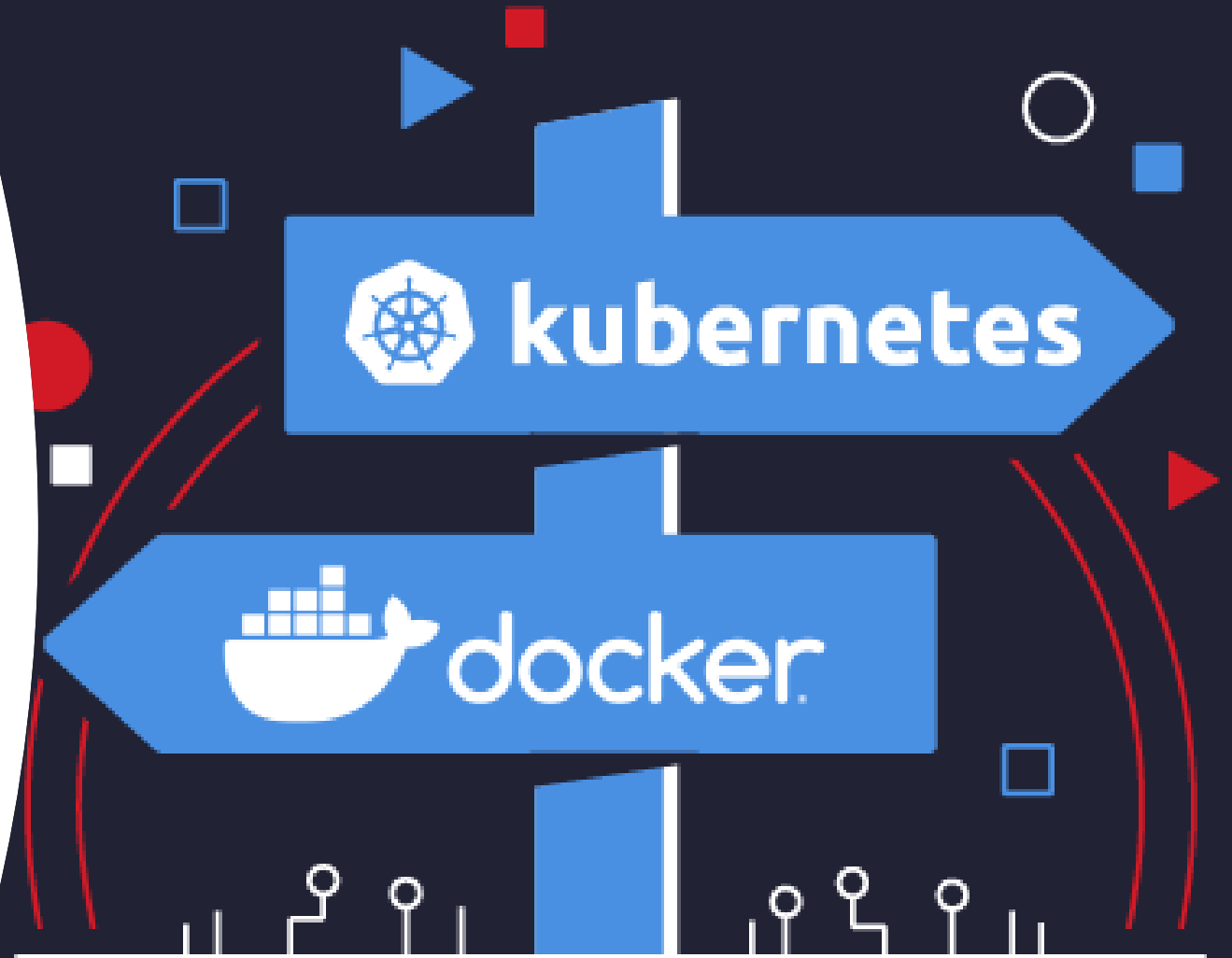


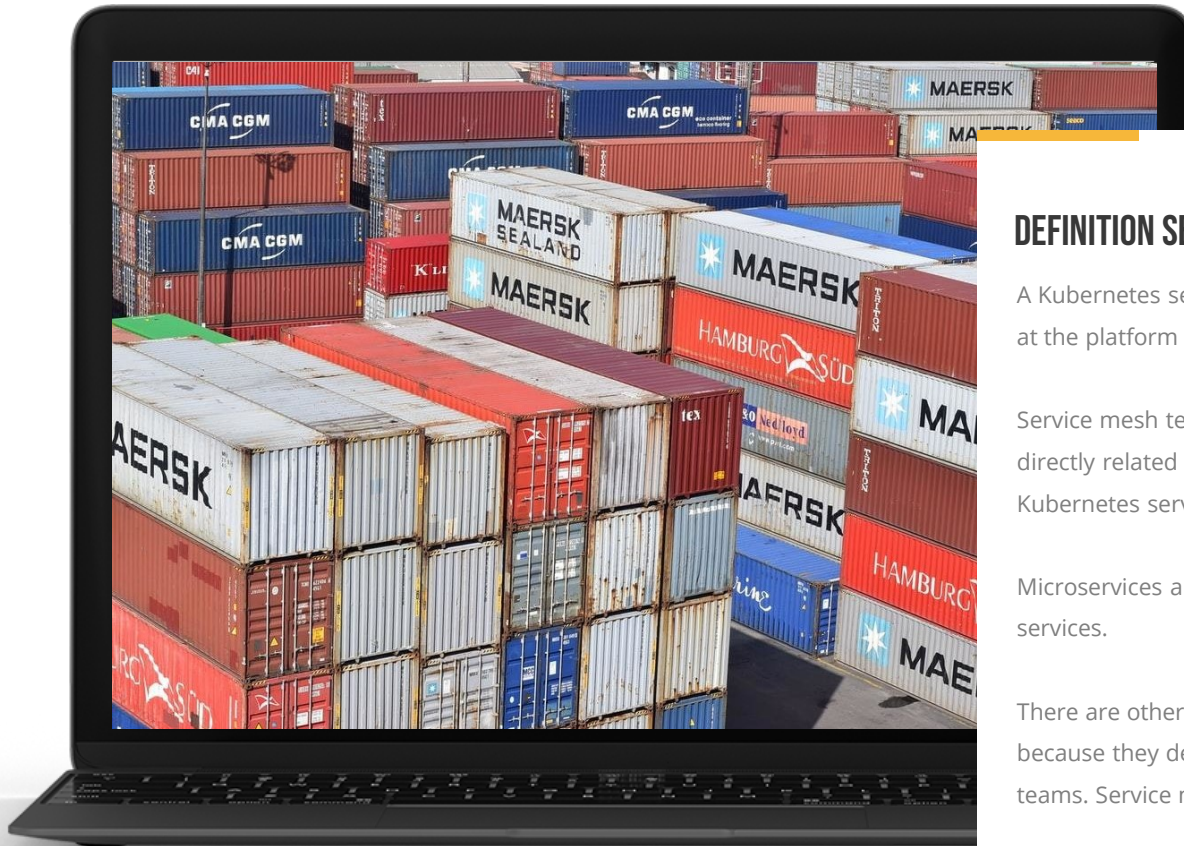
BENUTZUNG

Die Benutzung wird somit vereinfacht, da Sie nur noch über den DNS Name kommunizieren kann theoretisch der Container jede IP bekommen die er braucht.

NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

Service Mesh





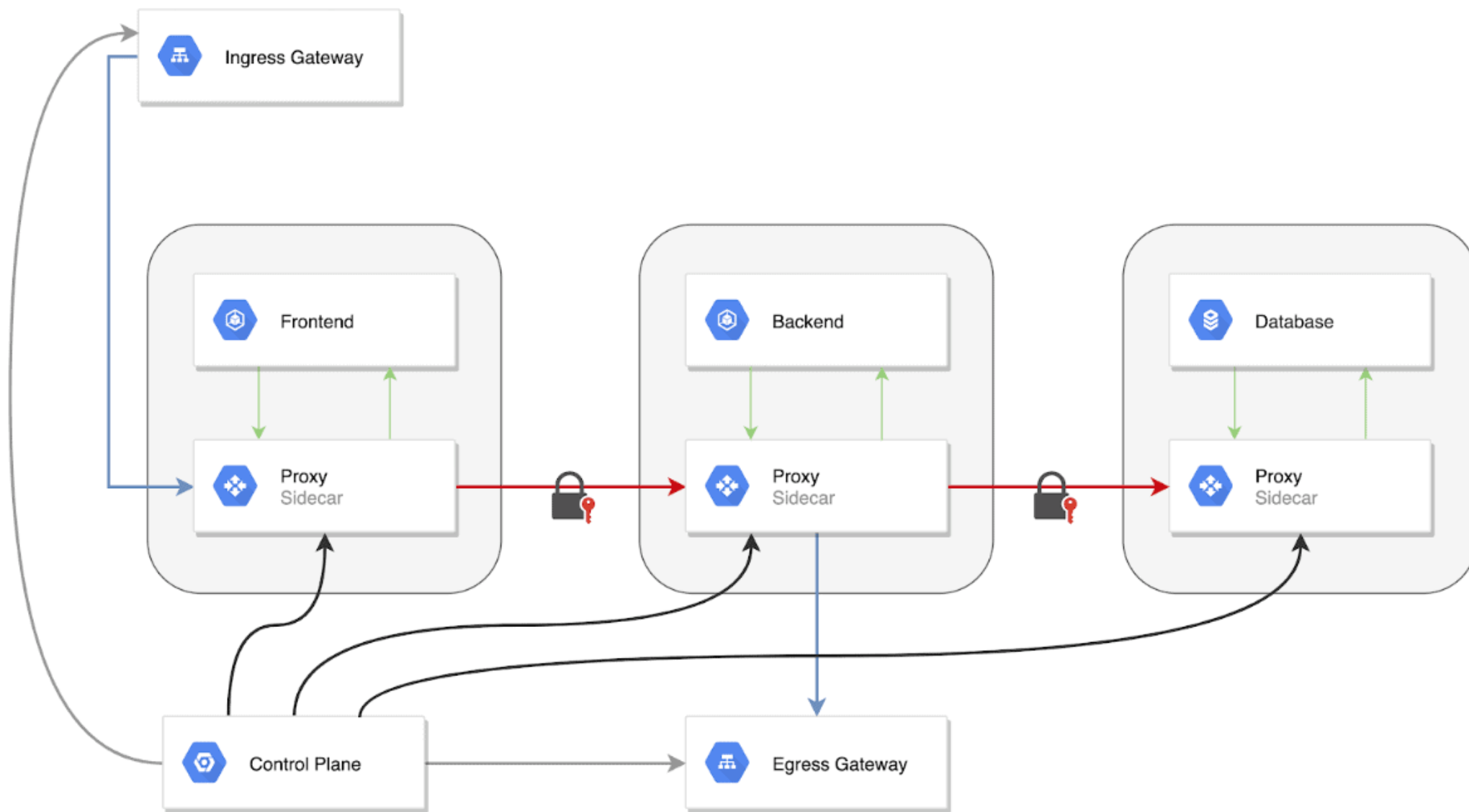
DEFINITION SERVICE MESH

A Kubernetes service mesh is a tool that inserts security, observability, and reliability features to applications at the platform layer instead of the application layer.

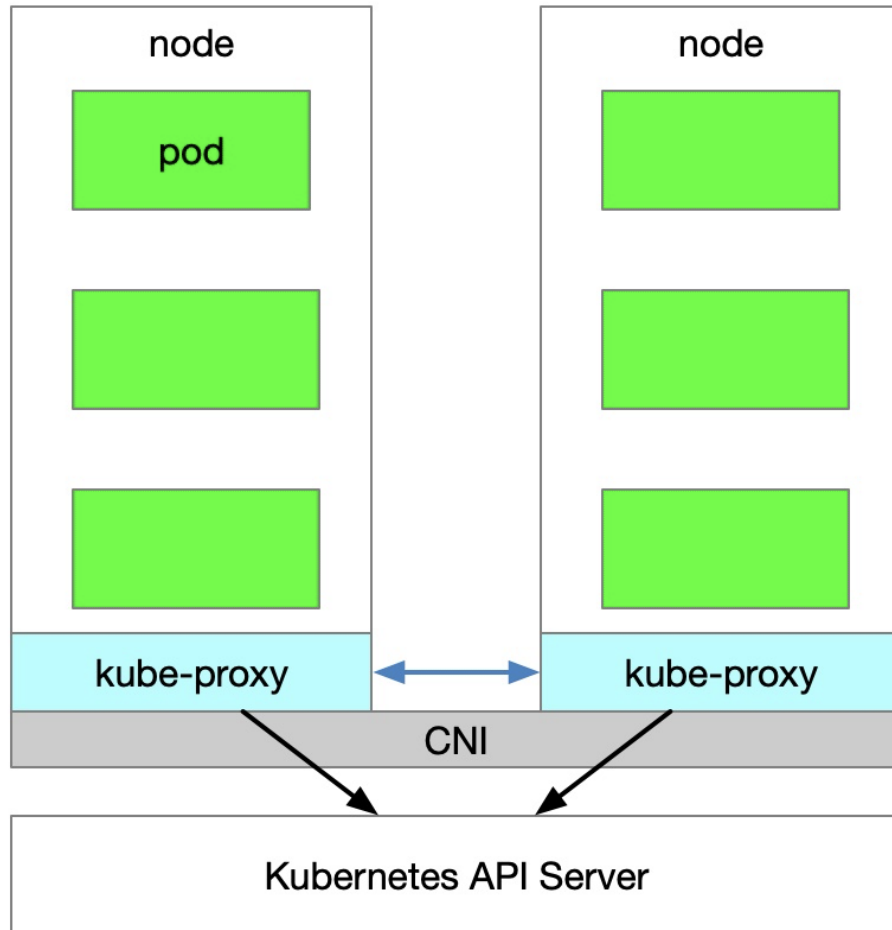
Service mesh technology predates Kubernetes. However, growing interest in service mesh solutions is directly related to the proliferation of Kubernetes-based microservices and a resulting interest in Kubernetes service mesh options.

Microservices architectures are heavily network reliant. Service mesh manages network traffic between services.

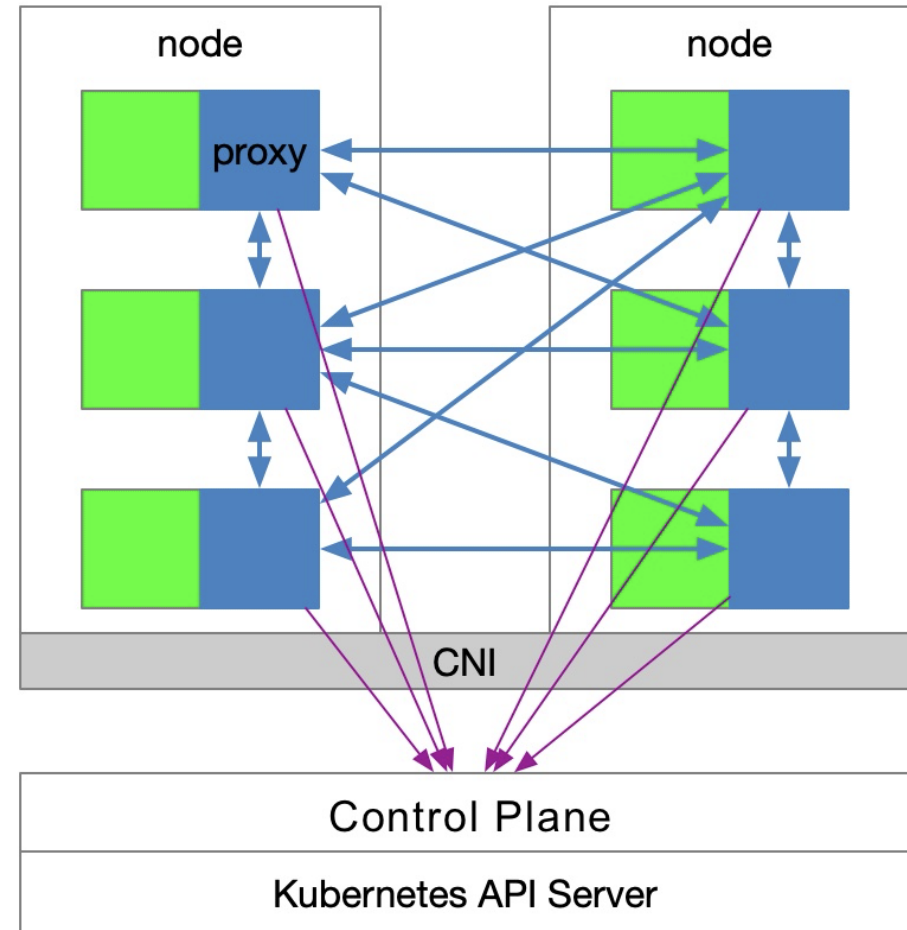
There are other ways to manage this network traffic, but they are less sustainable than service mesh because they demand more operational burden in the form of error-prone, manual labor from devops teams. Service mesh on Kubernetes achieves the same goal in a much more scalable manner.



Kubernetes Native



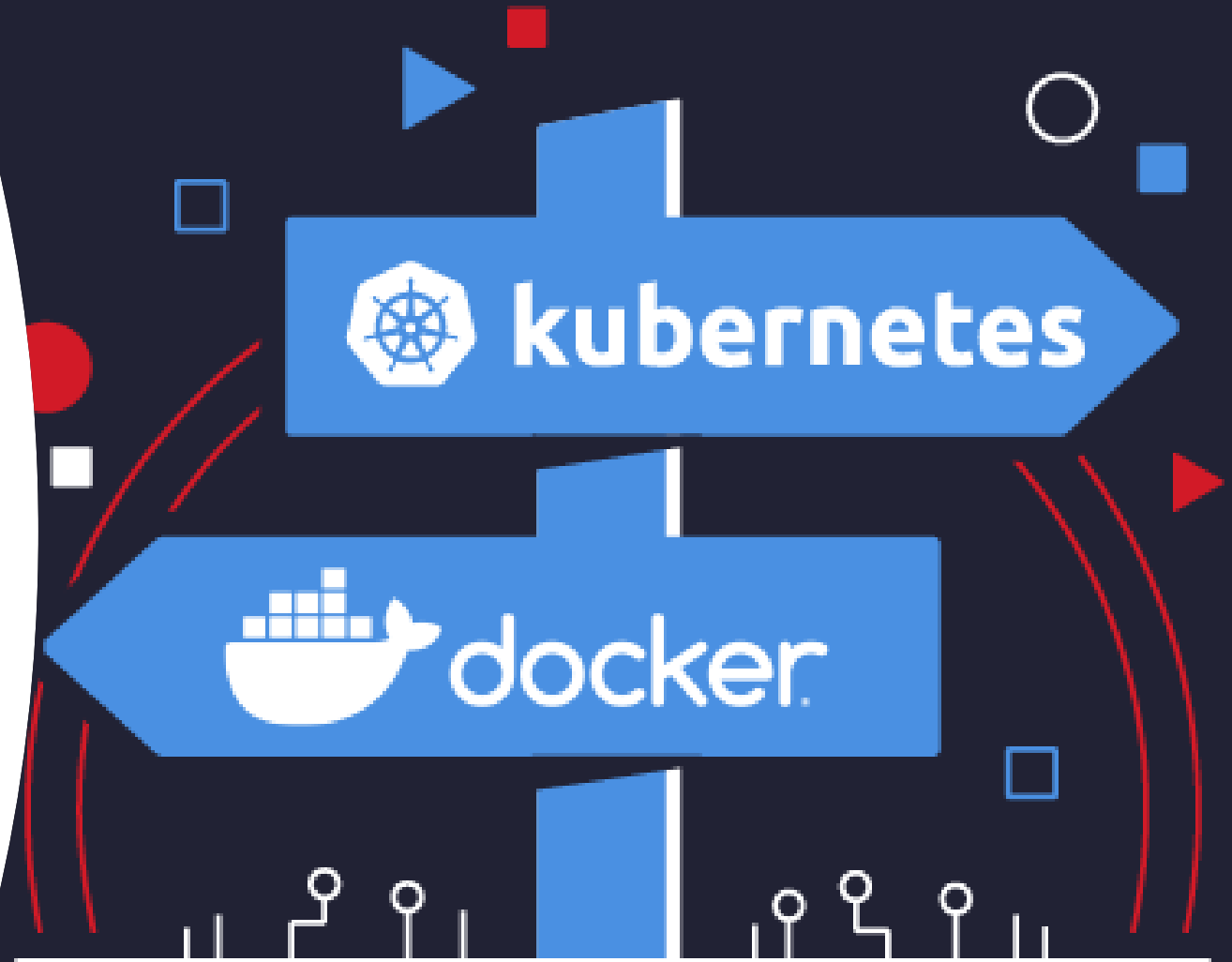
Service Mesh (One sidecar per pod)



NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

Realisierungen:

- IstIO
- Envoy



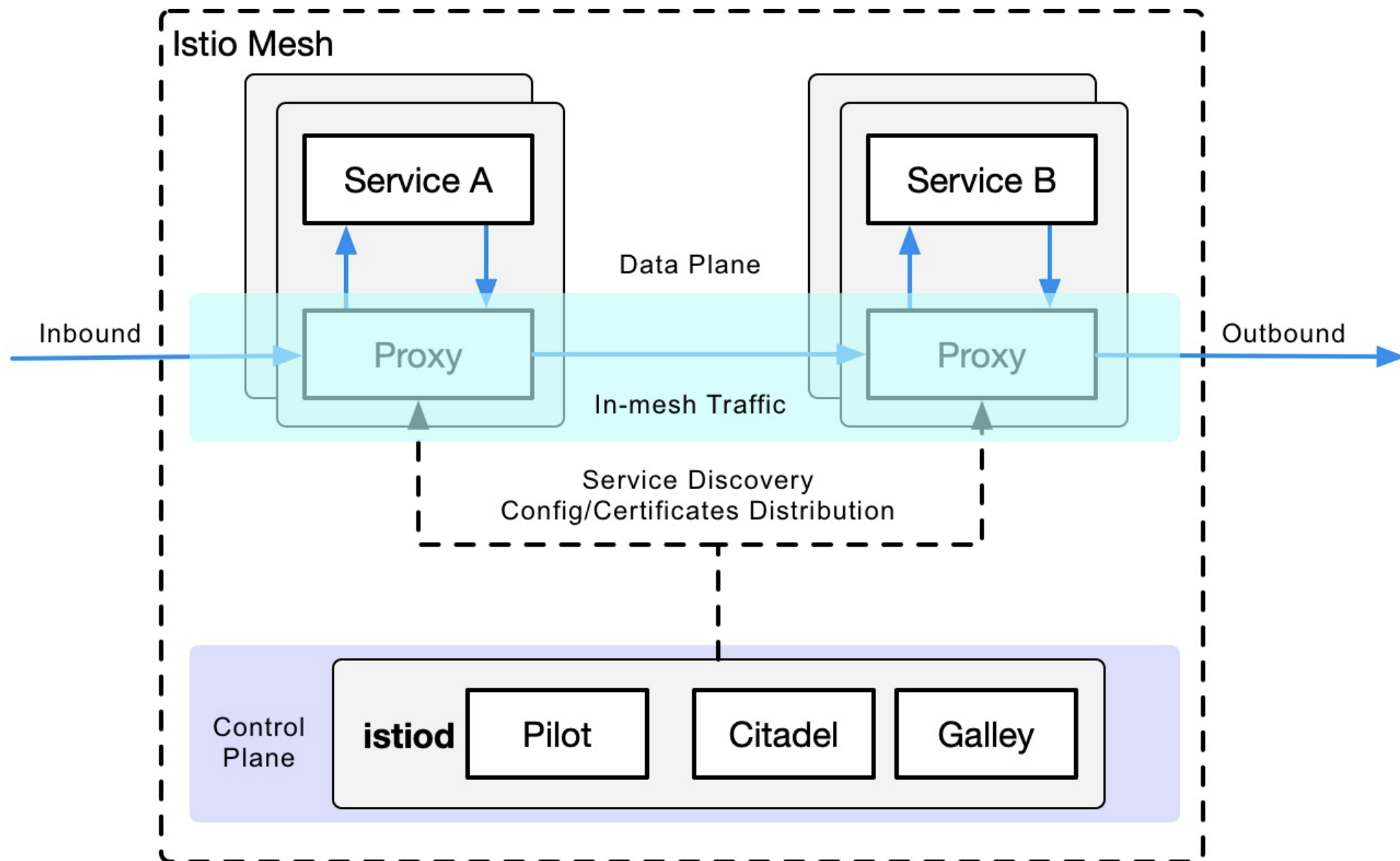
ISTIO

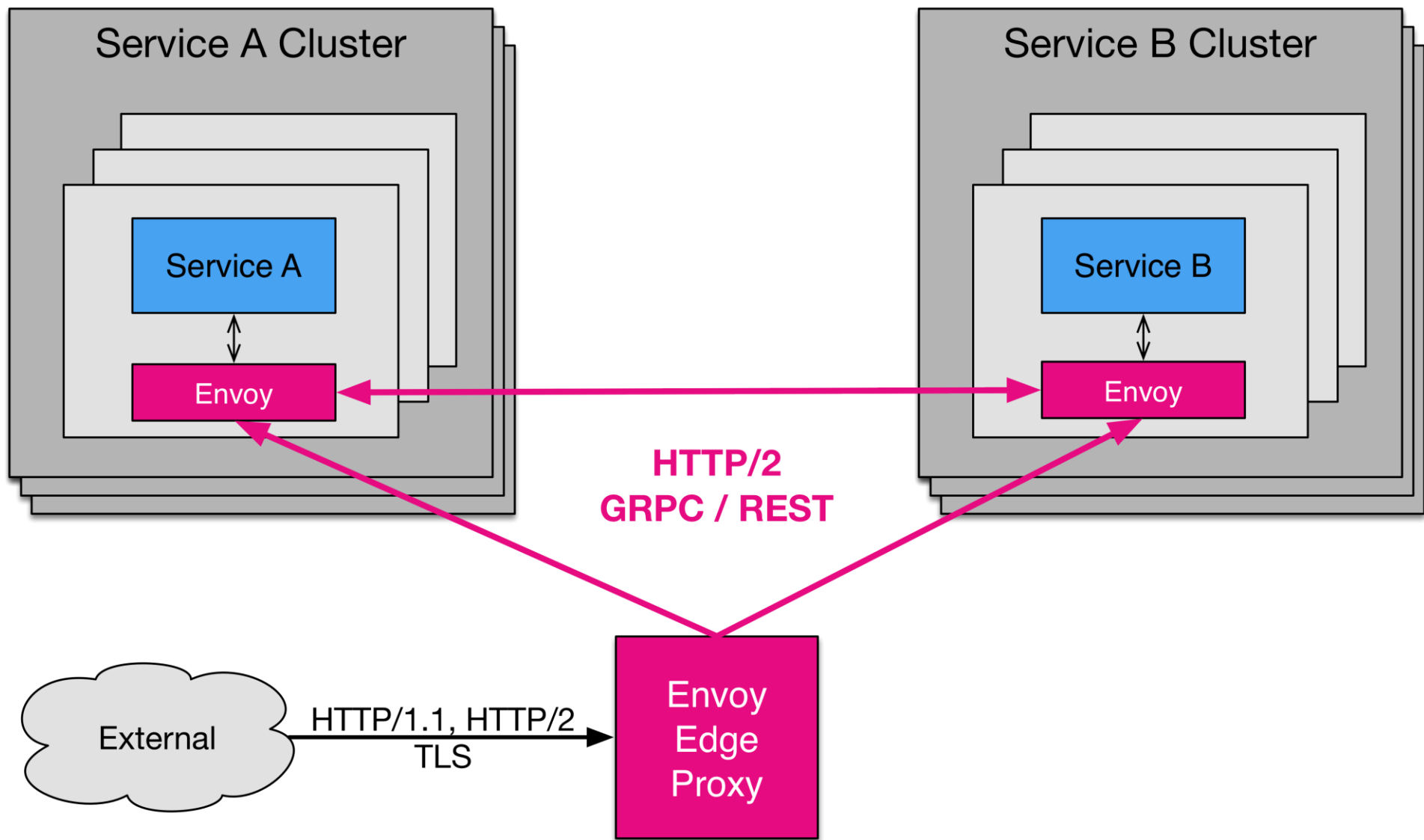
Istio is an open source service mesh that layers transparently onto existing distributed applications. Istio's powerful features provide a uniform and more efficient way to secure, connect, and monitor services. Istio is the path to load balancing, service-to-service authentication, and monitoring – with few or no service code changes. Its powerful control plane brings vital features, including:

- Secure service-to-service communication in a cluster with TLS encryption, strong identity-based authentication and authorization
- Automatic load balancing for HTTP, gRPC, WebSocket, and TCP traffic
- Fine-grained control of traffic behavior with rich routing rules, retries, failovers, and fault injection
- A pluggable policy layer and configuration API supporting access controls, rate limits and quotas
- Automatic metrics, logs, and traces for all traffic within a cluster, including cluster ingress and egress

ENVOY

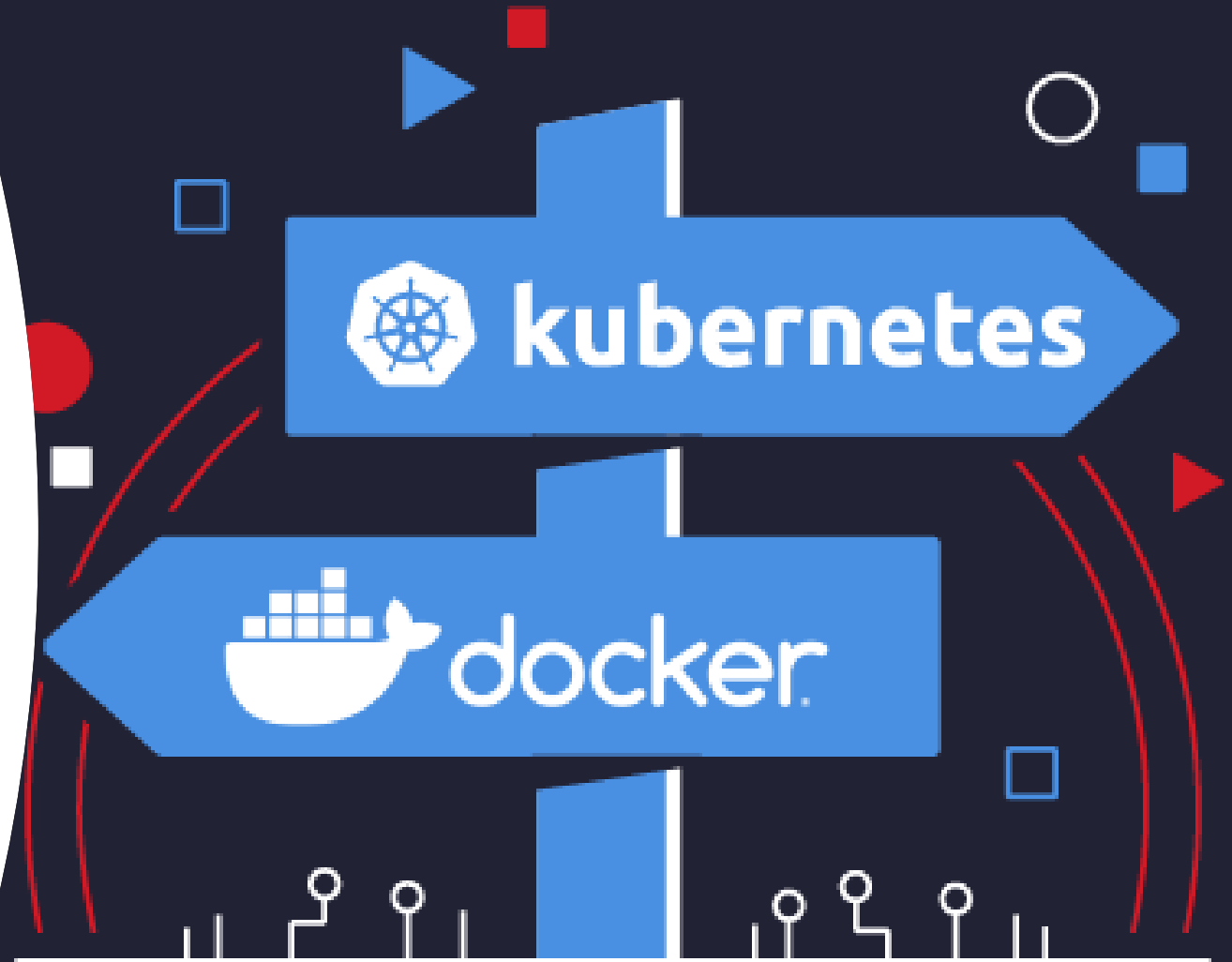
Originally built at Lyft, Envoy is a high performance C++ distributed proxy designed for single services and applications, as well as a communication bus and “universal data plane” designed for large microservice “service mesh” architectures. Built on the learnings of solutions such as NGINX, HAProxy, hardware load balancers, and cloud load balancers, Envoy runs alongside every application and abstracts the network by providing common features in a platform-agnostic manner. When all service traffic in an infrastructure flows via an Envoy mesh, it becomes easy to visualize problem areas via consistent observability, tune overall performance, and add substrate features in a single place.





NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

Möglichkeiten von Service Meshes



MÖGLICHKEITEN

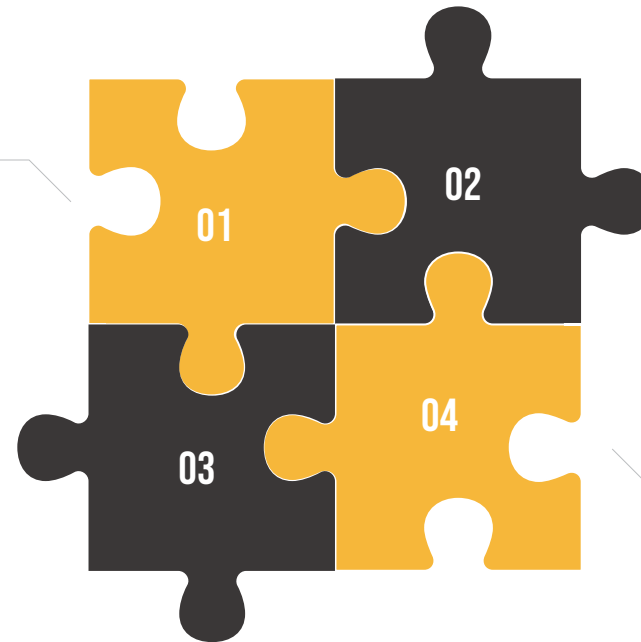
Auswahl an Möglichkeiten 1/2

VEREINFACHTE BEREITSTELLUNG

Mittels Meshes ist es möglich sehr schnell Container, Pods und Services miteinander zu verbinden

ERPROBTE TECHNOLOGIE

Meshes sind in der Entwicklung und Bereitstellung weit erprobt und können daher sicher eingesetzt werden



WENIGER MANUELLE KONFIGURATION

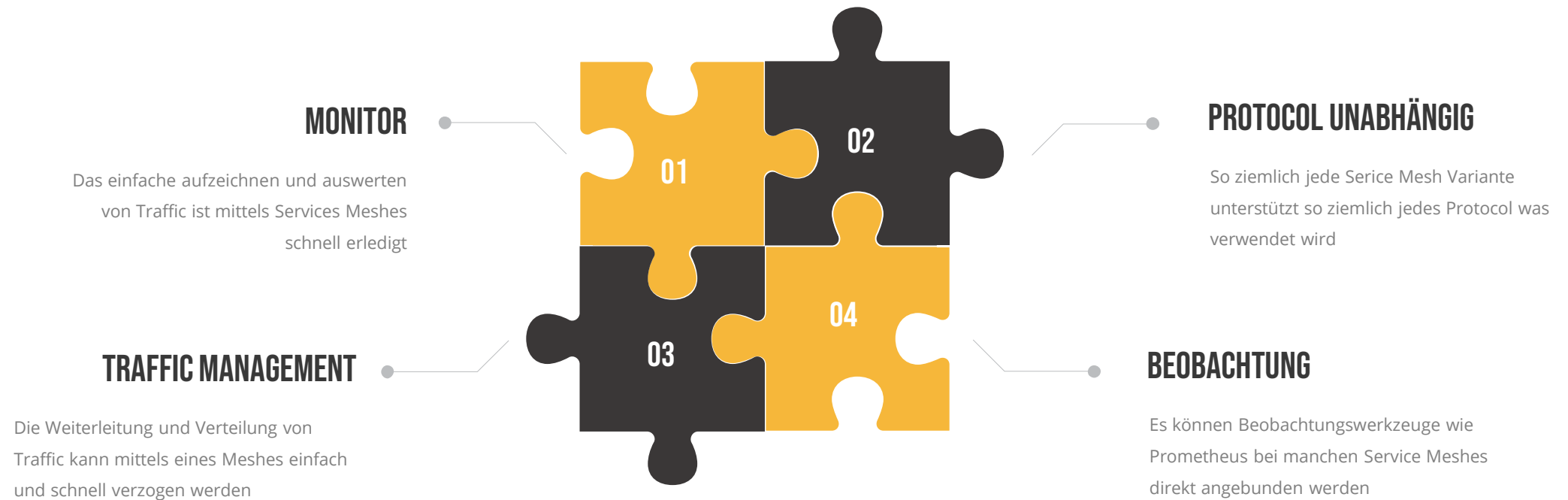
Für die Bereitstellung ist kaum manueller Aufwand nötig da das Mesh dies intern regelt

STANDARDISIERTE SICHERHEIT

Meshes werden von vielen großen Unternehmen verwendet und sind daher sehr standardisiert

MÖGLICHKEITEN

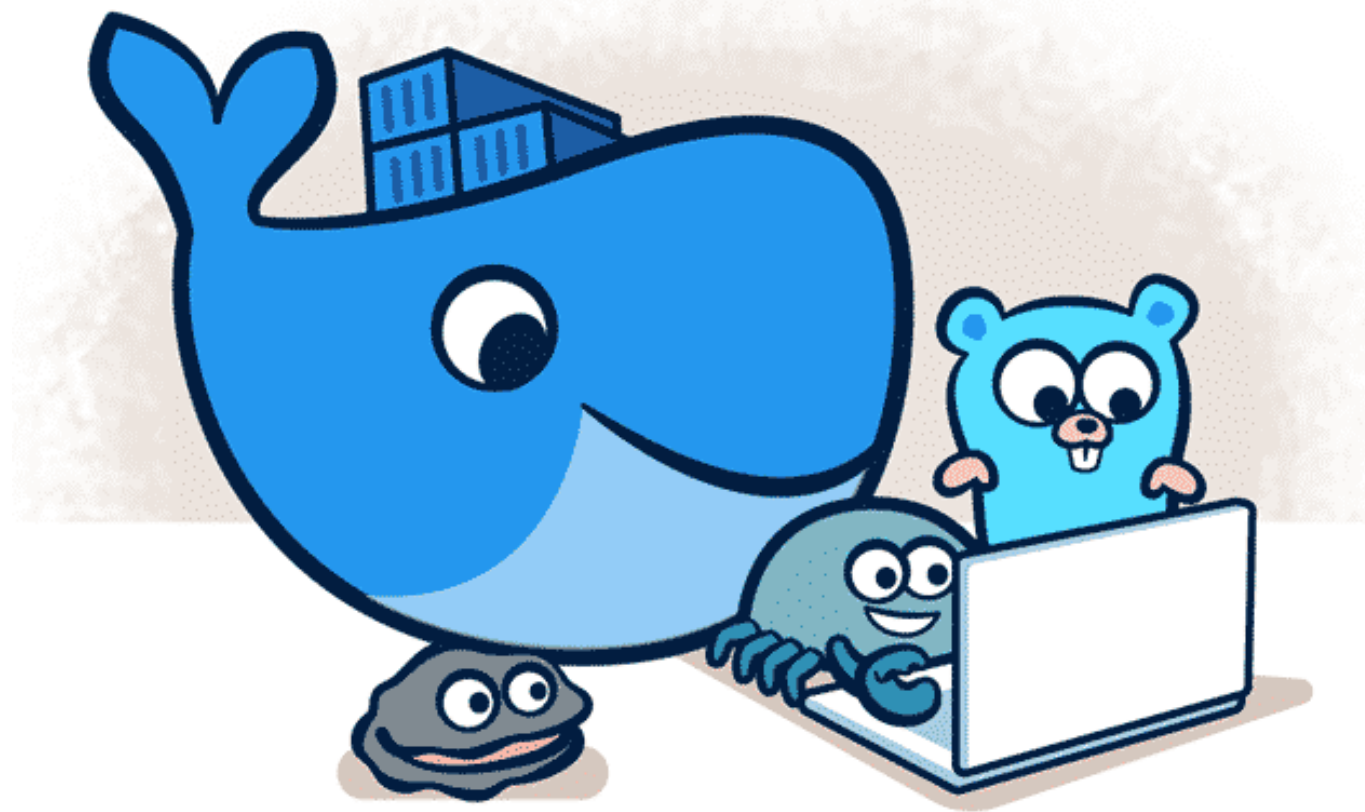
Auswahl an Möglichkeiten 2/2



RECHERCHE

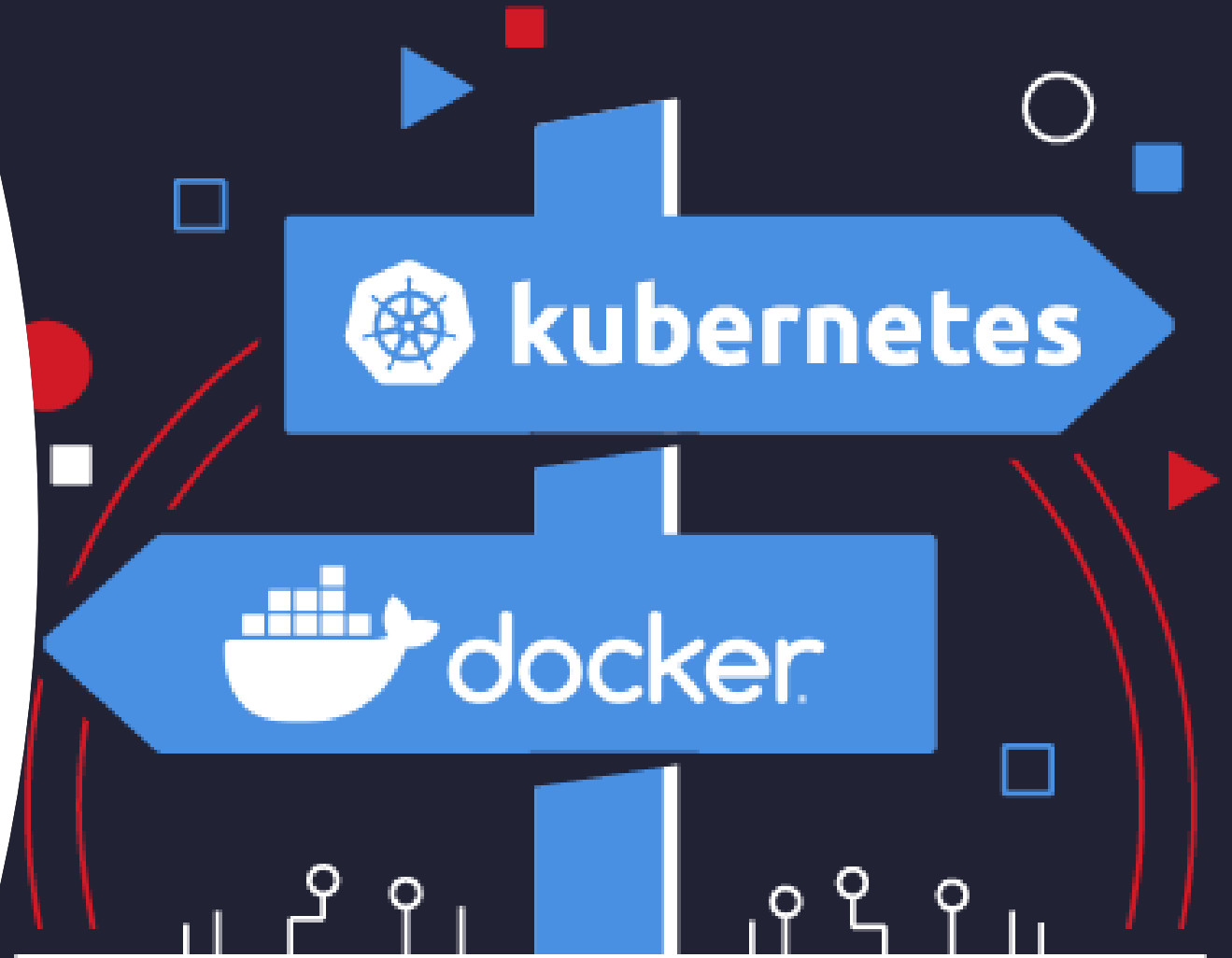
Denken Sie sich in der Gruppe eine Auswahl an Anwendungsbeispielen aus wo Service Meshes verwendet werden könnten und wo diese einen großen Vorteil bieten würden.

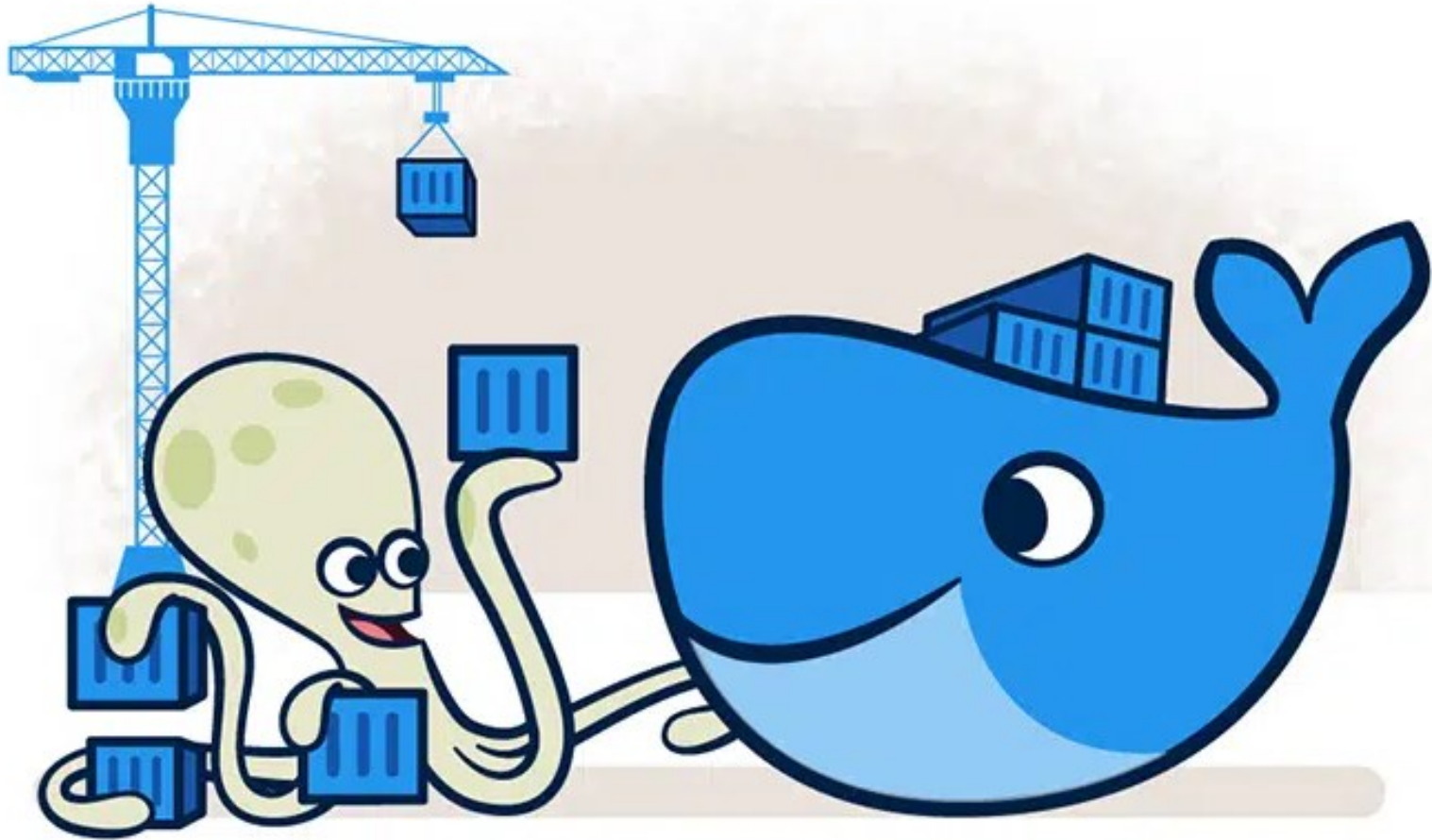
02.



NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

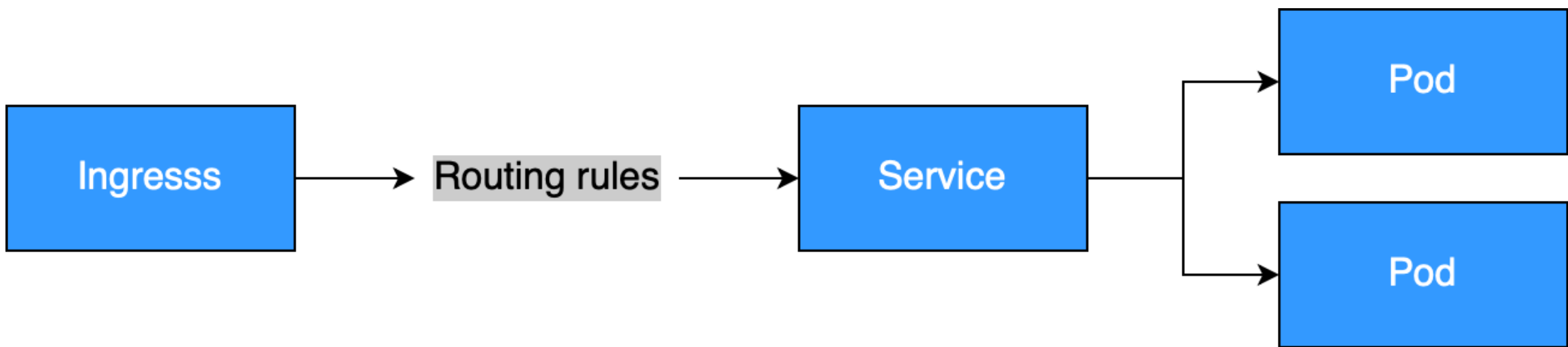
Canary Deployment

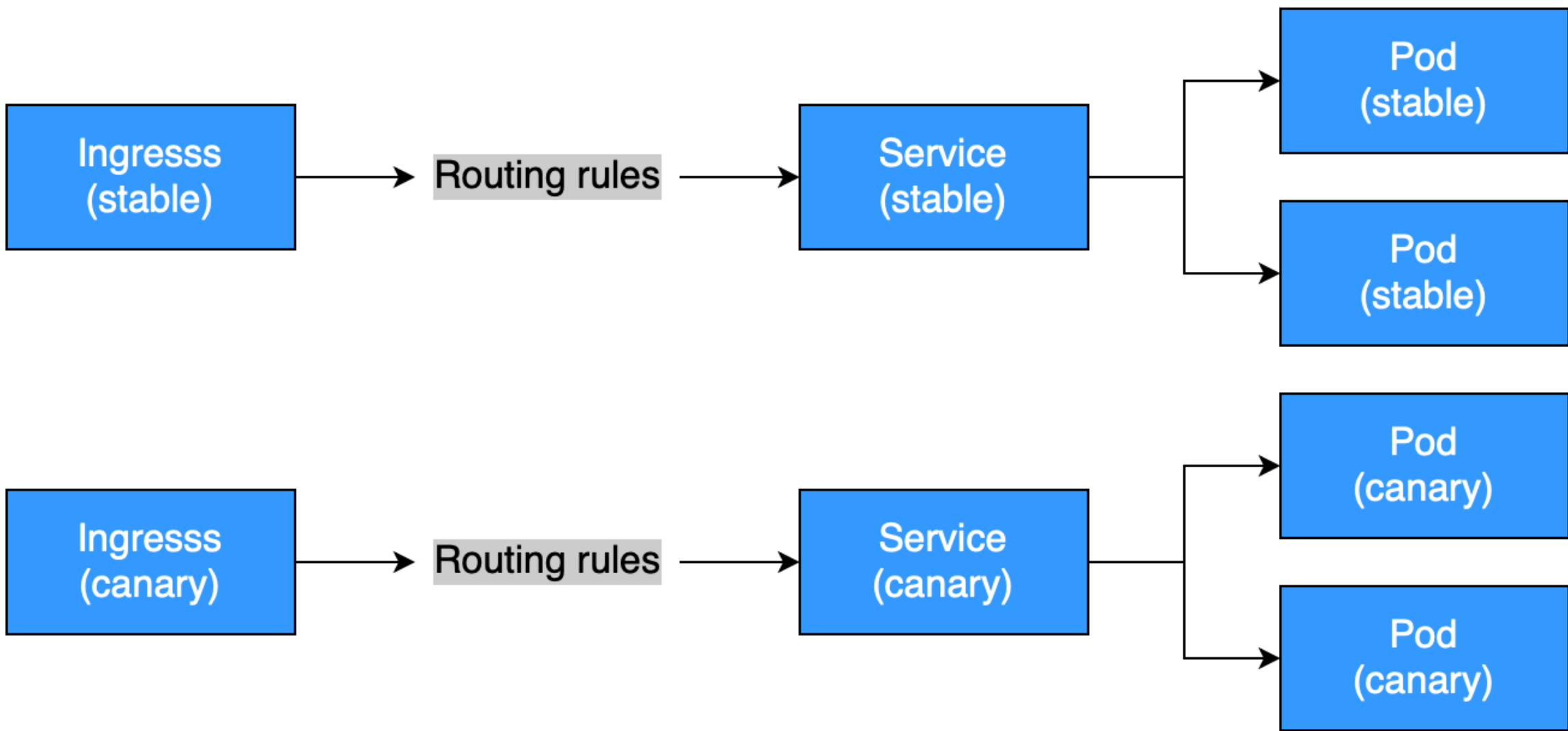


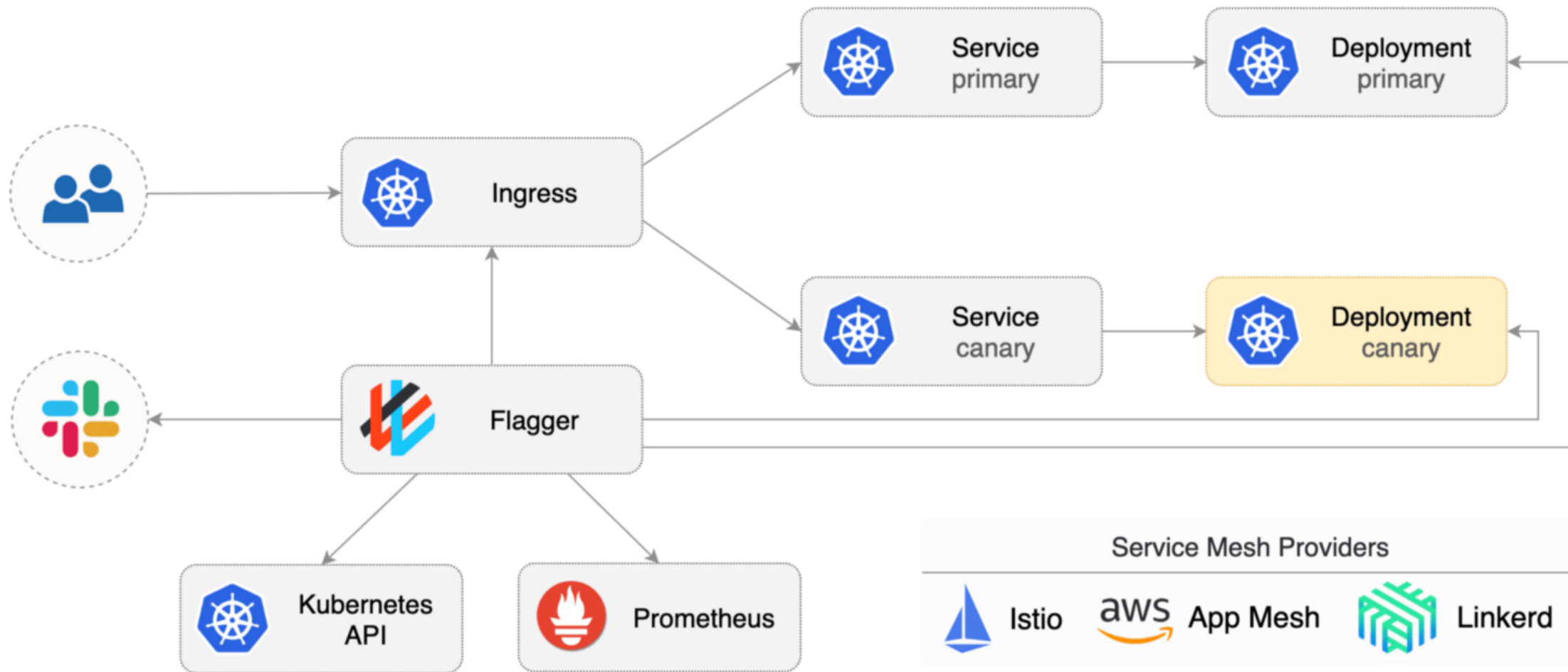


WAS IST SIND CANARY DEPLOYMENTS

They are a way to release updates to a shifting percentage of the total user base. What's alluring about that is that when new code isn't really ready to reach production, due to undetected bugs during the testing phase, performance issues, or other factors, it'll only affect a small percentage of the users before being detected and rolled back.

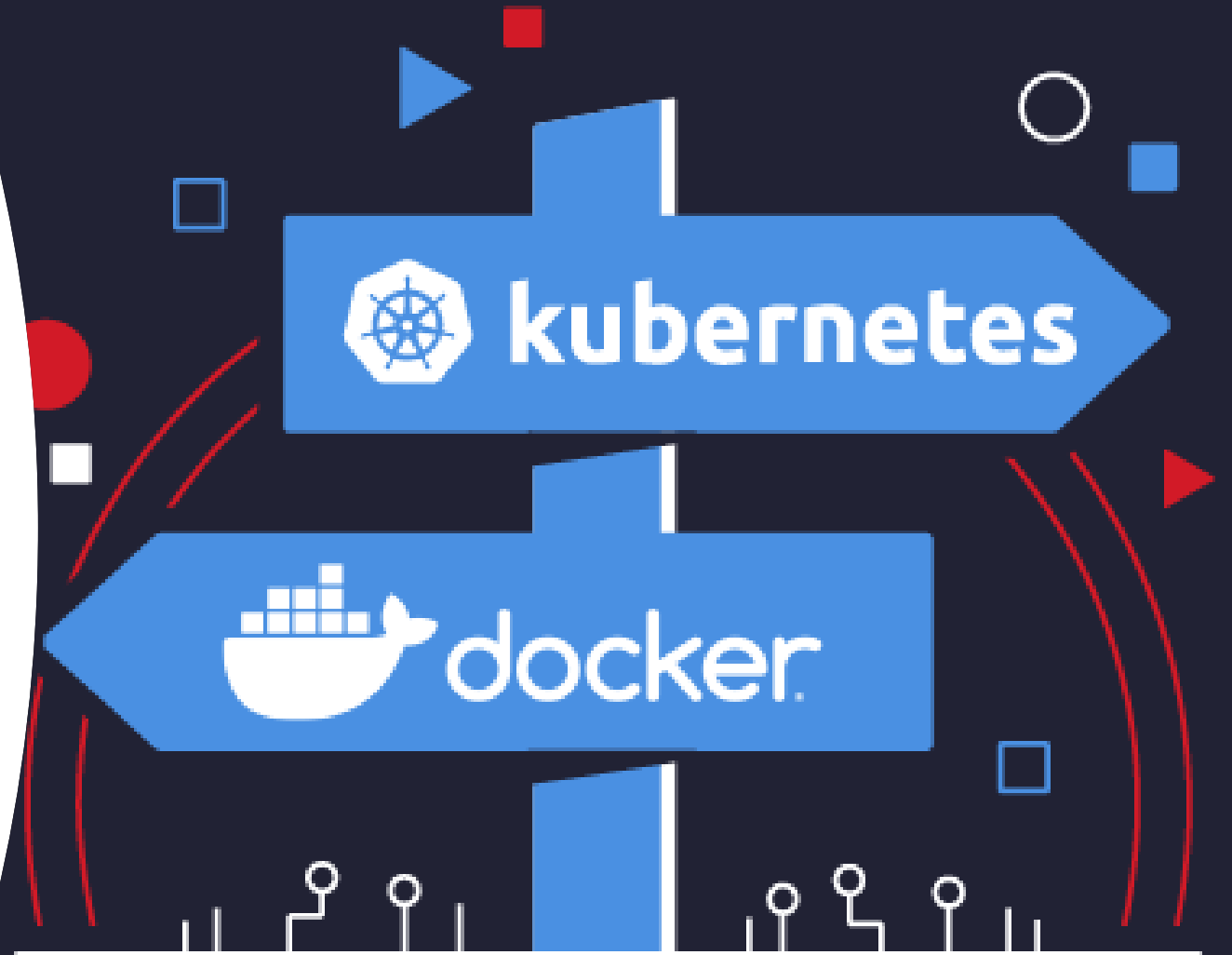




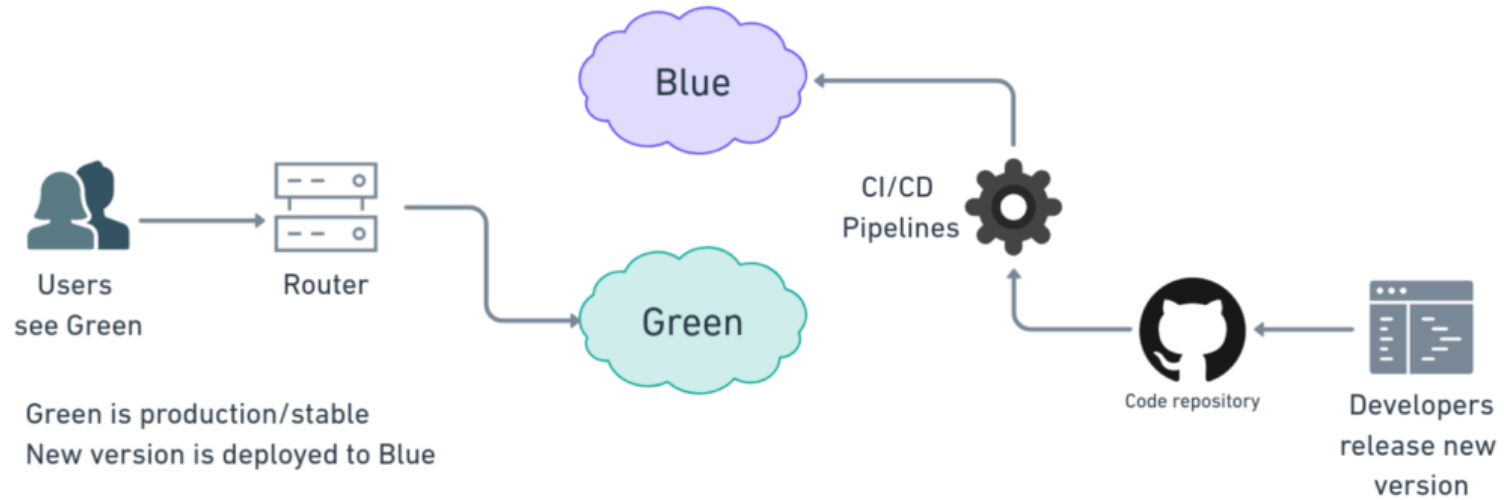
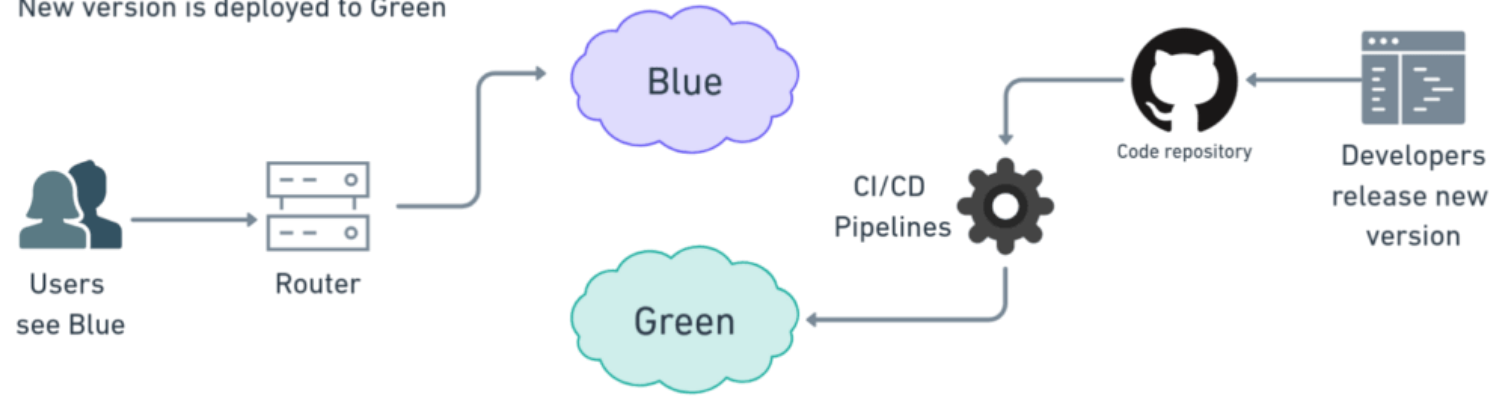


NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

Blue- / Green Deployment

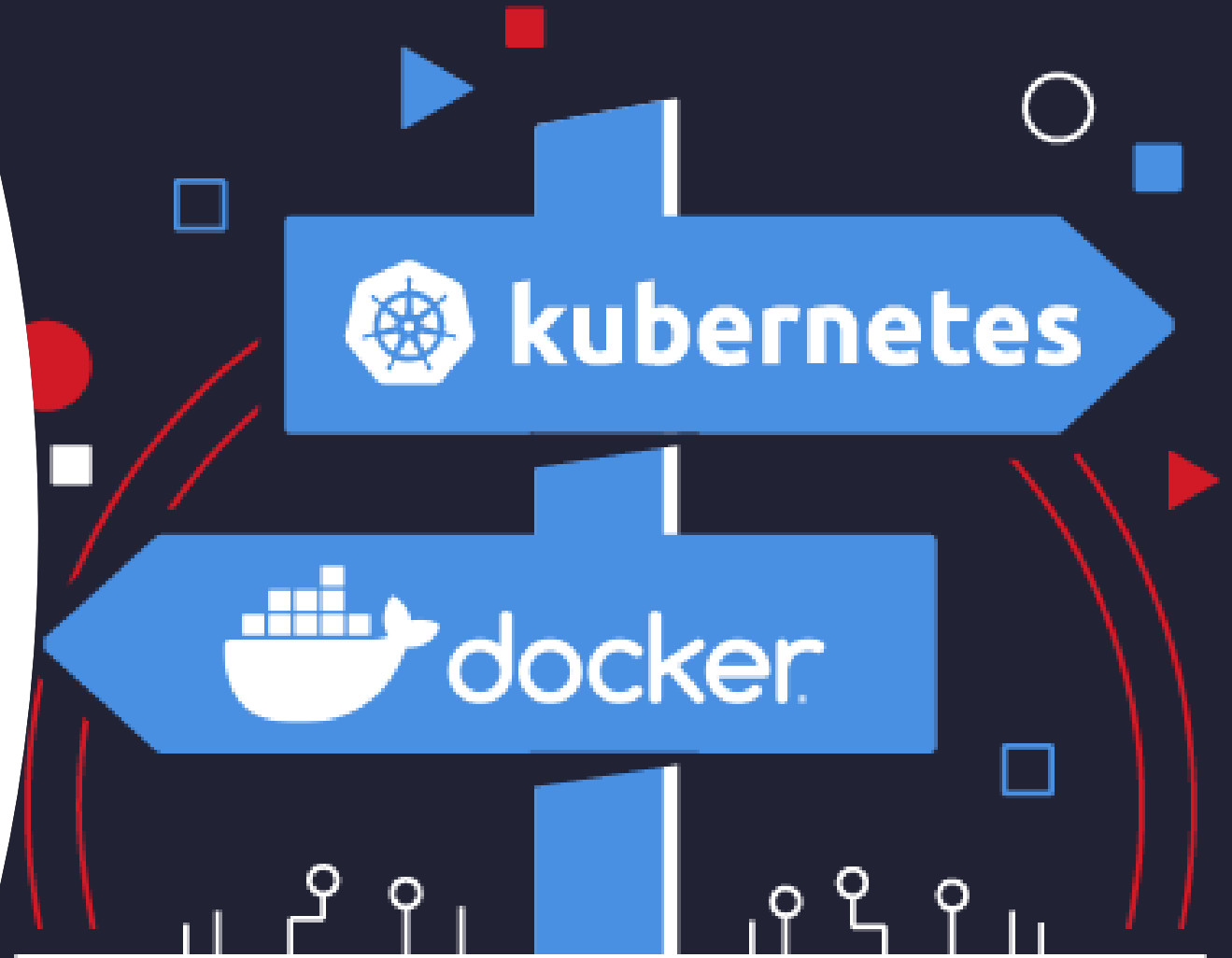


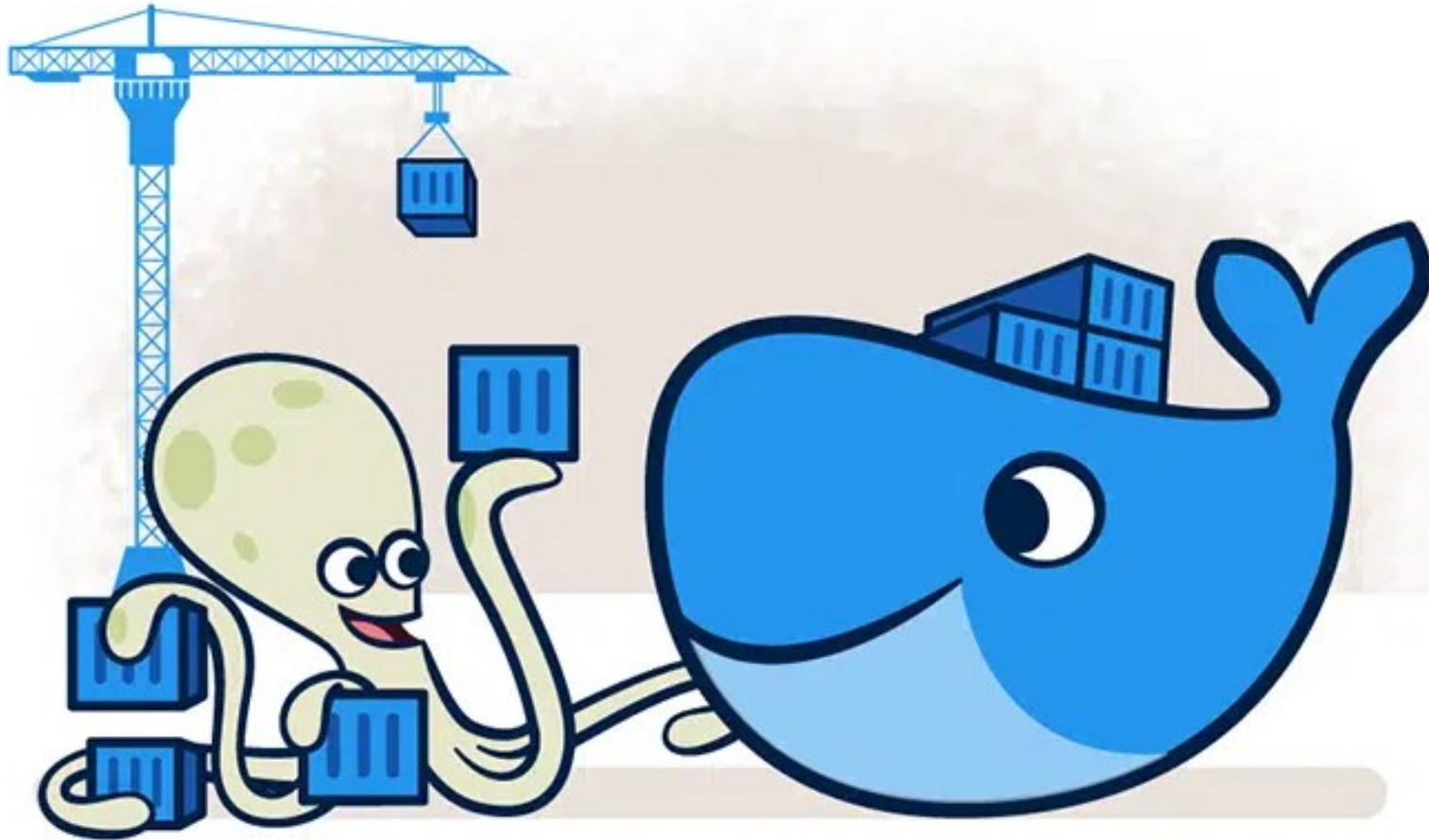
Blue is production/stable
New version is deployed to Green



NETZWERKKONZEPT FÜR MAXIMALE MÖGLICHKEITEN

A/B Deployment



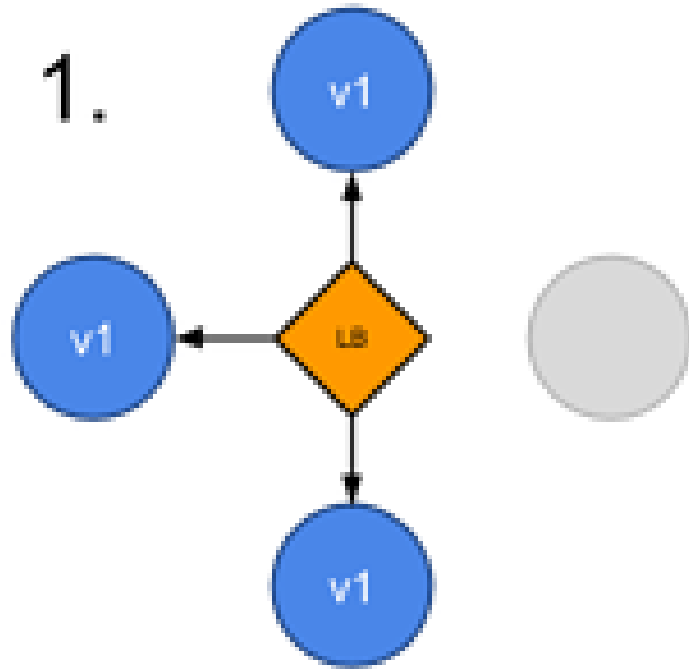


WAS IST SIND A/B DEPLOYMENTS

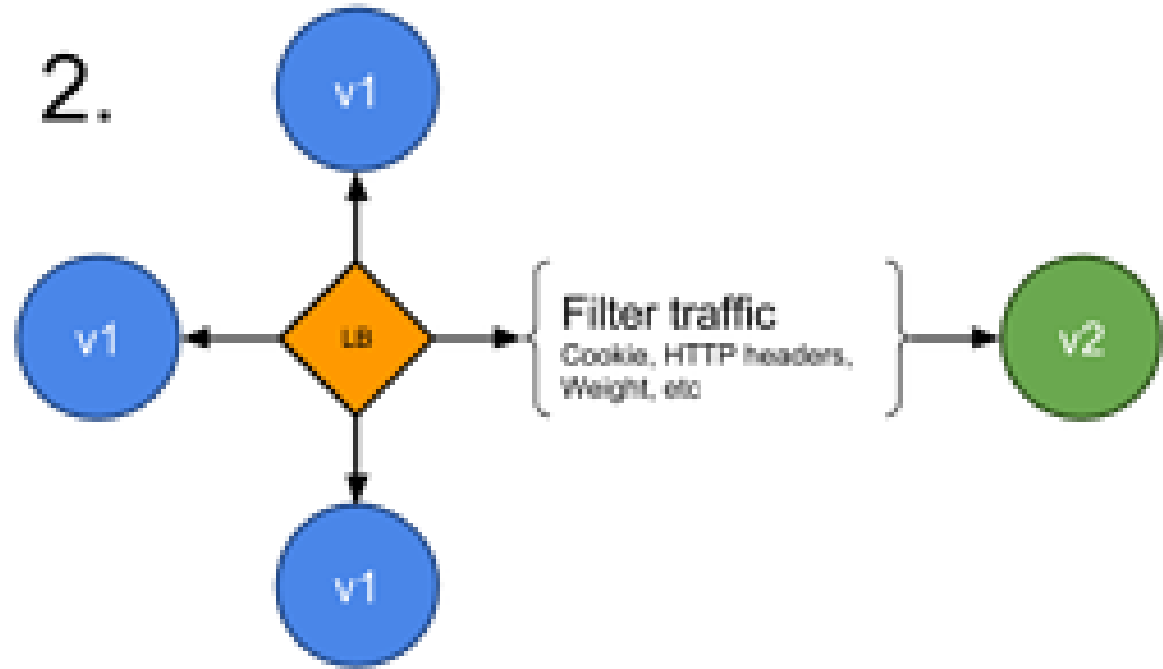
A/B testing is really a technique for making business decisions based on statistics, rather than a deployment strategy. However, it is related and can be implemented using a canary deployment so we will briefly discuss it here.

In addition to distributing traffic amongst versions based on weight, you can precisely target a given pool of users based on a few parameters (cookie, user agent, etc.). This technique is widely used to test conversion of a given feature and only rollout the version that converts the most.

1.



2.





KLAUSUR EINFÜHRUNG IN KUBERNETES TAG 2

Nun folgt eine kurze Wissensstand Überprüfung des heutigen Tages. Diese Klausur dient nur Ihrer eigenen Sicherheit in der Thematik. Sie ist auf ihren Wissenstand zugeschnitten und angepasst. Es ist davon auszugehen das Sie die Klausur in der genannten Zeit gerade schaffen werden.

■ KEINE HILFSMITTEL

Es sind weder technische noch nicht technische Hilfsmittel bei der Klausur gestattet. Bitte lösen Sie die Klausur ohne auf diese Mittel zu zugreifen.

■ KEINE GRUPPENARBEIT

Die Klausur wurde jedem Seminarteilnehmer ausgeteilt also bitte schauen Sie auf ihren eigenen Bogen und nicht auf den Bogen ihres Nachbarn.

ZEIT: 30 MINUTEN

IT-Schulungen.com

Technology & Transformation

Vielen Dank für Ihre Teilnahme!
Ihr Referent und das Team von IT-Schulungen.com

**Ben
Cöppicus**

IT-Schulungen.com

New Elements GmbH
Thurn-und-Taxis-Straße 10
D -90411 Nürnberg

Fon. +49 (0) 911 65 00 83 41
Fax. +49 (0) 911 65 00 83 99

Web. www.IT-Schulungen.com
Mail. info@it-schulungen.com

Ihr Digital Open Badge

Ab sofort stehen Ihre Zertifikate für Sie bereit!

In Ihrem persönlichen Bereich „Mein IT-Schulungen.com“ stehen als Nachweis für Ihren Fortbildungserfolg für Sie bereit:

- Ihr Zertifikat im PDF-Format
- Ihr persönlicher „Open Badge“ (Mehr Infos zu Badges)

Jetzt Zertifikat und Badge abholen!

Falls Sie noch keinen Account bei „Mein IT-Schulungen.com“ besitzen, ist eine Registrierung erforderlich. Benutzen Sie dabei unbedingt dieselbe E-Mail-Adresse, wie Sie an dieser Schulung angemeldet wurden und wählen Sie Ihr persönliches Passwort.

Falls dies nicht möglich sein sollte, kontaktieren sie uns unter
Telefon: 0911-65008-30 oder
Email: info@it-schulungen.com



QR CODE

AUFWIEDERSEHEN

BIS BALD